

有界示向误差下无线电干扰源的 覆盖认证搜索与快速清除

研究与复核稿：官方模拟器测试已按要求搁置，非可直接提交版本

摘要

无线电干扰源数量、接收半径以及定向源的发射方向均未知，因而快速定位不仅是多条示向线求交，还必须解决误差约束、全区域发现与正确停止三个问题。本文采用“确定性覆盖保证正确性、观测驱动调度降低耗时”的统一框架。

针对问题一，将每次示向度转为正向楔形半平面约束，给出空集、有界和无界情形的判别及顶点对直径算法。构造由合法测向观测实现的边长 36 米等边三角形反例，证明直径为区域直径的圆不一定覆盖区域；以最小包围圆半径代替区域直径作为一次清除判据。

针对问题二，利用首次接收到信号隐含的接收半径下界，推导无需已知目标距离的三圆交候选区域。以交会夹角与距离共同刻画测向条件，并以候选点的后验可行域包围圆半径进行采样极小极大评分，区分可证明的接收安全性与启发式的最优性。

针对问题三，建立七点全向覆盖与精确覆盖半径公式，以 1200 米收缩环保留 31.10 米接收余量；采用联合任务路线并停止对已知源的骨架重复扫描。针对问题四，重新构造含 12 个圆外点的 25 点错相双环，用 36 个闭三角形和辅助点平移证明任意发射方向下的严格覆盖：测点总位置误差不超过 1 米时，仍有检测点的前向投影至少 4 米、源距小于 999 米。相较原 31 点格点，完整频道检测义务上限由 620 降至 500。两问均以逐频道完成证书和有限光学后备保障正确停止，联合调度只优化效率，不代替证书。

固定策略后，在每问 1000 个全新合成世界上作同案例配对比较。问题三、四的平均单源总耗时分别为 261.46 秒和 544.74 秒，较已有增强策略分别降低 5.38% 和 31.21%；每问 1000 例均全清。由 10700 次独立于开发数据的运行完成消融、压力及敏感性检验，逐动作复算全部计时。问题四聚集源端点误差组平均反而增时 2.32%，不主张逐实例或全分布占优。结果仅为声明分布下的本地证据；官方正式测试和参赛队人工核验仍缺失。

关键词：有界误差；交会定位；最小包围圆；覆盖证书；主动测向；定向源搜索

1 问题理解、目标与证据边界

1.1 四问之间的依赖关系

题目给定半径 1800 米的目标圆域，最多 20 个频道、每频道至多一个静止干扰源；在问题三、四中目标总数为 10 至 16 但不向程序公开。每次检测只报告无信号、近源或带误差的示向度，没有场强数值、距离或源数接口 [1]。因此不能从信号强弱反推距离，不能把 20 个频道同时检测，也不能把“已发现列表清空”当成全清。

问题一回答怎样描述和计算定位不确定性；问题二回答下一次测向应走向何处；问题三在全向条件下把发现、定位、清除和调度结合；问题四进一步解决从发射背面完全看不到信号的困难。本文不采用需要训练真值标签的端到端学习，也不对未知目标先验进行未经说明的概率化。主模型只用题目给定的确定界，概率分布仅用于可复现的合成实验。

1.2 优化目标和两种时间

设总行程为 L ，检测次数为 M ，实际切换频道次数为 S ，失败清除次数为 F ，成功清除数为 K 。 N 表示该世界中隐藏的真实源总数，只在退出后的评价器中使用； K 表示已收到成功清除响应的频道数。由附件计时规则，虚拟任务时间为

$$T = \frac{L}{5} + 5M + S + 3F + 5K. \quad (1)$$

式中距离单位为米、时间单位为秒；成功清除已含 3 秒光学精确定位与 2 秒激光清除，不能另加一次 3 秒。/clear 中的频道只指定目标，不改变测向机频道。

优化是字典序的：先在模型与资源条件下保证全清，再减小 T 。题目评价量为清除率 K/N 和平均定位清除时间 T/K ($K > 0$)；所有列示合成运行满足 $K = N$ ，因此本文逐世界计算 $T/N = T/K$ 后再平均，不用所有时间之和除以所有源数之和。另记程序现实运行时间 T_{real} ；它不等于虚拟时间，也不包含为每次虚拟 5 秒检测实际休眠的要求。

由于目标状态未知且实际反馈尚未出现，不能预先给出一个对所有世界同时最短的固定路径。本文的覆盖和清除安全性有解析证明，调度效率则由同口径对照实验支持，不声称全局时间最优。

1.3 当前证据等级

原题和附件是已知条件来源，覆盖命题是解析推导，合成数据由本机执行产生；不得把这三类证据混为一谈。当前未取得官方演练、六次正式测试及加密日志，官方世界分布和 HTTP 开销也未经标定。本稿因此保留研究稿标识，不以合成全清或编译成功宣称已满足正式提交要求。

2 符号、假设和统一信息状态

表 1: 主要符号及来源

符号	含义	来源或单位
$\mathcal{D}$	$\{g : \ g\ \leq 1800\}$, 源所在区域	题目, 米
c, g_c	频道编号、该频道源坐标	$c \in \{1, \dots, 20\}$
R_c	源的有效接收半径	未知, $[1000, 1500]$ 米
n_c	定向源发射半平面的单位法向	未知, 固定
s_i, θ_i	第 i 次检测点及示向度	可观察量
α	示向度容差	理论 1° , 实现 1.005°
$\mathcal{P}_c$	包含真实源的保守位置多边形	在线维护
$D(\mathcal{P}), r_*(\mathcal{P})$	直径、最小包围圆半径	米
ρ_{opt}	规划使用的光学覆盖半径	19.9 米
$V_c \subseteq V$	已完成的骨架索引与完整骨架索引集	每频道单独记录

除题目条件外, 仅作以下实现约定:

1. 示向度接口保留两位小数。为兼容先加入 $\pm 1^\circ$ 误差再舍入的过程, 计算使用 1.005° 容差; 这是额外的保守余量, 不改变题面误差定义。
2. 以外切正 180 边形近似目标圆盘, 绝不使用内接近似删除边界可能源。该外切多边形的径向过估计为 $1800(\sec(\pi/180) - 1)$, 小于 0.275 米。
3. 为控制数值误差, 一次保证清除的计算半径设为 19.9 米, 小于设备允许的 20 米。几何半平面向外添加极小浮点余量, 约束矛盾时报告错误而不擅自丢弃旧观测。
4. 源静止、频道唯一、方向固定和未清除前持续发射均来自题意。不同地点误差的具体分布及独立性没有给定; 重复原地测向不得被当作独立样本平均降噪。

定向源的可检测条件写作

$$\|s - g_c\| \leq R_c, \quad n_c^\top (s - g_c) \geq 0. \quad (2)$$

全向源没有第二个条件。距离不超过 5 米且可检测时返回 `near`, 此时原地清除必定成功。光学清除只要求 $\|s - g_c\| \leq 20$, 与式(2)的方向条件无关。

完整隐藏状态可以表示为各频道的存在性、位置、类型、接收半径和方向。程序不读取这个隐藏状态, 而只维护位置多边形、正测向记录、失败清除记录、清除成功集合及逐频道骨架检测记录。这种保守的信息压缩可能放弃部分负观测信息, 但不会因此误删真实源。

3 问题一：交会区域直径与清除判据

3.1 从示向度到正向半平面交

定义 $u(\theta) = (\cos \theta, \sin \theta)$, $u_i^\pm = u(\theta_i \pm \alpha)$, 二维叉积为 $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。一次观测对应

$$W_i = \left\{ g : \begin{array}{l} \text{cross}(u_i^-, g - s_i) \geq 0, \\ \text{cross}(u_i^+, g - s_i) \leq 0, \\ u(\theta_i)^\top (g - s_i) \geq 0 \end{array} \right\}. \quad (3)$$

前两个半平面在 $\alpha > 0$ 时已经给出正向楔形, 第三个条件显式排除精确零误差情形下的背向射线。使用正弦、余弦表达使 359.9° 和 0.1° 之间的跨零问题无需分支处理。

原始交会区域为 $P = \bigcap_i W_i$ 。若题意需要同时利用已知目标区域或距离上限, 可再取交集; 但添加先验后的区域不是原始无限楔形交, 不能混淆二者的直径。

3.2 空集、无界集和有界多边形的算法

将全部约束写成 $Ag \leq b$ 。一个透明、易核查的实现如下。

1. 解二维可行性线性规划。无可行解时报告观测或条件矛盾, 不返回虚假的零直径。
2. 分别最大化及最小化 x, y 。任一坐标目标无界, 则非空区域无界, 直径为 $+\infty$; 四个坐标上下界都有限则区域有界。
3. 对每对非平行边界直线求交, 保留满足全部半平面约束的交点; 去除数值重复点, 按凸边界排序。
4. 求所有顶点对距离的最大值。单点直径为 0, 线段直径为两端点距离。

配套 `intersection.py` 直接完成上述分类, 不用任意大包围盒冒充原区域的有界性。若约束数为 m , 直线交点枚举并逐个检验的朴素复杂度为 $O(m^3)$; 通常只需少数测向, 足够迅速。大规模情形可改为按极角排序的半平面交和旋转卡壳, 几何阶段降至 $O(m \log m)$, 但这不是本题在线计算的瓶颈。

结论 1 (直径的顶点刻画). 若 $P = \text{conv}\{v_1, \dots, v_n\}$ 非空、有界, 则

$$D(P) = \max_{i,j} \|v_i - v_j\|. \quad (4)$$

证明. 任取 $p = \sum_i a_i v_i$ 与 $q = \sum_j b_j v_j$, 其中权重非负且各自和为 1, 有

$$\|p - q\| = \left\| \sum_{i,j} a_i b_j (v_i - v_j) \right\| \leq \sum_{i,j} a_i b_j \|v_i - v_j\| \leq \max_{i,j} \|v_i - v_j\|.$$

反向不等式由各顶点本身属于 P 得到。□

3.3 直径圆不能总是覆盖：一个真实可实现的反例

边长为 D 的等边三角形直径就是 D , 最小包围圆半径却是 $D/\sqrt{3} > D/2$ 。这不仅是任意多边形反例, 还能由本题 $\pm 1^\circ$ 测向直接生成。

取 $D = 36$, $A = (0, 0)$ 、 $B = (36, 0)$ 、 $C = (18, 18\sqrt{3})$ 。依次对三条逆时针有向边取单位方向 e_j , 令

$$S_j = V_j - L e_j, \quad L = 18\sqrt{3} \cot 2^\circ - 18. \quad (5)$$

测向读数取边方向加 1° , 即 $1^\circ, 121^\circ, 241^\circ$ 。每个楔形下边界沿对应边、上边界经过对角顶点；三个楔形都包含该三角形, 而三条下边界的内半平面交恰为该三角形。所有三角形点距对应检测站小于 1000 米, 故可取合法接收半径 1000 米。

实际计算得到直径约 36.000000 米, 最小包围圆半径约 20.784610 米。于是即使 $D < 40$, 仍不一定能在某一点以 20 米光学半径保证一次清除。

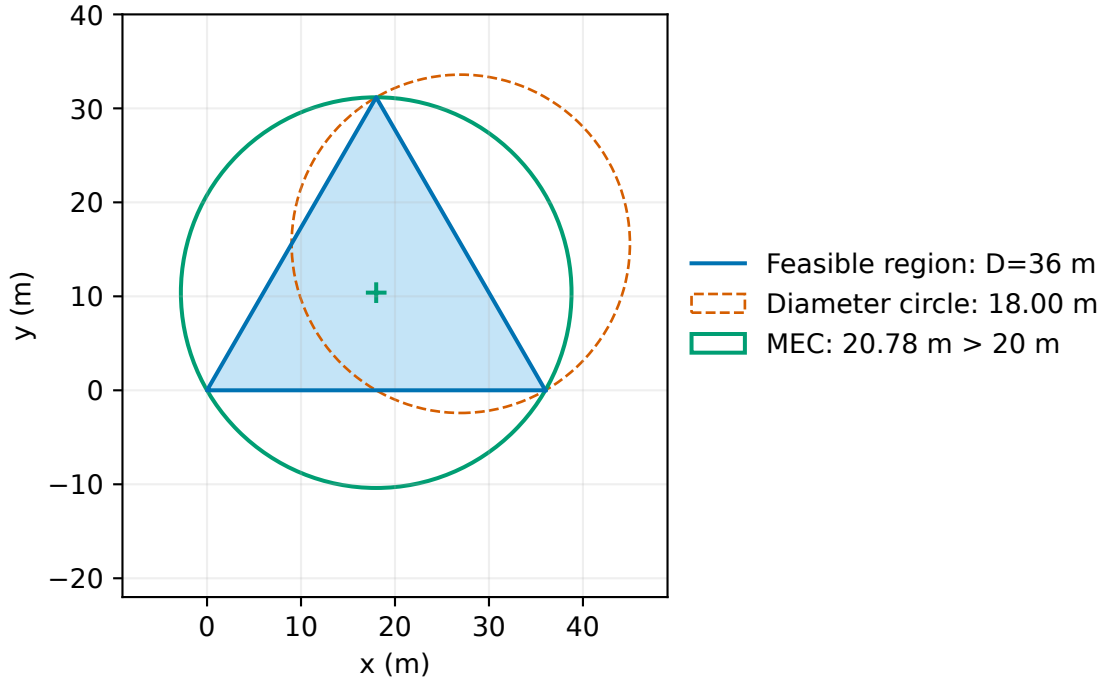


图 1: 直径为 36 米的可实现三角形反例。任何半径 18 米的圆都无法覆盖三顶点；正确包围半径约 20.785 米。

3.4 用最小包围圆直接连接定位与清除

定义

$$r_*(P) = \min_z \max_{p \in P} \|p - z\|, \quad z_*(P) = \arg \min_z \max_{p \in P} \|p - z\|. \quad (6)$$

圆盘是凸集, 包含多边形所有顶点即包含整个多边形。最小包围圆由一个、两个或三个边界支持点确定, 可用随机增量支持点算法求解 [2]。本文实现对近共线三点重算高精度行列式, 并在最终半径上保留向外余量。

平面 Jung 界为

$$\frac{D(P)}{2} \leq r_*(P) \leq \frac{D(P)}{\sqrt{3}}. \quad (7)$$

两点支持时左端取等；三点支持时圆心在支持三角形内, 三段圆心角中最大的一段至少 120° , 对应弦长至少 $\sqrt{3}r_*$, 得到右端。等边三角形取右端等号。因此:

- $r_*(P) \leq 20$ 是存在一次保证清除点的充要几何条件；执行点取 $z_*(P)$ 。
- 仅使用直径时， $D(P) \leq 20\sqrt{3}$ 是充分条件， $D(P) > 40$ 则必不可能一次保证覆盖。
- $20\sqrt{3} < D(P) \leq 40$ 时必须进一步算包围圆，不能凭直径直接清除。

3.5 在线使用的保守多边形

在问题三、四中，每次有效示向度还说明 $\|g - s_i\| \leq 1500$ 。为保持纯线性裁剪，可使用安全松弛

$$u(\theta_i)^\top (g - s_i) \leq 1500. \quad (8)$$

这不是把接收半径错误设为 1500，而是利用其已知上界。若初始外切多边形为 A_{180} ，则维护

$$\mathcal{P}_c = A_{180} \cap \bigcap_{i \in I_c} (W_i \cap \{g : u(\theta_i)^\top (g - s_i) \leq 1500\}). \quad (9)$$

真实源初始在 A_{180} 内，每条新增约束也包含它，故归纳得到 $g_c \in \mathcal{P}_c$ 。一次观测裁剪四个半平面， m 次观测最多形成 $180 + 4m$ 条边，复杂度 $O(180m + m^2)$ 。空交集是需要定位原因的异常，而不是允许重置状态的信号。

4 问题二：第二检测点的安全候选区域与选点策略

4.1 单次测向为什么没有唯一最优第二点

只有角度而没有距离时，目标位于长而窄的扇形中。若第二点沿首次示向方向移动，两条示向线可能几乎平行或反平行，径向误差难以压缩。若仅为了接近 90° 交会而绕行很远，则会增加移动时间，未必有利于题目总目标。

令 $r_i = \|g - s_i\|$ 、 $u_i = (g - s_i)/r_i$ 、 $n_i = (-u_{iy}, u_{ix})$ 。对小误差线性化，有

$$|n_i^\top \delta g| \leq \alpha r_i. \quad (10)$$

两条误差带交形成的平行四边形面积近似为

$$A \approx \frac{4\alpha^2 r_1 r_2}{|\sin \beta|}, \quad (11)$$

其中 β 为两条真实视线夹角， α 以弧度计。固定距离时， 90° 交会最有利；距离也变化时，应同时考虑缩短 r_2 、避免共线及移动代价。该式是局部解释，不能代替精确楔形裁剪。

若额外采用独立高斯角误差的辅助似然，才可写出 Fisher 信息 $J = \sigma^{-2} \sum_i n_i n_i^\top / r_i^2$ 及 $\det J = \sin^2 \beta / (\sigma^4 r_1^2 r_2^2)$ 。没有这种似然假设时，这至多是加权最小二乘的局部信息代理，不能当成题目提供的真实误差统计模型。最终安全性仍由式(9)保证。

4.2 利用首次已接收事实推导三圆交

取第一次检测点 S_1 为局部原点，首次读数方向为 u ，法向为 n ，第二点写为

$$S_2 = S_1 + tu + bn. \quad (12)$$

若可能真实距离为 r ，首次已经接收说明

$$R \geq \max(1000, r), \quad (13)$$

而不仅仅是 $R \geq 1000$ 。设 F_1 为当前全部可能源位置，则保证全向源在第二点仍有信号或返回近源的精确鲁棒条件为

$$S_2 \in \bigcap_{g \in F_1} B(g, \max(1000, \|g - S_1\|)). \quad (14)$$

用完整扇形 $r \in [0, 1500]$ 、角差 $\varphi \in [-\alpha, \alpha]$ 覆盖真实 F_1 ，可得闭式充分候选域。

结论 2 (三圆交接收保证). 令 $u_{\pm} = u(\theta_1 \pm \alpha)$ 。以下候选域内的任一点，对整个放宽扇形都能保证全向接收：

$$\mathcal{C} = S_1 + [B(0, 1000) \cap B(1000u_-, 1000) \cap B(1000u_+, 1000)]. \quad (15)$$

对这一完整扇形，式(15)也恰好是式(14)的交集；若再利用圆域等更强先验，它可能不是最大的安全候选域。

证明. 写 $q = S_2 - S_1$ 。必要性由 $r = 0$ 和 $r = 1000, \varphi = \pm\alpha$ 给出。反之，两端圆约束意味着 $q^T u_{\pm} \geq \|q\|^2 / 2000 \geq 0$ 。小于 180° 的中间短弧上， $q^T u_{\varphi}$ 的最小值在端点，因此对所有允许角度都满足同样下界。若 $r \leq 1000$ ，距离 $\|q - ru_{\varphi}\|$ 关于 r 是凸函数，两端 $r = 0, 1000$ 均不超过 1000，内部也不超过 1000；若 $r \geq 1000$ ，

$$\|q - ru_{\varphi}\|^2 - r^2 = \|q\|^2 - 2rq^T u_{\varphi} \leq 0.$$

故距离不超过 $\max(1000, r)$ ，由首次接收所隐含的半径下界即可保证可检测。 $\square$

4.3 显式候选区域与可执行搜索

在 (t, b) 局部坐标中，三圆交等价于

$$\begin{cases} t^2 + b^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b - 1000 \sin \alpha)^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b + 1000 \sin \alpha)^2 \leq 1000^2. \end{cases} \quad (16)$$

令

$$b_{\max}(t) = \min \left\{ \sqrt{1000^2 - t^2}, \sqrt{1000^2 - (t - 1000 \cos \alpha)^2} - 1000 \sin \alpha \right\}. \quad (17)$$

根号有定义且 $b_{\max} \geq 0$ 时， $0 < t < 1000$ 、 $0 < |b| \leq b_{\max}(t)$ 形成首次方位两侧的横向候选区。排除 $b = 0$ 用于避免把共线点作为唯一方案，而非禁止为了接近目标沿轴行进。

程序按下式构造 42 个候选：

$$\begin{aligned} t &\in \{125, 250, 375, 500, 625, 750, 875\} \text{ 米}, \\ b &= \pm \kappa b_{\max}(t), \quad \kappa \in \{1/4, 1/2, 3/4\}. \end{aligned} \quad (18)$$

对每个候选模拟允许的第二读数，计算新多边形最小包围圆；优先选择采样最坏半径较小的点，可再以本步移动和检测代价打破并列。所有这些点的接收安全性来自定理2，不是来自采样。

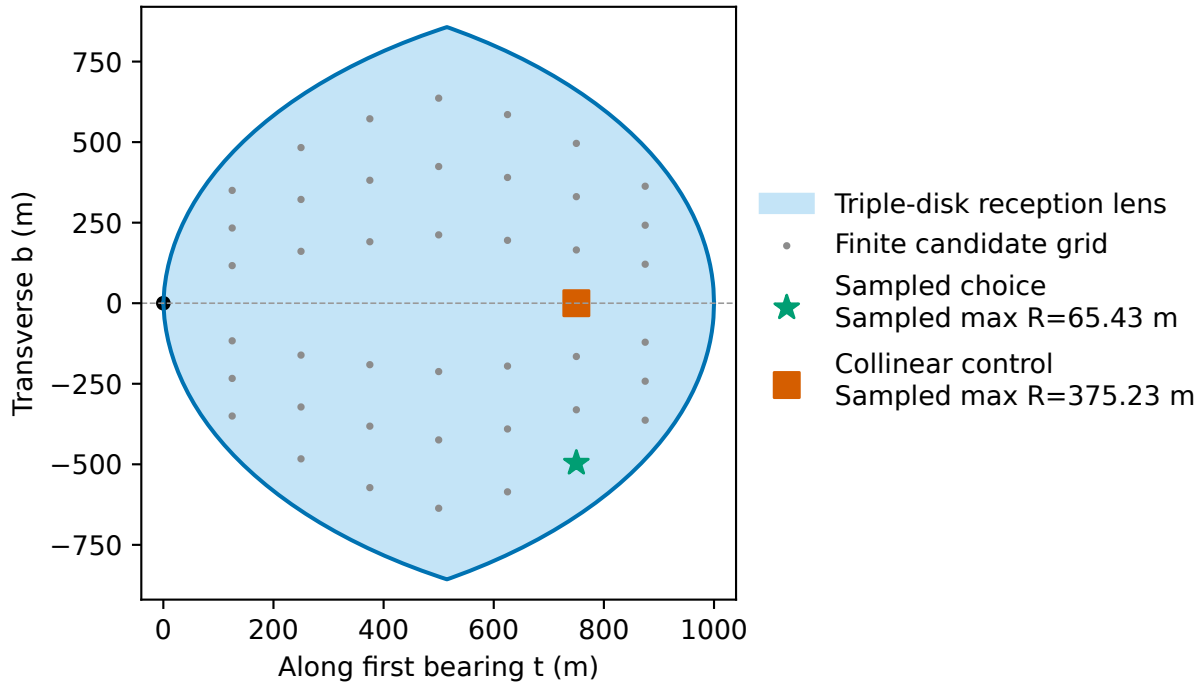


图 2: 第二点的三圆交候选区域。横向基线改善交会条件，但候选排序仍需兼顾移动耗时和未知径向距离。

在 $S_1 = (0, 0), \theta_1 = 0$ 的归一化示例中，取 60 个径向位置、3 个首次真实角差和 3 个第二次测向误差进行比较。程序得到采样最佳几何候选约为 (750, -496.078) 米，采样最坏包围半径约 65.426 米；共线对照 (750, 0) 的对应值约 375.231 米。两者本步耗时分别约 184.844 秒和 155 秒。这说明增加约 29.844 秒移动检测成本可显著改善此例的径向辨识，但第二次测向后仍未必能一次清除。

该示例是候选域与评分算法的演示，不是题目给定正式案例，也不是连续最坏情形或总时间的全局最优解。二维方位误差的连续极值、未知距离分布及后续代价若未严格处理，不能将网格最小值写成“最优定位策略”。

5 局部定位：快速测向与有限光学兜底

5.1 正常测向路径

针对已确认存在的频道，反复计算 (z_*, r_*) ：

1. 当前点到全部多边形顶点距离都不超过 19.9 米时，直接原地保证清除；否则若 $r_* \leq 19.9$ ，移动到 z_* 保证清除。
2. 若 $r_* \leq 38$ 米，可先在 z_* 进行一次光学尝试。失败仅花 3 秒且记录为失败，不将其写成保证成功；随后使用全区域覆盖，避免为一个很小区域继续绕远测向。

3. 设最近一次正测向点为 s^+ ，从 s^+ 指向 z_* 的单位方向为 u 、法向为 n ，候选测点取 $z_* \pm \delta n$ ，其中 $\delta = \min(120, \max(25, 0.22r_*))$ 米。优先较近且未重复正读数的位置。
4. 对实际返回的正示向度继续裁剪；`near` 立即原地清除。每次局部定位最多使用 6 个新测向点，随后转入光学覆盖。

这里的 25、120、38、0.22 和 6 是公开的工程策略参数，而非从题面推导的自然常数。它们影响效率，不影响后备覆盖的正确性。问题二给出的是保证全向接收的完整候选域；本节采用更直接的中心横向点以降低重复评分开销，**没有声称每个在线测点都属于三圆交或总能保持信号**，而是对失去信号设置了可靠处理。

5.2 定向源丢信号后的有限恢复

问题四中，新的点可能越过发射半平面的边界。设失败点相对最近正测向点的分解为 $q^- - s^+ = au + bn$ ，下一点建议取

$$q^{\text{next}} = s^+ + \frac{1}{2}(au - bn). \quad (19)$$

其含义是向已知能接收的位置退回，同时尝试另一侧。新正测向可收缩位置多边形并更新 s^+ ；持续无信号则继续有限恢复，直到达到 6 次测向上限。

式(19)只是利用正负观测进行选点的启发式，不据此删除任何可能源位置，也不假称一定重获信号。它的价值通过最低接收半径、朝外边界源和聚集源的消融实验评价；最终清除保证不依赖恢复成功。

5.3 覆盖整个可行多边形，而非只覆盖顶点

若无法继续有效测向，取多边形的一条直径方向为局部 x 轴，将 P 包含在投影矩形中。把矩形划分为边长分别不超过 $\sqrt{2}\rho_{\text{opt}}$ 的网格单元，在每个单元中心放置半径 $\rho_{\text{opt}} = 19.9$ 米的光学清除圆盘。每个单元半对角线不超过 ρ_{opt} ，故

$$P \subseteq \bigcup_{j=1}^J B(q_j, \rho_{\text{opt}}). \quad (20)$$

注意多个圆盘的并集一般不凸，覆盖所有顶点并不能推出覆盖内部；本文覆盖的是整个外包矩形，因此不存在内部孔洞。

按蛇形访问这些中心，可把起点循环移动到距离当前位置最近的中心；这样最多增加一次跨行程的长跳。每个失败清除只排除该清除圆盘里的目标可能性，不把剩余非凸集合强行当成一个更小的凸多边形。实际源始终在 P 内，故尝试完全部中心之前必有一次成功，随后立即停止该频道的搜索。

6 问题三：全向七点覆盖与环半径收缩

6.1 从 900 米覆盖基线到可调七点构型

记源区域为 $\mathcal{D} = \{g : \|g\| \leq R\}$ ，其中 $R = 1800$ 米，接收半径的题给下界为 1000 米。全向发现只需保证每个可能源位置至少落入一个检测点的接收圆盘；但覆盖半径越小，并不意味着式(1)中的总时间越少。为分离覆盖安全性与行程代价，考虑原点加正六边形环的同一构型族：

$$p_0 = (0, 0), \quad p_{k+1} = d u(k\pi/3), \quad k = 0, \dots, 5, \quad u(\theta) = (\cos \theta, \sin \theta). \quad (21)$$

原始保守基线取 $d = 900\sqrt{3}$ ，最终策略取 $d = 1200$ 。两者均保留七个检测位置，变化的是完整覆盖所需的圆盘半径，而不是逐频道排查的逻辑。

结论 3 (七点构型的精确覆盖半径). 对 $1200 \leq d \leq 900\sqrt{3}$ ，式(21)在 $\mathcal{D}$ 上的精确覆盖半径为

$$\rho(d) = \max_{g \in \mathcal{D}} \min_{0 \leq j \leq 6} \|g - p_j\| = \max \left\{ \frac{d}{\sqrt{3}}, \sqrt{R^2 + d^2 - \sqrt{3}Rd} \right\}. \quad (22)$$

特别地， $\rho(900\sqrt{3}) = 900$ ，故原 900 米圆盘覆盖基线成立。

证明. 将源写为 $g = r u(\theta)$ 。总有一个环点与之极角相差不超过 $\pi/6$ ；固定 r 时，相邻环点的角平分线给出到最近环点的最坏距离。因此该圆周上到最近检测点的最大距离恰为

$$f_d(r) = \min \left\{ r, \sqrt{r^2 + d^2 - \sqrt{3}dr} \right\}. \quad (23)$$

两项在 $r = d/\sqrt{3}$ 相等，此前原点更近，此后环点更近。在 $[d/\sqrt{3}, R]$ 上，后一项的平方是凸二次函数，其最大值位于两个端点；在 $[0, d/\sqrt{3}]$ 上， $f_d(r) = r$ 。于是全域最大值为式(22)。两端值均在角平分线上实际达到，故该式不是仅有充分性的上界，也不依赖空间网格采样。代入 $d = 900\sqrt{3}$ 后两项均为 900 米，证明包含圆周边界和扇区接缝。□

由式(22)，给定安全覆盖半径 $q \geq R/2$ 时，需要同时满足

$$d \leq \sqrt{3}q, \quad \frac{\sqrt{3}R - \sqrt{4q^2 - R^2}}{2} \leq d \leq \frac{\sqrt{3}R + \sqrt{4q^2 - R^2}}{2}. \quad (24)$$

当 $q = 900$ 时，这些条件将环半径固定为 $900\sqrt{3}$ ；允许覆盖半径向物理下界 1000 米靠近，则可以向内收缩检测环。最终采用 $d = 1200$ 米，解析式给出

$$\begin{aligned} \rho(1200) &= \sqrt{1800^2 + 1200^2 - \sqrt{3} \cdot 1800 \cdot 1200} \approx 968.901572 \text{ m}, \\ 1000 - \rho(1200) &\approx 31.098428 \text{ m}. \end{aligned} \quad (25)$$

因而收缩后仍保留约 31.098 米距离余量。若理想检测点到实际检测位置的总欧氏偏差不超过 η ，三角不等式给出实际覆盖半径至多为 $\rho(1200) + \eta$ ；例如 $\eta \leq 1$ 米时仍严格小于 1000 米。这里的余量属于全局发现骨架，与示向度误差和局部清除半径是不同的量。

6.2 骨架路径代价与七点下界的适用范围

从原点出发、访问全部六个环点而不要求返回时，纯骨架路径的精确最短长度为

$$L_{\text{ring}}^{\min}(d) = 6d. \quad (26)$$

起点到首个环点必须走 d ；任意两个不同环点的距离至少为相邻点弦长 d ，故其后五次转移各至少走 d 。沿圆周依次访问即可达到该下界。插入额外中间停靠点不能突破这一界，因为折线路径不短于对应直线。因此，环收缩使完整纯骨架路线由 $5400\sqrt{3}$ 米变为 7200 米。在所用参数区间内， $\rho(d)$ 由边界项控制，且

$$\rho'(d) = \frac{d - \sqrt{3}R/2}{\sqrt{R^2 + d^2 - \sqrt{3}Rd}} \leq 0, \quad \frac{dL_{\text{ring}}^{\min}}{dd} = 6.$$

这说明扩大环半径能改善最坏覆盖距离，却增加骨架行程；收缩则作相反交换。式(26)不包含定位支线、检测与切换费用，也不适用于达到 16 源上界后省略余点的路线，故其差额不能直接充当每局实际节省时间。

结论 4 (900 米覆盖要求下至少需要七点). 用半径 900 米的闭圆盘覆盖半径 1800 米的整个圆域，至少需要七个圆盘；式(21)在 $d = 900\sqrt{3}$ 时达到此下界。

证明. 设一个检测圆盘的圆心距原点为 $t > 0$ ，与源圆周的交弧角半宽为 β 。有交弧时余弦定理给出

$$\cos \beta = \frac{R^2 + t^2 - q^2}{2Rt} = \frac{1}{2R} \left(t + \frac{R^2 - q^2}{t} \right) \geq \frac{\sqrt{R^2 - q^2}}{R}, \quad q = 900.$$

所以 $\beta \leq \arcsin(q/R) = \pi/6$ ，等号仅在 $t = \sqrt{R^2 - q^2} = 900\sqrt{3}$ 时成立。圆心为原点的 900 米圆盘不与源圆周相交。六个圆盘若覆盖整条圆周，六条交弧都必须达到最大角宽 $\pi/3$ ，且不能有正长度重叠。于是六个圆心均在半径 $900\sqrt{3}$ 的环上，按 $\pi/3$ 间隔排列；然而它们距原点均大于 900 米，不能覆盖原点。六个及更少圆盘均不可能覆盖全域，结合前述七点构造即得结论。 $\square$

这一精确下界只针对 900 米覆盖要求。将允许半径改为 1000 米后，最大交弧变宽，上述等号论证不再成立；因此本文没有证明物理接收半径下任意布局仍至少需要七点，也不把 $d = 1200$ 称为连续参数空间的全局最优值。最终采用的七点环是在完整发现保证下减少骨架移动的明确构造，其任务时间表现另由冻结策略的实验评价。

7 问题四：具有严格方向余量的 25 点双环覆盖

7.1 由近邻覆盖转向方向覆盖

定向源不仅要求检测点距源不超过接收半径，还要求检测点位于发射前方。设源的单位发射法向为 n ，前方闭半平面为 $n^\top(p - g) \geq 0$ 。边界源若朝圆外发射，圆内近邻点可能全部位于背面，因此问题三的距离覆盖不能直接用于问题四。题目允许提交源圆域外的机器人位置，本文利用这一条件重新设计骨架，而不把外部检测点投影回圆内。

历史 enhanced 对照保留边长 990 米的 31 点三角格，配合原有滚动骨架排序、分离式调度和全部未清除频道扫描。该格点集合的逐点不可删性，说明从固定的 31 点集合直接删除一个点可能失去覆盖见证；它并不意味着任意布局都至少需要 31 点。最终 refined 采用下面独立证明的 25 点双环，其改进来自重新布点而不是未经证明的删点。

取 $a = 940$ 米、 $b = 1870$ 米，定义原点、内环和外环为

$$\begin{aligned} p_0 &= O = (0, 0), \\ p_{1+k} &= I_k = a u(k\pi/6), \\ p_{13+k} &= E_k = b u(k\pi/6 + \pi/12), \quad k = 0, \dots, 11. \end{aligned} \quad (27)$$

内环角间隔为 30° ，外环相对内环错开 15° 。共有 13 个点在源圆域内，12 个点在圆域外。索引只标识覆盖义务，不规定访问顺序。

7.2 36 个闭三角形组成的有限覆盖证书

令所有下标按模 12 计算，构造三类闭三角形

$$\begin{aligned} T_k^{(1)} &= \text{conv}\{O, I_k, I_{k+1}\}, \\ T_k^{(2)} &= \text{conv}\{I_k, E_k, I_{k+1}\}, \\ T_k^{(3)} &= \text{conv}\{I_{k+1}, E_k, E_{k+1}\}. \end{aligned} \quad (28)$$

这 36 个三角形完整剖分外正十二边形 $H = \text{conv}\{E_0, \dots, E_{11}\}$ 。具体而言，第一类剖分内正十二边形；内外连接线分别位于端点极角确定的 15° 扇区，不同扇区内部互不相交。外端点位于相应内边支撑线之外，而 $a < b \cos(\pi/12)$ 保证内点严格位于相应外边内侧。因此后两类恰好填满内外多边形之间的环带，不产生交叉边或孔洞。所有三角形按式中顺序逆时针定向，内部边成对反向出现，余下边界正是外十二边形；25 个顶点、60 条无向边与 36 个面还满足 $25 - 60 + 36 = 1$ 。边计数是有限结构的核对，而完整性依赖上述无交叉剖分。

记 $c = \cos(\pi/12)$ 、 $s = \sin(\pi/12)$ 。剖分中只有四种边长：

$$\begin{aligned} \ell_0 &= a, \quad \ell_I = 2as, \\ \ell_E &= 2bs, \quad \ell_{IE} = \sqrt{a^2 + b^2 - 2abc}. \end{aligned} \quad (29)$$

任一三角形的直径等于其最长边，故只需给出这四种边长的共同上界。利用精确根式

$$c = \frac{\sqrt{6} + \sqrt{2}}{4}, \quad s = \frac{\sqrt{6} - \sqrt{2}}{4},$$

以及可由平方比较得到的 $1.4142 < \sqrt{2} < 1.4143$ 、 $2.4494 < \sqrt{6} < 2.4495$ ，可得有理界

$$c > \frac{9659}{10000}, \quad \frac{1}{4} < s < \frac{259}{1000}. \quad (30)$$

以下比较均使用有限有理数，不依赖浮点近似决定不等号方向：

$$\begin{aligned}
bc &> 1870 \cdot \frac{9659}{10000} = 1806.233 > 1805, \\
\ell_0 &= 940 < 993, \quad \ell_I < 486.92 < 993, \\
\ell_E &< 968.66 < 993, \\
\ell_{IE}^2 &< 940^2 + 1870^2 - 2 \cdot 940 \cdot 1870 \cdot \frac{9659}{10000} \\
&= 984781.96 < 993^2 = 986049.
\end{aligned} \tag{31}$$

因此每个闭三角形的直径 $D_T < 993$ 米，而外十二边形的内切圆半径 $bc > 1805$ 米，特别有 $B(O, 1805) \subset H$ 。这两个严格界分别控制接收距离与源边界之外的辅助空间。

结论 5 (25 点的任意方向鲁棒覆盖). 对任意 $g \in \mathcal{D}$ 、任意单位发射法向 n ，式(27)中至少有一个点 v ，使得其实际检测位置 v' 在满足 $\|v' - v\| \leq 1$ 米时仍有

$$n^\top(v' - g) \geq 4 \text{ m}, \quad \|v' - g\| < 999 \text{ m}. \tag{32}$$

故该点严格位于发射前方，且严格处于题给最小接收半径 1000 米以内。

证明. 取仅用于证明的辅助点 $h = g + 5n$ 。由 $\|h\| \leq 1805$ 及式(31)，有 $h \in H$ ，故 h 属于剖分中的某个闭三角形 T 。写成重心组合

$$h = \sum_{i=1}^3 \lambda_i v_i, \quad \lambda_i \geq 0, \quad \sum_{i=1}^3 \lambda_i = 1.$$

投影满足

$$\sum_{i=1}^3 \lambda_i n^\top(v_i - g) = n^\top(h - g) = 5. \tag{33}$$

所以至少一个正权重顶点 v 满足 $n^\top(v - g) \geq 5$ 。又因 $v, h \in T$,

$$\|v - g\| \leq \|v - h\| + \|h - g\| \leq D_T + 5 < 998.$$

加入不超过 1 米的总位置偏差后，由柯西不等式与三角不等式得

$$n^\top(v' - g) \geq 5 - \|v' - v\| \geq 4, \quad \|v' - g\| \leq \|v - g\| + \|v' - v\| < 999.$$

这就证明了式(32)。 □

上述证明不要求源 g 与辅助点 h 位于同一个三角形，也不需要在线知道 g 或 n 。源在圆周上并朝外发射、源在圆心或内环点、辅助点恰落在三角形边或顶点等情况均包含在闭三角形论证中。正投影下界还排除了所选点与源重合的退化情形，不需依赖零距离方位定义或恰好落在发射边界上的浮点判断。1 米是理想点到实际测量位置的总欧氏误差预算，包括坐标计算、传输舍入和实际位置偏差之和；不能对每一种误差分别重复使用 1 米。仅将两个坐标四舍五入到整米所造成的误差至多 $\sqrt{1/2}$ 米，但仍须计入其余误差。

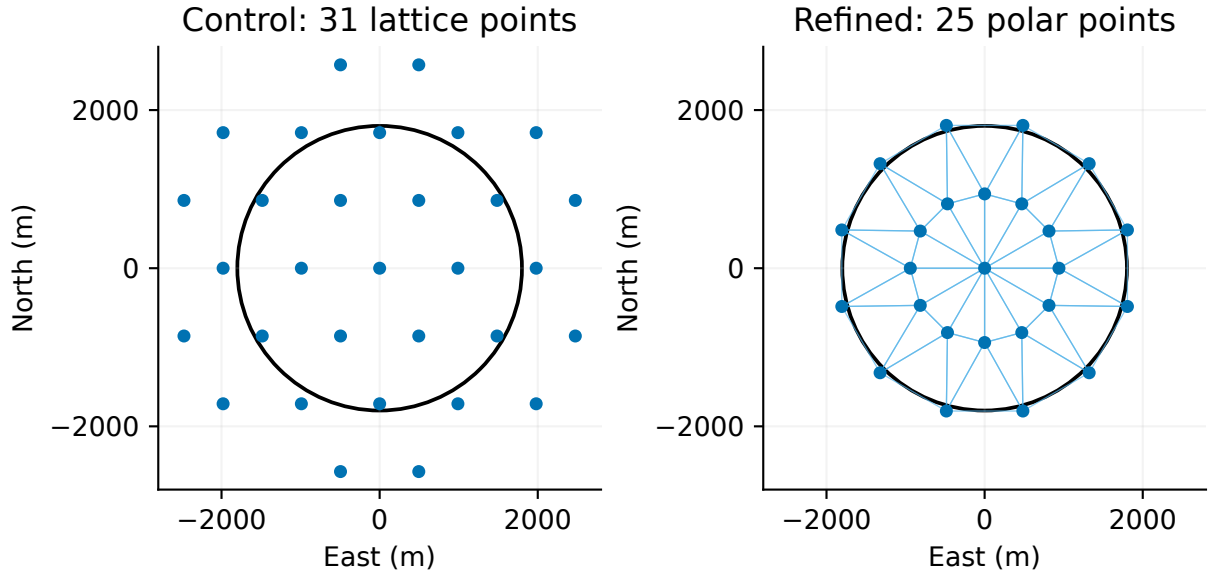


图 3: 问题四发现骨架的同尺度比较: 左为历史 31 点三角格, 右为最终 25 点错相双环及 36 个闭三角形。源区域均为半径 1800 米圆盘; 右图外十二边形包含辅助半径 1805 米圆盘, 圆外点是方向覆盖义务的一部分。

7.3 可实现路线、有限核验与点数结论

双环给出一条显式开放扫描路线

$$O, I_0, I_1, \dots, I_{11}, E_{11}, E_{10}, \dots, E_0,$$

其长度为

$$L_{25} = a + 22(a + b) \sin(\pi/12) + \sqrt{a^2 + b^2 - 2ab \cos(\pi/12)}. \quad (34)$$

它恰访问每个点一次, 不要求回到原点。保存的独立研究核验给出最大边长约 992.316061 米、该显式路线长度约 17932.509429 米, 并核对了 36 个三角形、60 条边及无交叉完整剖分。这些数值与实现检查补充解析证书, 不替代定理中的全称证明; 式(34)也只是一条可行路线, 不是旅行商最优值或联合调度的实际路径预测。

在 20 个频道都需完成全骨架扫描的义务计数下, 31 点对应 $31 \times 20 = 620$ 次检测, 25 点对应 500 次, 少 120 次检测, 即式(1)的检测项少 600 秒。实际任务会提前清除频道并跳过部分已知源补测, 还可能增加定位分支; 因此 600 秒不是每局固定节省, 路线和切换收益也须独立核算。另一方面, 本文没有证明 25 点是任意非规则布局、任意层数或任意方向覆盖方法中的全局最少点数。可成立的结论是: 31 点固定格不可删与 25 点重新设计可覆盖并不矛盾, 后者提供了点数更少且留有明确扰动余量的完整发现骨架。

8 联合调度、扫描筛选与逐频道停止证书

8.1 在共同任务路线中选择下一项

分开评价一个待扫描骨架点或一个已知源，会忽略执行当前任务后与其他任务的位置关系。最终策略将二者放入同一个开放路线：令当前位置为 q ，每个未完成骨架任务的代理点为其检测坐标、半径为 0；每个已发现未清除频道 c 的代理点为保守区域 P_c 的最小包围圆中心 z_c ，半径为 r_c 。对全部 n 个任务的排列 π 定义

$$J_\gamma(\pi) = \|q - z_{\pi_1}\| + \sum_{j=1}^{n-1} \|z_{\pi_{j+1}} - z_{\pi_j}\| + \gamma r_{\pi_1}. \quad (35)$$

问题三取 $\gamma = 0$ ，对应 **center**；问题四取 $\gamma = 1$ ，对应 **guarded**。后者只在首项为目标时增加其半径，抑制立即向一个不确定中心作长距离承诺；半径不是障碍半径，也不是未来真实任务时间的误差界。该代理不含未知的后续示向、光学尝试和频道切换，最终优劣仍以式(1)评价。

实现先用最近插入法构造完整排列：首项最小化当前位置距离加首项惩罚，此后选离已有路线最近的未插入任务，将其放入代理增量最小的空隙。再进行至多 n 轮 2-opt 扫描，每轮按固定次序检查连续区间反转，只接受严格改善的首个交换；没有改善即停止。开放路线不增加返回起点的虚拟边，反转从首项开始时还须计入半径惩罚的变化。距离矩阵占 $O(n^2)$ 空间，最近插入耗时 $O(n^2)$ ，每轮 2-opt 检查 $O(n^2)$ 个区间，总时间上界 $O(n^3)$ 。轮数上限保证计算量可控，但不保证到达局部或全局最优。

每次只执行排列首项，依据已接受动作确定的新位置与新观测重建任务集。选中目标仍调用既有局部定位与完整光学后备，不在代理中心直接假定清除成功。任务按类型和频道或骨架索引区分，即使坐标重合也不合并；插入和反转都保持全部输入任务。因此联合调度改变访问顺序，不会删除无源认证所需的骨架义务。

8.2 区分首次发现义务与已知源的补测价值

骨架扫描先剔除已确认成功清除的频道，并优先检测当前测向机频道，再按频道编号排序，以避免一次不必要的初始切换。对其余频道采用以下准确规则：

- 问题三的 **unknown** 规则：尚未建立源轨迹的频道必须检测；已有正测向轨迹的未清除频道跳过骨架补测，保留为独立定位清除任务。
- 问题四的 **useful** 规则：未知频道仍必须检测；对已有轨迹的频道，在骨架点 p 仅当 $r_c > 19.9$ 且 $\|p - z_c\| - r_c \leq 1500$ 时补测。换言之， $r_c \leq 19.9$ 或 $\|p - z_c\| - r_c > 1500$ 才跳过；距离下界恰为 1500 时不跳过。

第二条中， $r_c \leq 19.9$ 表示已具备既有局部清除条件；由 $P_c \subseteq B(z_c, r_c)$ 可知， $\|p - z_c\| - r_c > 1500$ 时整个保守区域都超过最大电磁接收距离，补充示向没有价值。反向条件只表示可能有用，不保证定向源处于发射前方。局部任务完成后的停靠共享同样保留区域大小与 1500 米筛选，并额外跳过该频道曾在完全相同坐标检测过的位置；此去重条件不被误写为 **useful** 骨架扫描本身的条件。

两种规则都不跳过未知频道的必需检测。已知源从骨架扫描中省下的观测不算作无信号，也不填入已完成索引集合；其完成义务由“发现”转为“收到该频道成功清除

反馈”。尤其在问题四，`no_signal` 可以来自距离过远或位于发射背面，不能据此删除 1000 米位置圆盘，也不能将已知源改判为不存在。

8.3 安全停止不依赖启发式代理是否准确

记 V 为当前完整骨架索引集，问题三为 $\{0, \dots, 6\}$ ，最终问题四为 $\{0, \dots, 24\}$ ； V_c 只记录该频道已接受的实际骨架检测。定义频道完成证书

$$\text{Cert}_c = \begin{cases} C, & \text{已收到频道 } c \text{ 的 } \text{success} \text{ 清除反馈,} \\ N, & \text{从未发现该频道源, 且全部 } V \text{ 上的有效检测均无信号,} \\ U \text{ 或 } D, & \text{尚未确定或已发现未清除.} \end{cases} \quad (36)$$

若一个从未发现的频道在 $V_c = V$ 后仍全部无信号，则假设其有源将与问题三或问题四的覆盖定理矛盾，故 N 证书可靠；一旦进入 D 状态，只能以成功清除完成，不能转成 N 。合法停止条件为

$$[\forall c \in \{1, \dots, 20\}, \text{Cert}_c \in \{C, N\}] \quad \text{或} \quad [\#\{c : \text{Cert}_c = C\} = 16]. \quad (37)$$

第二项利用题给总数上界，统计的是 16 个不同频道的成功清除；已清除 10 个、队列暂时为空或长时间没有新发现均不满足停止条件。动作异常或时间中断也不能替代证书。

式(9)的真实源包含关系仅依赖先验和实际返回的有效正观测，不依赖选点代理是否准确。联合调度不改写观测，扫描筛选不虚构负证据，定向失信号不作不安全的位置删减；因此这些效率层动作不会破坏既有几何包含不变量。配合保留的有限局部测向和完整光学覆盖，发现、清除与停止分别拥有可核查的依据，而效率由真实执行的 L, M, S, F, K 统一结算。

8.4 默认配置的有限动作与虚拟时间上界

本节覆盖最终 `refined` 及历史 `enhanced` 策略：问题三均用原点及半径 1200 的六点环，问题四分别用 25 点双环和 990 米 31 点三角格。最终联合调度与已知频道扫描筛选只重排或减少任务外补测；均保留每局部任务至多 6 次固定测向和原矩形光学后备。假定接口正常、误差模型成立且无现实时间中断。以下沿用较大的 31 点范数与次数界，故同时覆盖两策略；不涵盖条带光学或自适应候选。距离单位均为米。

可行域尺寸与 216 单元证书。 记真源为 g ，最近正示向点为 s ， $\alpha = 1.005^\circ$ 。示向轴坐标满足 $0 \leq x \leq 1500$ 、 $|y| \leq x \tan \alpha$ 。计入显式外扩，令 $\varepsilon = 10^{-7}$ ，取

$$X = 1500 + 2\varepsilon, \quad W = 2((1500 + \varepsilon) \tan \alpha + \varepsilon / \cos \alpha).$$

于是纵向长度至多 X ，横向宽度至多 $W < 53$ ， $\sqrt{X^2 + W^2} < 1501$ 。最小包围圆圆心 $z \in P$ ，故 $\text{diam}(P) < 1501$ 且 $\|z - s\| < 1501$ 。正测点范数至多 3300、初始外切多边形范数小于 1801，使相关 `_padding` 为 $10^{-9} < \varepsilon$ ；此为几何与显式外扩的解析界，不是全部浮点运算的形式化认证。

单点域直接用一圆覆盖。否则设直径端点差为 (a, b) 、 $d = \sqrt{a^2 + b^2} > 0$ 。任意点差满足 $|\Delta x| \leq d$ 、 $|\Delta y| \leq W$ 、 $|b| \leq W$ ，故直径法向投影差 $|-b\Delta x + a\Delta y|/d \leq 2W$ 。计入投影外扩，直径对齐矩形仍长小于 1501、宽小于 106，对角线小于 1505。令 $h_- = \sqrt{2}(19.9 - 8\varepsilon)$ ，不大于代码单元边长上限，则

$$J \leq \left\lceil \frac{1501}{h_-} \right\rceil \left\lceil \frac{106}{h_-} \right\rceil = 54 \times 4 = 216 \quad (38)$$

个半径 19.9 的圆盘覆盖整个矩形。蛇形相邻中心距至多 $\sqrt{2} 19.9$ ，循环移位至多增加一条矩形内跳跃，故

$$L_{\text{snake}} \leq (216 - 1)\sqrt{2} 19.9 + \sqrt{1501^2 + 106^2} < 7570. \quad (39)$$

每个覆盖中心 q_{opt} 与 g 均在同一矩形中，故 $\|q_{\text{opt}} - g\| < 1505$ 。

恢复点必须以最近正测点为中心估计。 固定候选 $q_0 = z \pm \delta n$ 满足 $n \perp (z - s)$ 、 $\delta \leq 120$ ，因此 $\|q_0 - s\| \leq A := \sqrt{1501^2 + 120^2} < 1506$ 。在获得新正示向前，所有恢复仍使用同一个 s 。设 R_k 为相应轴向反射，则

$$q_{k+1} - s = \frac{1}{2}R_k(q_k - s), \quad \|q_{k+1} - s\| = \frac{1}{2}\|q_k - s\|.$$

反射只保持到 s 的距离，不能把失败点的源距与 1500 取平均。故连续恢复有 $\|q_k - s\| \leq 2^{-k}A$ ；新正示向更新 s 后重用首点界。由 $\|s - g\| \leq 1500$ ，全部局部测向点满足

$$\|q - g\| \leq B := 1500 + A < 3006, \quad \|q\| \leq 1800 + B < 4806. \quad (40)$$

圆心源距小于 1501，覆盖中心源距小于 1505，两者范数亦小于 4806。测向间连线至多 $2B < 6020$ ；额外圆心清除接入至多 $B + 1501 < 4520$ ；光学接入至多 $B + 1505 < 4520$ ，若紧接圆心试清则至多 $1501 + 1505 < 4520$ 。

任务进出、动作计数与总界。 每任务至多 6 次测向，各提前返回分支均成功清除或转入有限光学覆盖；真源在覆盖域内，故每个正常完成任务必清除一新源，任务数至多 16。成功终点距该源至多 20、范数至多 1820；共享补测及其 `near` 清除不移动。这处理了从另一源清除点进入下一任务的情形。

骨架点范数至多 $990\sqrt{7} < 2620$ 。集成策略整次骨架扫描不插入局部移动，故至多 31 次骨架抵达，出发于原点、骨架或成功清除点，每段小于 5240。任务首个动作由范数至多 2620 的任务外位置进入范数小于 4806 的局部点，入口小于 7500。此后至多 5 条测向连线、一次额外圆心清除移动（保证清除或小区域试清）、一次光学接入及一条蛇形，故

$$L \leq 31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570). \quad (41)$$

若首动作即圆心清除或光学首点，入口已支付该段，多余预算仅为放宽。任务至下一骨架的退出段已计入 31 次骨架抵达；无需返回原锚点。

骨架至多 31×20 次测向；共享只在任务结束后调用，至多 16 次、每次至多 16 源；局部至多 16×6 次。放宽共享预算得到

$$M \leq 31 \times 20 + 16 \times 16 + 16 \times 6 \leq 31 \times 20 + 31 \times 16 + 16 \times 6 = 1212.$$

每源无信号后试清至多 6 次、小区域试清至多一次、覆盖至多 216 次，另留一次给非覆盖成功清除（骨架、共享、局部 near 或保证清除）；成功后不再处理，故 $C \leq 16(216 + 6 + 1 + 1) = 3584$ 。移动速度为 5 米/秒，测向含换频至多 6 秒，清除至多 5 秒，因而

$$T \leq \frac{31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570)}{5} + 1212 \times 6 + 3584 \times 5 < 100 \times 3600 \text{ s.} \quad (42)$$

这是虚拟时间保守界，不是预期成绩；现实 20 分钟限制须由客户端期限机制另行遵守。

网络故障可能耗尽现实时间，任何算法都不能对无限网络延迟保证 20 分钟内完成。客户端必须按真实剩余时间停止并报告未完成，不能将这类异常伪装成全清；本文尚未验证官方接口条件下的现实耗时。

9 实验设计：先筛选，再冻结验证

9.1 分布、误差与信息隔离

本地环境按公开动作协议生成反馈。常规世界源数在 10 至 16 间离散均匀，频道无放回抽样，位置在半径 1800 米圆盘内按面积均匀分布，接收半径在 1000 至 1500 米间均匀。问题三全部全向；问题四随机选择至少一个但非全部为定向源，发射方向均匀。这些是**合成分布假设**，不是官方数据的统计结论。策略只调用动作接口，总数、类型和真实位置不能参与在线决策；退出后评价器才用总数核对清除率。

同地点同频道的误差固定。实验设置三类误差场：有界双正弦平滑场、按位置哈希取 $\pm 1^\circ$ 端点的极端场、按位置哈希在 $[-1^\circ, 1^\circ]$ 中取值的场。第三类不表示原地重复读数独立。接口统一进行两位小数舍入，几何层按 1.005° 保守处理。

三类压力位置分别为：全部接收半径 1000 米；16 个圆周源且问题四中 15 个向外发射、另一个全向；16 个源聚集在距圆心 1650 米处的 35 米小圆盘。它们分别检验最小接收边界、定向搜索不能局限于圆内，以及集中多源时调度可能改变效率的情况。

9.2 基线、开发与验证数据的明确分工

历史 baseline 逐源离开扫描点定位再返回，batched 先完成当前点频道扫描，integrated 另加清除后的停靠共享。历史 enhanced 在此基础上取问题三 1200 米环、问题四 31 点滚动骨架排序。它是本轮直接对照，不与最弱的逐源往返策略混淆。

1200 米环的先前筛选保留在支撑材料：150 个开发世界比较 8 个半径，之后 500 个新世界比较冻结的 1200 和 1250 米，前者每源节省均值为 2.377 秒，近似 95% 区间为 $[1.608, 3.145]$ ；2200 次执行不是 2200 个独立世界。本轮不再用最终验证组回调该环参数。

新方法的开发只使用种子 3100000 至 3100149，每问 150 个世界。问题三 6 个、问题四 11 个有限候选共 2550 次执行；先筛覆盖、联合调度与条带光学，再针对开发中观察到的重复补测代价比较扫描规则。表2保留全部候选，包括无收益者。条带方案虽有

全域光学证书，完整任务收益却很小，默认仍采用更简单的矩形后备；问题四不能直接套用问题三的“只扫未知频道”，否则丢失的几何信息会增加后续移动。

表 2: 有限候选开发筛选，单位为秒/源；每行 150 个相同开发世界，非最终检验

问题	开发候选	均值	较对照节省	全清
Q3	原增强	277.85	0.00	150/150
Q3	条带光学	277.77	0.08	150/150
Q3	中心联合	266.96	10.89	150/150
Q3	半径联合	270.70	7.16	150/150
Q3	中心联合 + 未知扫描	261.88	15.97	150/150
Q3	中心联合 + 筛选补测	264.32	13.53	150/150
Q4	原增强	813.85	0.00	150/150
Q4	条带光学	813.34	0.51	150/150
Q4	中心联合	823.40	-9.55	150/150
Q4	半径联合	815.48	-1.63	150/150
Q4	双环 990/1865	618.77	195.08	150/150
Q4	双环 940/1870	615.51	198.34	150/150
Q4	双环 + 中心联合	595.96	217.89	150/150
Q4	双环 + 半径联合	579.66	234.19	150/150
Q4	双环中心联合 + 条带	595.54	218.31	150/150
Q4	双环联合 + 未知扫描	627.98	185.87	150/150
Q4	双环联合 + 筛选补测	551.63	262.22	150/150

冻结默认配置后，另用 4100000 至 4100999 的 1000 个新世界/问作主配对比较。问题三保留“对照、联合调度、最终”，问题四保留“对照、仅双环、双环联合、最终”，共 7000 次。压力组种子 4200000 至 4200049，在每问三场景、三误差和两策略上共 1800 次；随机位置的另两种误差用 4210000 至 4210199，共 1600 次；横偏移敏感性用 4300000 至 4300049，共 300 次。合计 10700 次运行。开发、主检验、压力及敏感性数据不混用，候选冻结后不依据这些结果继续调参。

9.3 配对指标与不确定性

令第 j 个相同世界上对照与候选的单源耗时差为

$$d_j = \frac{T_j^{\text{control}}}{N_j} - \frac{T_j^{\text{candidate}}}{N_j}, \quad \bar{d} = \frac{1}{n} \sum_j d_j. \quad (43)$$

报告均值、中位数、最近秩 p_{90} 、最坏值及更快案例数；对 $\bar{d}$ 给出近似 95% 区间 $\bar{d} \pm 1.96s_d/\sqrt{n}$ 。该区间描述声明合成分布下的配对均值不确定性，不是官方成绩区间，也不证明每个世界都更快。多个开发候选的筛选表不用作无选择偏差的统计显著性检验。

10 结果、消融与适用边界

10.1 冻结后的主要效果

表3直接比较已有增强策略和最终策略，全部主检验运行均全清。问题三均值由 276.33 降至 261.46 秒/源，问题四由 791.91 降至 544.74 秒/源；对应下降 5.38% 和 31.21%。每问更快的世界数分别为 737/1000、961/1000。问题四 p_{90} 从 1011.36 降至 682.56 秒/源，最坏值从 1215.65 降至 812.31 秒/源；这比仅展示均值更能说明本组尾部表现。两问整体配对节省均值分别为 14.88 与 247.17 秒/源；近似 95% 区间分别为 [13.46, 16.30] 和 [240.71, 253.64]。这里直接计算最终策略与对照的逐世界差值，不把相关消融步骤的区间端点相加。

表 3: 每问 1000 个全新世界的冻结策略比较，单位为秒/源；合成而非官方成绩

问题	配置	均值	中位数	p_{90}	最坏	全清
Q3	原增强	276.33	275.26	328.13	393.21	1000/1000
Q3	联合调度	266.31	262.80	312.40	360.94	1000/1000
Q3	最终策略	261.46	256.77	309.47	356.93	1000/1000
Q4	原增强	791.91	801.09	1011.36	1215.65	1000/1000
Q4	仅双环	600.23	603.40	752.01	920.12	1000/1000
Q4	联合调度	572.64	563.33	713.27	851.05	1000/1000
Q4	最终策略	544.74	535.74	682.56	812.31	1000/1000

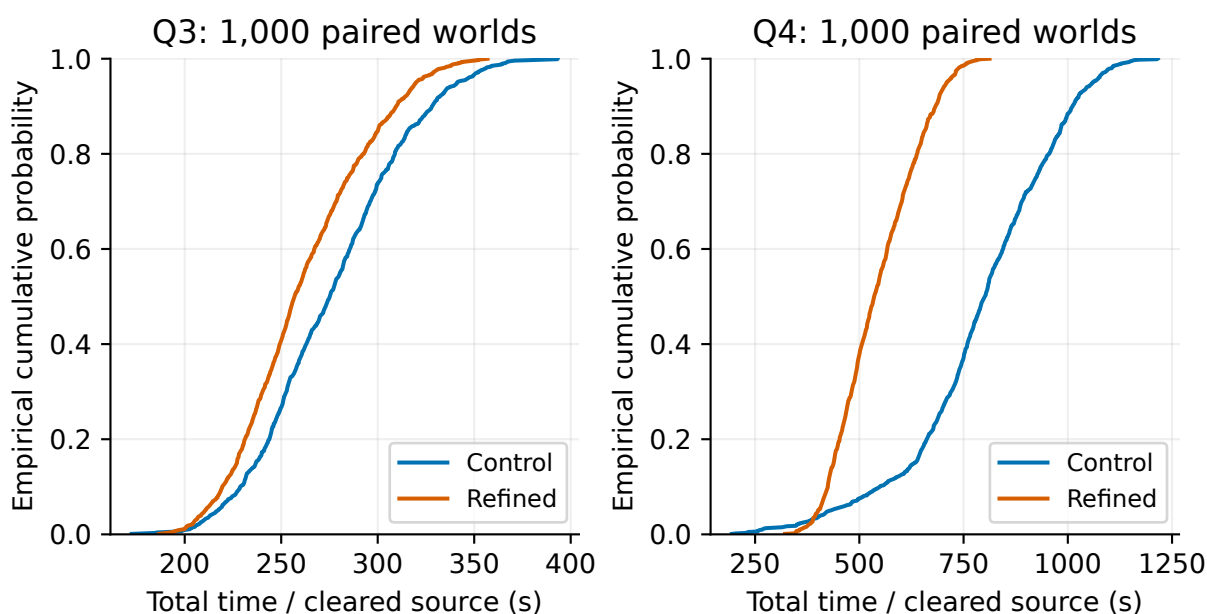


图 4: 主检验中单源完整耗时的经验分布。每条曲线以世界为等权统计单位；曲线左移表示在该分位数上更省时。

10.2 收益究竟来自哪里

表4每次只沿冻结配置链加入一个机制。问题四仅重新布置 25 点骨架就节省 191.69 秒/源；加入联合任务路线再节省 27.59 秒/源；筛去已可清除或完全超接收范围的已知频道补测，再节省 27.90 秒/源。该消融顺序具有条件性，不能把增量解释成脱离其他机制的普适独立贡献。

表 4: 相邻配置的同世界配对消融，节省与区间单位为秒/源

问题	单步增量	节省均值	近似 95% 区间	更快案例
Q3	原增强至联合调度	10.03	[8.62, 11.44]	665/1000
Q3	联合调度至最终策略	4.85	[4.40, 5.31]	838/1000
Q4	原增强至仅双环	191.69	[186.49, 196.88]	963/1000
Q4	仅双环至联合调度	27.59	[24.08, 31.10]	733/1000
Q4	联合调度至最终策略	27.90	[27.09, 28.71]	987/1000

完整成本分解仍为

$$T = T_{\text{move}} + T_{\text{measure}} + T_{\text{switch}} + T_{\text{fail}} + T_{\text{clear}} = L/5 + 5M + S + 3F + 5K. \quad (44)$$

表5先把每个世界的各项费用除以该世界源数，再对世界取平均。单个分项变好并不足够：问题三联合路线允许检测次数略增，但移动显著减少；问题四最终移动均值从 39337.68 降至 24430.10 米，检测均值从 341.89 降至 328.60 次。相比之下，“双环联合但不筛补测”的检测均值为 389.24 次，说明减少重复测量是必要的成本控制。

表 5: 逐世界每源完整成本分解；舍入后分项和可与总计差 0.01 秒/源

问题	配置	移动	检测	换频	失败	成功	合计
Q3	原增强	213.89	48.32	8.95	0.18	5.00	276.33
Q3	最终策略	197.58	49.31	9.38	0.19	5.00	261.46
Q4	原增强	623.06	137.81	24.46	1.59	5.00	791.91
Q4	最终策略	384.04	131.10	24.20	0.40	5.00	544.74

10.3 压力检验不隐去负结果

表6列出全部压力组。所有组均全清，但效率不逐场景占优：问题四边界外向三误差组平均下降约 24%，最小半径组约 30% 至 31%；聚集源端点误差组反而平均增加 8.14 秒/源，即 2.32%，其配对节省区间为 $[-57.61, 41.34]$ 秒/源，不能宣称改善。聚集时少数局的发现顺序和提前达到 16 源上界会显著影响耗时；这类分布的效率是保留的局限，而不是用新验证集再次调参的理由。

表 6: 完整压力结果；最小半径、边界外向和边缘聚集各组每行 50 对，随机位置附加误差各组每行 200 对，均全清；单位秒/源

问题	位置场景	误差场	对照	最终	节省	更快
Q3	最小半径	平滑	278.47	272.32	6.15	29/50
Q3	最小半径	端点	282.01	278.67	3.34	24/50
Q3	最小半径	位置哈希	279.90	273.34	6.56	31/50
Q3	边界外向	平滑	326.66	285.87	40.79	50/50
Q3	边界外向	端点	329.64	286.51	43.13	50/50
Q3	边界外向	位置哈希	327.59	285.71	41.88	50/50
Q3	边缘聚集	平滑	109.70	91.84	17.86	31/50
Q3	边缘聚集	端点	109.97	92.15	17.81	30/50
Q3	边缘聚集	位置哈希	109.64	91.85	17.79	34/50
Q3	随机位置	端点	278.45	261.03	17.43	157/200
Q3	随机位置	位置哈希	276.53	260.28	16.24	149/200
Q4	最小半径	平滑	806.79	560.91	245.88	47/50
Q4	最小半径	端点	806.66	554.42	252.24	49/50
Q4	最小半径	位置哈希	813.04	561.22	251.82	49/50
Q4	边界外向	平滑	870.31	661.83	208.49	50/50
Q4	边界外向	端点	872.66	663.36	209.30	50/50
Q4	边界外向	位置哈希	869.11	659.46	209.64	49/50
Q4	边缘聚集	平滑	353.69	348.84	4.84	24/50
Q4	边缘聚集	端点	351.06	359.20	-8.14	25/50
Q4	边缘聚集	位置哈希	354.96	345.46	9.50	28/50
Q4	随机位置	端点	785.13	534.36	250.76	192/200
Q4	随机位置	位置哈希	785.50	541.30	244.20	192/200

10.4 参数敏感性与实际轨迹

将横偏移系数 0.22 扰动为 0.12 和 0.40，其他参数冻结，每问各用 50 个独立世界。表7检验最终策略对这一工程参数的依赖；各组均保留全清，但平均耗时会随测向角度与移动代价的交换发生变化。所列值是敏感性结果，不把其中最小者追认成新的默认参数。

表 7: 最终策略的横偏移系数敏感性，单位秒/源

问题	横偏移比例	均值	p_{90}	全清
Q3	0.12	254.26	306.06	50/50
Q3	0.22	254.78	304.77	50/50
Q3	0.4	255.87	306.02	50/50
Q4	0.12	526.15	648.72	50/50
Q4	0.22	526.46	648.80	50/50
Q4	0.4	525.74	646.00	50/50

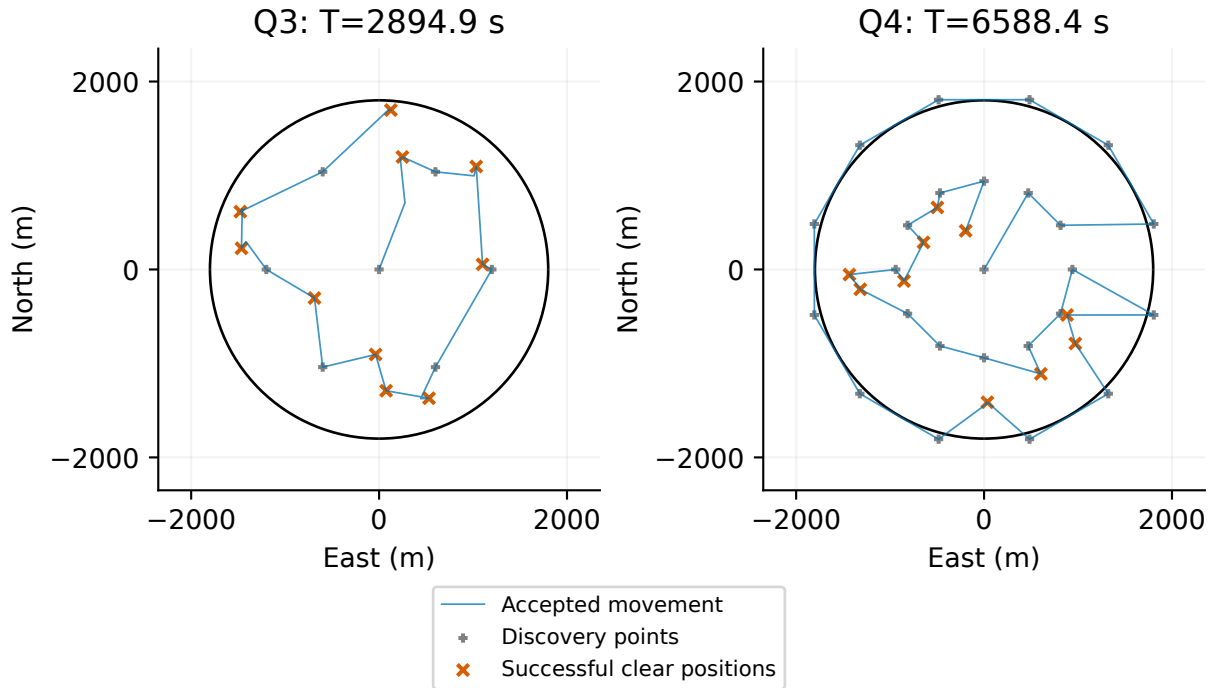


图 5: 种子 20260910 上最终策略的完整接受动作轨迹。叉号是成功清除时的机器狗位置，不是真实源坐标。问题四仍保留圆外检测；行程由公开反馈在线决定。

11 正确性核验与工程实现

11.1 独立几何检查与状态回归

解析证明之外，另用不同计算路径检查实现：包围圆与 1/2/3 点支持圆穷举交叉核对；测向区间加入两位小数舍入后检查真实源包含；新 25 点构造核对三角剖分的内部反向边、无交叉和面积一致，使用 70 位十进制根式构造核对实际双精度坐标。双环坐标对 70 位构造的最大差约 1.52×10^{-12} 米，远小于 1 米预算。方向采样补充而不取代全称证明；光学条带候选也另核对 125 个凸域的 24072 个内外边界样点，但其小收益不足以让默认策略承担精确有理分割的额外复杂度。

11.2 协议、计时及期限

每个新动作使用唯一请求编号，传输失败只重发同路径、同编号和同请求字节；完整 HTTP 响应和 `accepted=true` 同时成立才提交位置、频道和虚拟时间。`/clear` 不改变测向机频道，拒绝响应中的零时间不能重置当前时刻。程序不等待虚拟动作耗时；现实截止采用 `/enter` 返回的剩余时间，虚拟上限为 360000 秒。

本地截断响应夹具在首次检测已经执行后中断响应，客户端重试同 ID，只计费一次；附件示例最终为 199 秒。慢速响应夹具验证绝对期限而非仅空闲 socket 超时。达到期限后保留最后确认状态，不再通过 `/exit` 查询已经关闭的接口。所有合成实验逐接受动作独立累计距离、换频和操作费用，并核对清除频道唯一性。已接受终端动作可以恰好完成证书；没有完成时则明确报告不完整。

表 8: 核验项目及其证明范围, 所有执行均为本地

项目	实际规模	核验目的
最小包围圆独立穷举	504 组点集	含点、线、近共线和随机点集
真实源包含	2000 组、每组 4 测向	角度跨零、边界源、端点误差及舍入
双环有限结构	每构型 36 三角形、60 边	无交叉、闭合剖分与最大边界
双环方向及扰动	每构型 22050 名义例、44100 扰动	包含辅助点恰在边或顶点, 分别恶化投影与源距
已知频道跳过与停止	两题默认扫描规则	跳过不产生无源证书, 负观测仍保持已发现, 成功清除才完成
终端状态与 HTTP 重试	期限、near、16 源及截断响应	不确定动作不推进, 超时不伪造退出

11.3 复现与不能替代的官方证据

支撑包保留源代码、全部逐世界结果、开发与检验种子、生成图表脚本及输入哈希。复现按几何核验、实验、资产生成、LaTeX 编译的顺序进行, 解压到独立目录重建, 不以原目录已有文件充当复现成功。PDF 从摘要页起编号, 正文不超过 30 页, 完整代码置于附录 [3]。

官方演练必须由模拟器 GUI 在线登录并开启正确题号, 接口就绪后运行 `official_run.py`; 程序不能导入本地世界, 也不能在正式案例中读取源数。演练结束后用 GUI 披露的 N 核验 K/N ; 正式测试各题三次, 按题表记录实际案例编码、 K 、 T/K 和 GUI 定义的程序时间, 导出六份原名原字节加密日志并核对上传。当前这些材料尚缺, 本文不填写虚构正式表。含队号的明文 HTTP 日志只作私有工作材料, 不替代官方加密文件。

12 结论与模型局限

问题一由正向半平面交求区域直径, 最小包围圆而非直径圆是一次保证清除的正确判据; 问题二给出有全向接收保证的三圆交候选域, 采样评分与在线启发式的最优性边界分开说明。问题三以七点收缩环和逐频道证书保证发现与停止; 问题四重新构造具有严格方向和距离余量的 25 点双环。联合调度与已知频道扫描筛选在不删未知频道义务的前提下, 提高了声明合成分布下的整体效率。

优势是几何安全层与启发式效率层分离、每条停止结论有证书、每个效率提升有同世界增量对照; 不依赖读出隐藏源数或猜测原地误差独立。局限是 25 点及任务路线没有全局最优证明, 保守凸多边形没有充分利用负观测的非凸信息, 聚集源场景未体现稳定效率优势, 合成误差场与官方世界均未标定。实际 HTTP 延迟和现实 20 分钟约束不能由本地毫秒级计算替代。后续使用仍须完成官方实测和真实参赛队对核心模型、程序及 AI 输出的逐项人工核验, 不以自动审查替代参赛责任 [4]。

AI 工具使用声明

本研究过程中使用了 AI 工具，参与核心模型推导、代码实现、实验设计、结果整理、独立自动审查及论文写作，详细过程见支撑材料中的《AI 工具使用详情》。本稿尚无真实参赛队完成逐项人工核验的记录，不将自动审查写作人工审查；该声明用于如实披露研究过程，不能替代正式提交前应完成的参赛队责任。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除，以及附件 1、附件 2[Z]. 2026. https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629fb.html.
- [2] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]//New Results and New Trends in Computer Science. Lecture Notes in Computer Science, 555. Springer, 1991: 359–370. DOI: 10.1007/BFb0038202.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html.
- [4] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/fef94648f2836ab6cc81586f4c38512b.html.

A 匿名复现附录

本附录只列出匿名包中实际存在、可供复现使用的源文件。代码路径均相对于本文件所在的 report/目录；结果路径均相对于匿名包项目根目录；不含个人绝对路径、队伍身份或私有凭据。主文件应在导言区加载 listings 并设置合适的 Python 样式。

A.1 运行与验证源代码

Listing 1: 规划器与恢复策略

```
1  """Coverage-certified search; consumes observations only, never simulator truth."""
2
3  from dataclasses import dataclass, field
4  from math import dist, hypot
5  from time import monotonic
6
7  from active_policy_candidate import choose_probe
8  from client import AmbiguousActionError, DeadlineExceeded
9  from coverage import directional_waypoints, omni_waypoints
10 from geometry import clip_bearing, enclosing_circle, initial_polygon, optical_cover
11 from route_policy_candidate import optimize_waypoints
12 from joint_dispatch_candidate import choose_task
13 from optical_policy_candidate import certified_optical_cover
14 from polar_coverage_candidate import polar_waypoints
15
16
17 class SessionTerminated(RuntimeError):
18     """The simulator closed after an accepted action reached its limit."""
19
20
21 class InconsistentObservations(RuntimeError):
22     """The specified bounded-error model no longer contains a possible source."""
23
24
25 @dataclass
26 class Track:
27     polygon: list = field(default_factory=initial_polygon)
28     readings: list = field(default_factory=list)
29     observations: list = field(default_factory=list)
30     failures: list = field(default_factory=list)
31
32
33 class Planner:
34     """Baseline clears detections immediately; integrated batches and shares stops.
35
36     Sampling and route heuristics affect time only. Completeness comes from the
37     waypoint certificate and the conservative, finite optical-cover fallback.
38     """
39
40     def __init__(
41         self,
42         environment,
43         problem=3,
44         strategy="refined",
45         spacing=990.0,
46         probe_scale=0.22,
47         max_probes=6,
48         loss_recovery=True,
49         anchor_policy=None,
50         probe_policy="fixed",
51         ring_radius=None,
52         coverage_policy=None,
53         optical_policy="rectangle",
```

```

54     dispatch_policy=None,
55     scan_policy=None,
56 ):
57     if problem not in (3, 4) or strategy not in (
58         "baseline",
59         "batched",
60         "integrated",
61         "enhanced",
62         "refined",
63     ):
64         raise ValueError("Unsupported problem or strategy")
65     if anchor_policy is None:
66         anchor_policy = "rolling" if strategy in ("enhanced", "refined") and problem == 4 else "nearest"
67     if ring_radius is None and strategy in ("enhanced", "refined") and problem == 3:
68         ring_radius = 1200.0
69     if coverage_policy is None:
70         coverage_policy = "compact25" if strategy == "refined" and problem == 4 else "lattice"
71     if dispatch_policy is None:
72         dispatch_policy = ("center" if problem == 3 else "guarded") if strategy == "refined" else "separate"
73
74     if scan_policy is None:
75         scan_policy = ("unknown" if problem == 3 else "useful") if strategy == "refined" else "all"
76     if anchor_policy not in ("nearest", "two_opt", "rolling"):
77         raise ValueError("Unsupported anchor policy")
78     if probe_policy not in ("fixed", "adaptive"):
79         raise ValueError("Unsupported probe policy")
80     if not 900 <= spacing <= 999:
81         raise ValueError("Directional lattice spacing must be in [900, 999] m")
82     if not 0 < probe_scale <= 1 or max_probes < 1:
83         raise ValueError("Invalid localization policy parameters")
84     if coverage_policy not in ("lattice", "seed25", "compact25"):
85         raise ValueError("Unsupported discovery coverage policy")
86     if optical_policy not in ("rectangle", "slabs"):
87         raise ValueError("Unsupported optical covering policy")
88     if dispatch_policy not in ("separate", "center", "guarded"):
89         raise ValueError("Unsupported task dispatch policy")
90     if scan_policy not in ("all", "unknown", "useful"):
91         raise ValueError("Unsupported known-channel scan policy")
92     self.scan_policy = scan_policy
93     self.coverage_policy = coverage_policy if problem == 4 else "ring"
94     self.optical_policy = optical_policy
95     self.dispatch_policy = dispatch_policy
96     self.environment = environment
97     self.problem = problem
98     self.strategy = strategy
99     self.waypoints = (
100         omni_waypoints(ring_radius) if problem == 3 else
101         directional_waypoints(spacing) if coverage_policy == "lattice" else
102         polar_waypoints(coverage_policy)
103     )
104     self.anchor_policy = anchor_policy
105     self.anchor_order = optimize_waypoints(self.waypoints) if anchor_policy == "two_opt" else []
106     self.anchor_rank = {index: rank for rank, index in enumerate(self.anchor_order)}
107     self.pending = set(range(len(self.waypoints)))
108     self.visited = []
109     self.tracks = {}
110     self.cleared = set()
111     self.scan_records = {ch: set() for ch in range(1, 21)}
112     self.position = (0.0, 0.0)
113     self.radio_channel = 1
114     self.virtual_time = 0.0
115     self.deadline = float("inf")
116     self.max_virtual_duration = float("inf")
117     self.terminal_reason = None
118     self.observation_points = {ch: [] for ch in range(1, 21)}
119     self.channel_state = {ch: "unresolved" for ch in range(1, 21)}
120     self.probe_scale = probe_scale

```

```

120     self.max_probes = max_probes
121     self.loss_recovery = bool(loss_recovery) and problem == 4
122     self.fallbacks = 0
123     self.probe_policy = probe_policy
124     self.adaptive_probes = 0
125     self.termination = None
126
127     def action(self, path, point=None, channel=None):
128         if self.terminal_reason is not None:
129             raise SessionTerminated(self.terminal_reason)
130         if monotonic() >= self.deadline - 2.0 and path != "/exit":
131             raise TimeoutError("Insufficient real time: completeness not certified")
132         response = self.environment.act(path, position=point, channel=channel)
133         if response.get("accepted") is not True:
134             raise RuntimeError("Rejected action; cached state was not advanced")
135         self.virtual_time = float(response["virtual_time_s"])
136         if path == "/enter":
137             self.max_virtual_duration = float(response.get("max_virtual_duration_s", float("inf")))
138         if point is not None:
139             self.position = tuple(point)
140         if path == "/measure":
141             self.radio_channel = channel
142         if path in ("/measure", "/clear") and self.virtual_time >= self.max_virtual_duration:
143             self.terminal_reason = "virtual_timeout"
144         return response
145
146     def clear(self, channel, point, guaranteed=False):
147         result = self.action("/clear", point, channel)["clear_result"]
148         if result == "success":
149             self.cleared.add(channel)
150             self.channel_state[channel] = "cleared"
151             self.tracks.pop(channel, None)
152             return True
153         if result != "no_target_in_range":
154             raise RuntimeError(f"Unknown clear result: {result}")
155         if guaranteed:
156             raise InconsistentObservations("Certified radius-20 clearing failed")
157         if channel in self.tracks:
158             self.tracks[channel].failures.append(tuple(point))
159         return False
160
161     def measure(self, channel, point):
162         result = self.action("/measure", point, channel)
163         status = result["measure_result"]
164         self.observation_points[channel].append(tuple(point))
165         track = self.tracks.get(channel)
166         if track is None and status == "direction":
167             track = Track(observations=self.observation_points[channel])
168             self.tracks[channel] = track
169         if track is not None:
170             track.observations = self.observation_points[channel]
171         if status == "near":
172             self.channel_state[channel] = "detected"
173             if self.terminal_reason is None:
174                 self.clear(channel, point, guaranteed=True)
175         elif status == "direction":
176             self.channel_state[channel] = "detected"
177             track.polygon = clip_bearing(track.polygon, point, result["svd_deg"])
178             if not track.polygon:
179                 raise InconsistentObservations(
180                     f"Empty feasible region on channel {channel}"
181                 )
182             track.readings.append((tuple(point), float(result["svd_deg"])))
183         elif status != "no_signal":
184             raise RuntimeError(f"Unknown measure result: {status}")
185         # No-signal observations are not convex halfplanes. In Q4 in particular,
186         # deleting a 1000-m disk would silently exclude real sources.

```

```

187         return status
188
189     def scan_anchor(self, index):
190         point = self.waypoints[index]
191         channels = [ch for ch in range(1, 21) if ch not in self.cleared]
192         channels.sort(key=lambda ch: (ch != self.radio_channel, ch))
193         for channel in channels:
194             if channel in self.cleared:
195                 continue
196             if channel in self.tracks:
197                 if self.scan_policy == "unknown":
198                     continue
199                 if self.scan_policy == "useful":
200                     center, radius = enclosing_circle(self.tracks[channel].polygon)
201                     if radius <= 19.9 or dist(point, center) - radius > 1500:
202                         continue
203             self.measure(channel, point)
204             self.scan_records[channel].add(index)
205             if len(self.cleared) == 16:
206                 self.termination = "known_count_upper_bound_16"
207                 break
208             if self.terminal_reason is not None:
209                 break
210             if self.strategy == "baseline" and channel in self.tracks:
211                 self.localize(channel)
212             if len(self.cleared) == 16:
213                 self.termination = "known_count_upper_bound_16"
214                 break
215             if self.terminal_reason is not None:
216                 break
217         self.pending.discard(index)
218         if index not in self.visited:
219             self.visited.append(index)
220
221     def optical_finish(self, channel):
222         self.fallbacks += 1
223         covering = certified_optical_cover if self.optical_policy == "slabs" else optical_cover
224         cover = covering(self.tracks[channel].polygon, radius=19.9)
225         if not cover:
226             raise InconsistentObservations("An empty region cannot certify clearing")
227         start = min(range(len(cover)), key=lambda k: dist(self.position, cover[k]))
228         # Rotate the certified snake to the nearest cell. At most one long
229         # wraparound jump; the remaining edges are adjacent rectangle cells.
230         order = cover[start:] + cover[:start]
231         for point in order:
232             if self.clear(channel, point):
233                 return
234             if self.terminal_reason is not None:
235                 return
236         raise InconsistentObservations("Exhausted a certified optical cover without clearing")
237
238     def localize(self, channel):
239         losses = 0
240         failed_probe = None
241         for probe_index in range(self.max_probes):
242             if channel in self.cleared or self.terminal_reason is not None:
243                 return
244             track = self.tracks[channel]
245             center, radius = enclosing_circle(track.polygon)
246             if max(dist(self.position, p) for p in track.polygon) <= 19.9:
247                 self.clear(channel, self.position, guaranteed=True)
248                 return
249             if radius <= 19.9:
250                 self.clear(channel, center, guaranteed=True)
251                 return
252             # When the remaining uncertainty is small, a 3-s failed optical
253             # attempt can cost less than travelling to take another bearing.

```

```

254     if radius <= 38 and center not in track.failures:
255         if self.clear(channel, center):
256             return
257         if self.terminal_reason is not None:
258             return
259         self.optical_finish(channel)
260         return
261     previous = track.readings[-1][0]
262     dx, dy = center[0] - previous[0], center[1] - previous[1]
263     length = hypot(dx, dy)
264     if length < 1e-8:
265         self.optical_finish(channel)
266         return
267     unit, normal = (dx / length, dy / length), (-dy / length, dx / length)
268     if failed_probe is not None and self.loss_recovery:
269         vector = (failed_probe[0] - previous[0], failed_probe[1] - previous[1])
270         axial = vector[0] * unit[0] + vector[1] * unit[1]
271         lateral = vector[0] * normal[0] + vector[1] * normal[1]
272         point = (
273             previous[0] + 0.5 * (axial * unit[0] - lateral * normal[0]),
274             previous[1] + 0.5 * (axial * unit[1] - lateral * normal[1]),
275         )
276         if any(point == p for p in track.observations) or point in track.failures:
277             self.optical_finish(channel)
278             return
279     else:
280         point = None
281         if self.probe_policy == "adaptive":
282             point = choose_probe(
283                 track.polygon,
284                 track.readings,
285                 self.position,
286                 self.problem,
287                 probes_remaining=self.max_probes - probe_index,
288                 rejected_points=track.failures + track.observations,
289                 probe_scale=self.probe_scale,
290             )
291         if point is not None:
292             self.adaptive_probes += 1
293         if point is None:
294             offset = min(120.0, max(25.0, self.probe_scale * radius))
295             candidates = [
296                 (
297                     center[0] + sign * offset * normal[0],
298                     center[1] + sign * offset * normal[1],
299                 )
300                 for sign in (-1, 1)
301             ]
302             fresh = [
303                 p
304                 for p in candidates
305                 if p not in track.observations
306                 and all(p != s for s in track.failures)
307             ]
308             if not fresh:
309                 self.optical_finish(channel)
310                 return
311             point = min(fresh, key=lambda p: dist(self.position, p))
312     status = self.measure(channel, point)
313     if channel in self.cleared or self.terminal_reason is not None:
314         return
315     if status == "no_signal":
316         losses += 1
317         if self.clear(channel, point):
318             return
319         if self.terminal_reason is not None:
320             return

```

```

321         failed_probe = point
322         if losses >= 2 and not self.loss_recovery:
323             self.optical_finish(channel)
324             return
325         else:
326             failed_probe = None
327         if channel not in self.cleared and self.terminal_reason is None:
328             self.optical_finish(channel)
329
330     def share_current_stop(self):
331         """Fuse an extra bearing for known sources at an already reached stop."""
332         if self.strategy not in ("integrated", "enhanced", "refined") or self.terminal_reason is not None or
333         len(self.cleared) == 16:
334             return
335         for channel in list(self.tracks):
336             if channel in self.cleared:
337                 continue
338             track = self.tracks[channel]
339             center, radius = enclosing_circle(track.polygon)
340             if radius <= 19.9:
341                 continue
342             if any(self.position == p for p in track.observations):
343                 continue
344             if dist(self.position, center) - radius > 1500:
345                 continue
346             self.measure(channel, self.position)
347             if self.terminal_reason is not None:
348                 return
349
350     def next_anchor(self):
351         if self.anchor_policy == "two_opt":
352             return min(self.pending, key=lambda i: self.anchor_rank[i])
353         if self.anchor_policy == "rolling":
354             indices = sorted(self.pending)
355             order = optimize_waypoints([self.waypoints[i] for i in indices], self.position)
356             return indices[order[0]]
357         return min(self.pending, key=lambda i: (dist(self.position, self.waypoints[i]), i))
358
359     def _result(self):
360         complete = len(self.cleared) == 16
361         if not complete:
362             for channel in range(1, 21):
363                 if channel in self.cleared:
364                     self.channel_state[channel] = "cleared"
365                 elif (self.channel_state[channel] != "detected"
366                     and len(self.scan_records[channel]) == len(self.waypoints)):
367                     self.channel_state[channel] = "absent-certified"
368                 else:
369                     complete = False
370                     break
371             else:
372                 complete = True
373         if self.terminal_reason is not None:
374             self.termination = ("complete_after_" if complete else "incomplete_") + self.terminal_reason
375
376     return {
377         "problem": self.problem,
378         "strategy": self.strategy,
379         "cleared_channels": sorted(self.cleared),
380         "n_cleared": len(self.cleared),
381         "virtual_time_s": self.virtual_time,
382         "average_time_s": self.virtual_time / len(self.cleared)
383         if self.cleared else None,
384         "probe_policy": self.probe_policy,
385         "anchor_policy": self.anchor_policy,
386         "ring_radius_m": hypot(*self.waypoints[1]) if self.problem == 3 else None,
387         "loss_recovery": self.loss_recovery,
388         "adaptive_probes": self.adaptive_probes,
389         "channel_certificates": dict(self.channel_state),

```

```

387         "certificate_complete": complete,
388         "terminal_reason": self.terminal_reason,
389         "visited_anchors": len(self.visited),
390         "total_anchors": len(self.waypoints),
391         "optical_fallbacks": self.fallbacks,
392         "termination": self.termination,
393         "coverage_policy": self.coverage_policy,
394         "optical_policy": self.optical_policy,
395         "dispatch_policy": self.dispatch_policy,
396         "scan_policy": self.scan_policy,
397     }
398
399     def run(self):
400         try:
401             return self._run_body()
402         except (TimeoutError, ConnectionError, AmbiguousActionError,
403                 DeadlineExceeded, SessionTerminated) as exc:
404             self.terminal_reason = self.terminal_reason or type(exc).__name__
405             self.termination = self.termination or "incomplete_" + type(exc).__name__
406             return self._result()
407
408     def _run_body(self):
409         entry_started = monotonic()
410         entered = self.action("/enter")
411         self.deadline = entry_started + entered["remaining_real_duration_s"]
412         self.scan_anchor(0)
413         while (self.pending or self.tracks) and self.terminal_reason is None:
414             if len(self.cleared) == 16:
415                 self.termination = "known_count_upper_bound_16"
416                 break
417             if self.strategy == "baseline":
418                 if self.tracks:
419                     self.localize(next(iter(self.tracks)))
420                 elif self.pending:
421                     self.scan_anchor(self.next_anchor())
422                 continue
423             if self.dispatch_policy != "separate":
424                 kind, index = choose_task(
425                     self.position,
426                     {i: self.waypoints[i] for i in sorted(self.pending)},
427                     {ch: enclosing_circle(track.polygon) for ch, track in self.tracks.items()},
428                     choice=self.dispatch_policy,
429                 )
430                 if kind == "target":
431                     self.localize(index)
432                     self.share_current_stop()
433                 else:
434                     self.scan_anchor(index)
435                 continue
436             targets = [
437                 (dist(self.position, enclosing_circle(t.polygon)[0]), ch)
438                 for ch, t in self.tracks.items()
439             ]
440             anchors = [
441                 (
442                     dist(self.position, self.waypoints[i])
443                     + 30.0 * (20 - len(self.cleared)),
444                     i,
445                 )
446                 for i in ([self.next_anchor()] if self.pending and self.anchor_policy == "rolling" else self.
pending)
447             ]
448             # Convert a full scan to equivalent travel metres (6 s * 5 m/s).
449             # This is a myopic time heuristic, not a global-optimality claim.
450             if targets and (not anchors or min(targets)[0] <= min(anchors)[0]):
451                 self.localize(min(targets)[1])
452                 self.share_current_stop()

```

```

453         else:
454             self.scan_anchor(self.next_anchor())
455     if self.termination is None:
456         if self.terminal_reason is None:
457             for channel in range(1, 21):
458                 if channel in self.cleared:
459                     self.channel_state[channel] = "cleared"
460                 elif (self.channel_state[channel] != "detected"
461                     and len(self.scan_records[channel]) == len(self.waypoints)):
462                     self.channel_state[channel] = "absent-certified"
463                 else:
464                     raise RuntimeError("Incomplete per-channel discovery coverage")
465             self.termination = "all_channels_cleared_or_coverage_certified_absent"
466         else:
467             current = self._result()
468             if current["certificate_complete"]:
469                 self.termination = "complete_after_" + self.terminal_reason
470             else:
471                 self.termination = "incomplete_" + self.terminal_reason
472     if self.terminal_reason is None:
473         self.action("/exit")
474     return self._result()

```

Listing 2: 实验主程序

```

1  """Reproducible synthetic comparison; these are NOT official simulator scores."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import math
8  import statistics
9  from pathlib import Path
10
11  from environment import LocalSimulator
12  from planner import Planner
13
14
15  def run_case(
16      seed,
17      problem,
18      strategy,
19      scenario="random",
20      error_mode="smooth",
21      probe_scale=0.22,
22      spacing=990.0,
23      loss_recovery=True,
24      anchor_policy=None,
25      ring_radius=None,
26      probe_policy="fixed",
27      coverage_policy=None,
28      optical_policy="rectangle",
29      dispatch_policy=None,
30      scan_policy=None,
31  ):
32      env = LocalSimulator(seed, problem, error_mode, scenario)
33      planner = Planner(
34          env,
35          problem,
36          strategy,
37          spacing=spacing,
38          probe_scale=probe_scale,
39          loss_recovery=loss_recovery,
40          anchor_policy=anchor_policy,
41          ring_radius=ring_radius,
42          probe_policy=probe_policy,

```

```

43     coverage_policy=coverage_policy,
44     optical_policy=optical_policy,
45     dispatch_policy=dispatch_policy,
46     scan_policy=scan_policy,
47 )
48 p = planner.run()
49 assert p["certificate_complete"] is True
50 s = env.summary()
51 # Independent action-by-action accounting, without reading hidden world data.
52 position, radio, total = (0.0, 0.0), 1, 0.0
53 successes = set()
54 for event in env.log:
55     request, response = event["request"], event["response"]
56     if not response["accepted"]:
57         continue
58     path = request["path"]
59     if path in ("/measure", "/clear"):
60         q = (request["position"]["x"], request["position"]["y"])
61         total += round(math.dist(position, q) / 5, 6)
62         position = q
63         if path == "/measure":
64             total += 5 + int(radio != request["channel"])
65             radio = request["channel"]
66         else:
67             success = response["clear_result"] == "success"
68             total += 5 if success else 3
69             if success:
70                 assert request["channel"] not in successes
71                 successes.add(request["channel"])
72     assert abs(total - response["virtual_time_s"]) < 2e-6
73 assert len(successes) == s["n_cleared"] == p["n_cleared"]
74 assert s["n_sources"] == s["n_cleared"], (seed, problem, scenario, error_mode, s)
75 return {
76     **s,
77     "strategy": strategy,
78     "visited_anchors": p["visited_anchors"],
79     "optical_fallbacks": p["optical_fallbacks"],
80     "probe_scale": probe_scale,
81     "loss_recovery_requested": loss_recovery,
82     "loss_recovery": p["loss_recovery"],
83     "anchor_policy": p["anchor_policy"],
84     "ring_radius_m": p["ring_radius_m"],
85     "spacing_m": spacing,
86     "certificate_complete": p["certificate_complete"],
87     "timing_independently_verified": True,
88     "coverage_policy": p["coverage_policy"],
89     "optical_policy": p["optical_policy"],
90     "dispatch_policy": p["dispatch_policy"],
91     "scan_policy": p["scan_policy"],
92 }
93
94
95 def aggregate(rows):
96     values = sorted(row["average_time_s"] for row in rows)
97     return {
98         "cases": len(rows),
99         "all_cleared_cases": sum(r["fraction_cleared"] == 1 for r in rows),
100         "mean_average_time_s": statistics.mean(values),
101         "median_average_time_s": statistics.median(values),
102         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
103         "worst_average_time_s": values[-1],
104         "mean_total_time_s": statistics.mean(r["virtual_time_s"] for r in rows),
105         "mean_distance_m": statistics.mean(r["distance_m"] for r in rows),
106         "mean_measures": statistics.mean(r["measures"] for r in rows),
107         "mean_failed_clears": statistics.mean(r["failed_clears"] for r in rows),
108         "max_real_duration_s": max(r["real_duration_s"] for r in rows),
109     }

```

```

110
111
112 def main():
113     parser = argparse.ArgumentParser()
114     parser.add_argument("--cases", type=int, default=200)
115     parser.add_argument("--output", type=Path, default=Path("results/experiments.json"))
116     args = parser.parse_args()
117     if args.cases < 2:
118         parser.error("At least two paired cases required")
119     rows, grouped, paired, stress, sensitivity = [], {}, {}, {}, {}
120     for problem in (3, 4):
121         strategy_rows = {}
122         for strategy in ("baseline", "batched", "integrated"):
123             current = [
124                 run_case(600000 + k, problem, strategy) for k in range(args.cases)
125             ]
126             rows.extend(current)
127             strategy_rows[strategy] = current
128             grouped[f"Q{problem}_{strategy}"] = aggregate(current)
129         differences = [
130             a["average_time_s"] - b["average_time_s"]
131             for a, b in zip(strategy_rows["baseline"], strategy_rows["integrated"])
132         ]
133         delta = statistics.mean(differences)
134         standard_error = statistics.stdev(differences) / math.sqrt(len(differences))
135         paired[f"Q{problem}"] = {
136             "mean_saved_s_per_source": delta,
137             "approx_95pct_ci_s": [
138                 delta - 1.96 * standard_error,
139                 delta + 1.96 * standard_error,
140             ],
141             "mean_time_reduction_pct": 100
142             * delta
143             / grouped[f"Q{problem}_baseline"]["mean_average_time_s"],
144             "faster_cases": sum(d > 0 for d in differences),
145             "paired_cases": len(differences),
146         }
147         share_differences = [
148             a["average_time_s"] - b["average_time_s"]
149             for a, b in zip(strategy_rows["batched"], strategy_rows["integrated"])
150         ]
151         paired[f"Q{problem}_sharing_ablation"] = {
152             "mean_saved_s_per_source": statistics.mean(share_differences),
153             "faster_cases": sum(d > 0 for d in share_differences),
154             "paired_cases": len(share_differences),
155         }
156         for scenario in ("minimum_radius", "outward_boundary", "clustered"):
157             for mode in ("smooth", "extreme", "iid_location"):
158                 current = [
159                     run_case(700000 + k, problem, "integrated", scenario, mode)
160                     for k in range(20)
161                 ]
162                 rows.extend(current)
163                 stress[f"Q{problem}_{scenario}_{mode}"] = aggregate(current)
164             # Sensitivity uses different seeds, no held-out retuning afterwards.
165             for scale in (0.12, 0.22, 0.40):
166                 current = [
167                     run_case(800000 + k, problem, "integrated", probe_scale=scale)
168                     for k in range(30)
169                 ]
170                 rows.extend(current)
171                 sensitivity[f"Q{problem}_probe_{scale}"] = aggregate(current)
172     for spacing in (900.0, 950.0, 990.0):
173         current = [
174             run_case(900000 + k, 4, "integrated", spacing=spacing) for k in range(30)
175         ]
176         rows.extend(current)

```

```

177     sensitivity[f"Q4_spacing_{spacing}"] = aggregate(current)
178     recovery_ablation = {}
179     for scenario in ("random", "minimum_radius", "outward_boundary", "clustered"):
180         for enabled in (False, True):
181             current = [
182                 run_case(610000 + k, 4, "integrated", scenario, loss_recovery=enabled)
183                 for k in range(100)
184             ]
185             rows.extend(current)
186             recovery_ablation[f"{scenario}_recovery_{enabled}"] = aggregate(current)
187     result = {
188         "evidence": "local_synthetic_not_official",
189         "assumptions": "See environment.py module docstring",
190         "paired_random_cases_per_problem": args.cases,
191         "total_runs": len(rows),
192         "summary": grouped,
193         "paired_comparison": paired,
194         "stress": stress,
195         "sensitivity": sensitivity,
196         "recovery_ablation": recovery_ablation,
197         "runs": rows,
198     }
199     args.output.parent.mkdir(parents=True, exist_ok=True)
200     args.output.write_text(
201         json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
202     )
203     print(
204         json.dumps(
205             {k: v for k, v in result.items() if k != "runs"},
206             ensure_ascii=False,
207             indent=2,
208         )
209     )
210
211
212 if __name__ == "__main__":
213     main()

```

Listing 3: 统一运行入口

```

1 """Run local synthetic practice; official GUI sessions require client.HTTPClient separately."""
2
3 from __future__ import annotations
4
5 import argparse
6 import json
7 from pathlib import Path
8
9 from environment import LocalSimulator
10 from planner import Planner
11
12
13 def run_local(
14     seed: int,
15     problem: int,
16     scenario: str = "random",
17     error_mode: str = "smooth",
18     strategy: str = "refined",
19 ) -> dict:
20     env = LocalSimulator(
21         seed, problem=problem, scenario=scenario, error_mode=error_mode
22     )
23     planner = Planner(env, problem=problem, strategy=strategy)
24     planner_result = planner.run()
25     result = {"planner": planner_result, "simulator": env.summary()}
26     return result
27

```

```

28
29 def main() -> None:
30     parser = argparse.ArgumentParser()
31     parser.add_argument("--seed", type=int, default=20260910)
32     parser.add_argument("--problem", type=int, choices=(3, 4), default=3)
33     parser.add_argument("--scenario", default="random")
34     parser.add_argument("--error-mode", default="smooth")
35     parser.add_argument(
36         "--strategy",
37         choices=("baseline", "batched", "integrated", "enhanced", "refined"),
38         default="refined",
39     )
40     parser.add_argument("--output", type=Path)
41     args = parser.parse_args()
42     result = run_local(
43         args.seed, args.problem, args.scenario, args.error_mode, args.strategy
44     )
45     text = json.dumps(result, ensure_ascii=False, indent=2)
46     if args.output:
47         args.output.parent.mkdir(parents=True, exist_ok=True)
48         args.output.write_text(text + "\n", encoding="utf-8")
49     print(text)
50
51
52 if __name__ == "__main__":
53     main()

```

Listing 4: 几何计算

```

1  """Conservative bearing geometry and whole-region optical clearing certificates.
2
3  Polygons are ordered convex vertex lists (either winding); [] means infeasible,
4  never unbounded. All distances are metres and bearings are counterclockwise degrees.
5  """
6
7  from __future__ import annotations
8
9  from decimal import Decimal, localcontext
10 from math import ceil, cos, hypot, isfinite, nextafter, pi, radians, sin, sqrt, ulp
11 from random import Random
12
13 Point = tuple[float, float]
14 Circle = tuple[Point, float]
15 ARENA_RADIUS = 1800.0
16 MAX_RADIO_RADIUS = 1500.0
17 BEARING_ERROR_DEG = 1.005
18
19
20 def _check_points(points: list[Point]) -> None:
21     if any(not (isfinite(x) and isfinite(y)) for x, y in points):
22         raise ValueError("Geometry requires finite coordinates")
23
24
25 def _padding(scale: float) -> float:
26     return max(1e-9, 64.0 * ulp(max(1.0, scale)))
27
28
29 def initial_polygon(sides: int = 180) -> list[Point]:
30     """Return a CCW circumscribed polygon, never an inscribed approximation."""
31     if isinstance(sides, bool) or not isinstance(sides, int) or sides < 3:
32         raise ValueError("sides must be an integer at least three")
33     radius = (ARENA_RADIUS + _padding(ARENA_RADIUS)) / cos(pi / sides)
34     return [
35         (radius * cos(2 * pi * i / sides), radius * sin(2 * pi * i / sides))
36         for i in range(sides)
37     ]
38

```

```

39
40 def _clip_halfplane(
41     poly: list[Point], origin: Point, normal: Point, bound: float, padding: float
42 ) -> list[Point]:
43     """Clip n.(x-origin) <= bound+padding using the same shifted boundary."""
44     if not poly:
45         return []
46     nx, ny = normal
47     ox, oy = origin
48     limit = bound + padding
49     result: list[Point] = []
50     previous = poly[-1]
51     dp = nx * (previous[0] - ox) + ny * (previous[1] - oy) - limit
52     for current in poly:
53         dc = nx * (current[0] - ox) + ny * (current[1] - oy) - limit
54         if (dp <= 0.0) != (dc <= 0.0):
55             # Opposite signs prevent cancellation in dp-dc. Interpolate from
56             # the nearer endpoint to reduce cancellation near a polygon vertex.
57             if abs(dp) <= abs(dc):
58                 t = dp / (dp - dc)
59                 crossing = (
60                     previous[0] + t * (current[0] - previous[0]),
61                     previous[1] + t * (current[1] - previous[1]),
62                 )
63             else:
64                 t = dc / (dc - dp)
65                 crossing = (
66                     current[0] + t * (previous[0] - current[0]),
67                     current[1] + t * (previous[1] - current[1]),
68                 )
69             if not result or crossing != result[-1]:
70                 result.append(crossing)
71             if dc <= 0.0 and (not result or current != result[-1]):
72                 result.append(current)
73             previous, dp = current, dc
74     if len(result) > 1 and result[0] == result[-1]:
75         result.pop()
76     return result
77
78
79 def clip_bearing(
80     poly: list[Point],
81     point: Point,
82     bearing_deg: float,
83     error_deg: float = BEARING_ERROR_DEG,
84 ) -> list[Point]:
85     """Intersect a bounded region with a forward wedge and a safe range cap.
86
87     The default includes the 0.005-degree two-decimal API rounding allowance.
88     The cap is a relaxation of distance <= 1500, not an assumed known range.
89     Contradictions remain []; no fallback region or constraint reset is applied.
90     """
91     _check_points(poly)
92     _check_points([point])
93     if not isfinite(bearing_deg) or not isfinite(error_deg) or not 0 <= error_deg < 90:
94         raise ValueError("Use finite bearing and 0 <= error_deg < 90")
95     if not poly:
96         return []
97     theta = radians(bearing_deg % 360.0)
98     error = radians(error_deg)
99     low, high = theta - error, theta + error
100     scale = max(
101         MAX_RADIO_RADIUS,
102         abs(point[0]),
103         abs(point[1]),
104         *(max(abs(x), abs(y)) for x, y in poly),
105     )

```

```

106 padding = _padding(scale)
107 # cross(u_low, x-point)>=0 and cross(u_high, x-point)<=0.
108 result = _clip_halfplane(poly, point, (sin(low), -cos(low)), 0.0, padding)
109 result = _clip_halfplane(result, point, (-sin(high), cos(high)), 0.0, padding)
110 result = _clip_halfplane(result, point, (-cos(theta), -sin(theta)), 0.0, padding)
111 return _clip_halfplane(
112     result, point, (cos(theta), sin(theta)), MAX_RADIO_RADIUS, padding
113 )
114
115
116 def _farthest_pair(poly: list[Point]) -> tuple[Point, Point, float]:
117     first = second = poly[0]
118     distance = 0.0
119     for i, a in enumerate(poly):
120         for j in range(i):
121             b = poly[j]
122             candidate = hypot(a[0] - b[0], a[1] - b[1])
123             if candidate > distance:
124                 first, second, distance = a, b, candidate
125     return first, second, distance
126
127
128 def diameter(poly: list[Point]) -> float:
129     """Exact vertex-pair characterization,  $O(n^2)$ ; empty input is an error."""
130     _check_points(poly)
131     if not poly:
132         raise ValueError("An infeasible region has no localization diameter")
133     return _farthest_pair(poly)[2]
134
135
136 def _pair_circle(a: Point, b: Point) -> Circle:
137     center = (a[0] + (b[0] - a[0]) / 2, a[1] + (b[1] - a[1]) / 2)
138     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b))
139
140
141 def _cross(a: Point, b: Point, c: Point) -> float:
142     bx, by = b[0] - a[0], b[1] - a[1]
143     cx, cy = c[0] - a[0], c[1] - a[1]
144     determinant = bx * cy - by * cx
145     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
146     if abs(determinant) > 64 * ulp(max(1.0, scale * scale)):
147         return determinant
148     with localcontext() as context:
149         context.prec = 80
150         ax, ay = Decimal(a[0]), Decimal(a[1])
151         return float(
152             (Decimal(b[0]) - ax) * (Decimal(c[1]) - ay)
153             - (Decimal(b[1]) - ay) * (Decimal(c[0]) - ax)
154         )
155
156
157 def _circumcircle(a: Point, b: Point, c: Point) -> Circle | None:
158     bx, by = b[0] - a[0], b[1] - a[1]
159     cx, cy = c[0] - a[0], c[1] - a[1]
160     determinant = bx * cy - by * cx
161     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
162     if abs(determinant) <= 64 * ulp(max(1.0, scale * scale)):
163         # Decimal fallback resolves almost-collinear triples from the actual
164         # binary input values, without pretending a small determinant is zero.
165         with localcontext() as context:
166             context.prec = 80
167             axd, ayd = Decimal(a[0]), Decimal(a[1])
168             bxd, byd = Decimal(b[0]) - axd, Decimal(b[1]) - ayd
169             cxd, cyd = Decimal(c[0]) - axd, Decimal(c[1]) - ayd
170             det = 2 * (bxd * cyd - byd * cxd)
171             if not det:
172                 return None

```

```

173         bb, cc = bxd * bxd + byd * byd, cxd * cxd + cyd * cyd
174         center = (
175             float(axd + (cyd * bb - byd * cc) / det),
176             float(ayd + (bxd * cc - cxd * bb) / det),
177         )
178     else:
179         bb, cc = bx * bx + by * by, cx * cx + cy * cy
180         center = (
181             a[0] + (cy * bb - by * cc) / (2 * determinant),
182             a[1] + (bx * cc - cx * bb) / (2 * determinant),
183         )
184     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b, c))
185
186
187 def _contains(circle: Circle, point: Point) -> bool:
188     center, radius = circle
189     return hypot(point[0] - center[0], point[1] - center[1]) <= nextafter(
190         radius, float("inf")
191     )
192
193
194 def _two_boundary_circle(points: list[Point], end: int, a: Point, b: Point) -> Circle:
195     base = _pair_circle(a, b)
196     left: Circle | None = None
197     right: Circle | None = None
198     left_cross = right_cross = 0.0
199     for i in range(end):
200         p = points[i]
201         if _contains(base, p):
202             continue
203         side = _cross(a, b, p)
204         candidate = _circumcircle(a, b, p)
205         if candidate is None:
206             continue
207         center_side = _cross(a, b, candidate[0])
208         if side > 0 and (left is None or center_side > left_cross):
209             left, left_cross = candidate, center_side
210         elif side < 0 and (right is None or center_side < right_cross):
211             right, right_cross = candidate, center_side
212     if left is None:
213         return base if right is None else right
214     if right is None:
215         return left
216     return left if left[1] <= right[1] else right
217
218
219 def enclosing_circle(poly: list[Point]) -> Circle:
220     """Minimum enclosing circle, with a tiny outward floating-point guard.
221
222     Randomized incremental 1/2/3-support construction, not a diameter-disk or
223     approximate iterative fit. The local fixed shuffle never alters global RNG.
224     Empty input raises instead of manufacturing a spurious clearing location.
225     """
226     _check_points(poly)
227     if not poly:
228         raise ValueError("An infeasible region has no enclosing-circle center")
229     points = list(dict.fromkeys(poly))
230     Random(0).shuffle(points)
231     circle: Circle | None = None
232     for i, p in enumerate(points):
233         if circle is not None and _contains(circle, p):
234             continue
235         circle = (p, 0.0)
236         for j in range(i):
237             q = points[j]
238             if _contains(circle, q):
239                 continue

```

```

240         if circle[1] == 0.0:
241             circle = _pair_circle(p, q)
242         else:
243             circle = _two_boundary_circle(points, j + 1, p, q)
244     assert circle is not None
245     center = circle[0]
246     radius = max(hypot(p[0] - center[0], p[1] - center[1]) for p in points)
247     if radius == 0.0:
248         return center, 0.0
249     scale = max(radius, abs(center[0]), abs(center[1]))
250     return center, nextafter(radius + _padding(scale), float("inf"))
251
252
253 def optical_cover(poly: list[Point], radius: float = 19.9) -> list[Point]:
254     """Cover the ENTIRE convex polygon, not merely its vertices or boundary.
255
256     Use its MEC when possible; otherwise tile an enclosing rectangle aligned
257     with a diameter. Every grid cell fits in its own radius-radius disk. Some
258     centers may lie outside the polygon or arena, which the API permits.
259     [] stays []: callers must treat it as infeasibility, not successful clearing.
260     """
261     _check_points(poly)
262     if not isfinite(radius) or radius <= 0:
263         raise ValueError("Optical covering radius must be finite and positive")
264     if not poly:
265         return []
266     center, enclosing_radius = enclosing_circle(poly)
267     if enclosing_radius <= radius:
268         return [center]
269     a, b, distance = _farthest_pair(poly)
270     if distance == 0.0:
271         return [a]
272     ux, uy = (b[0] - a[0]) / distance, (b[1] - a[1]) / distance
273     # Re-normalize before forming the orthogonal frame.
274     norm = hypot(ux, uy)
275     ux, uy = ux / norm, uy / norm
276     projected = [
277         ((x - a[0]) * ux + (y - a[1]) * uy, -(x - a[0]) * uy + (y - a[1]) * ux)
278         for x, y in poly
279     ]
280     scale = max(distance, *(max(abs(x), abs(y)) for x, y in poly))
281     guard = _padding(scale)
282     safe_radius = radius - 8 * guard
283     if safe_radius <= 0:
284         raise ValueError(
285             "Radius is below floating-point resolution at these coordinates"
286         )
287     xmin, xmax = (
288         min(p[0] for p in projected) - guard,
289         max(p[0] for p in projected) + guard,
290     )
291     ymin, ymax = (
292         min(p[1] for p in projected) - guard,
293         max(p[1] for p in projected) + guard,
294     )
295     side = sqrt(2.0) * safe_radius
296     nx = max(1, ceil((xmax - xmin) / side))
297     ny = max(1, ceil((ymax - ymin) / side))
298     dx, dy = (xmax - xmin) / nx, (ymax - ymin) / ny
299     centers: list[Point] = []
300     for j in range(ny):
301         y = ymin + (j + 0.5) * dy
302         indices = range(nx) if j % 2 == 0 else range(nx - 1, -1, -1)
303         for i in indices:
304             x = xmin + (i + 0.5) * dx
305             centers.append((a[0] + x * ux - y * uy, a[1] + x * uy + y * ux))
306     return centers

```

Listing 5: 几何验证

```

1  """Independent numerical geometry checks; no official simulator calls."""
2
3  from __future__ import annotations
4
5  import itertools
6  import json
7  import math
8  import random
9  from pathlib import Path
10
11 import numpy as np
12 from scipy.optimize import linprog
13
14 from coverage import coverage_certificate, directional_waypoints, omni_waypoints
15 from geometry import (
16     clip_bearing,
17     diameter,
18     enclosing_circle,
19     initial_polygon,
20     optical_cover,
21 )
22
23
24 def contains(poly, p, tolerance=1e-6):
25     if not poly:
26         return False
27     area = sum(a[0] * b[1] - a[1] * b[0] for a, b in zip(poly, poly[1:] + poly[:1]))
28     sign = 1 if area >= 0 else -1
29     return all(
30         sign * ((b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0]))
31         >= -tolerance
32         for a, b in zip(poly, poly[1:] + poly[:1])
33     )
34
35
36 def reference_circle(points):
37     """Exhaust every 1/2/3-point support, independently of incremental MEC."""
38     candidates = [(p, 0.0) for p in points]
39     for a, b in itertools.combinations(points, 2):
40         c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
41         candidates.append((c, math.dist(a, b) / 2))
42     for a, b, c in itertools.combinations(points, 3):
43         ab, ac = np.asarray(b) - a, np.asarray(c) - a
44         matrix = 2 * np.array([ab, ac])
45         if abs(np.linalg.det(matrix)) < 1e-12:
46             continue
47         offset = np.linalg.solve(matrix, np.array([ab @ ab, ac @ ac]))
48         center = np.asarray(a) + offset
49         candidates.append((tuple(center), math.dist(center, a)))
50     feasible = [
51         (c, r)
52         for c, r in candidates
53         if all(math.dist(c, p) <= r + 1e-7 for p in points)
54     ]
55     return min(feasible, key=lambda item: item[1])
56
57
58 def main():
59     exact_ray = clip_bearing(
60         [(-100.0, 0.0), (100.0, 0.0)], (0.0, 0.0), 0.0, error_deg=0.0
61     )
62     assert min(p[0] for p in exact_ray) >= -1e-6, (
63         "Zero-error bearing retained its backward ray"
64     )
65     try:
66         directional_waypoints(1e-300)

```

```

67     except ValueError:
68         pass
69     else:
70         raise AssertionError("Extreme lattice refinement was not rejected")
71     rng = random.Random(831762)
72     max_mec_error = 0.0
73     sets = [
74         [(0.0, 0.0)],
75         [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))],
76         [(0.0, 0.0), (1.0, 1e-12), (2.0, 0.0)],
77         [(7.0, 5.0), (7.0, 5.0)],
78     ]
79     sets += [
80         [
81             (rng.uniform(-2000, 2000), rng.uniform(-2000, 2000))
82             for _ in range(rng.randint(2, 9))
83         ]
84         for _ in range(500)
85     ]
86     for points in sets:
87         actual = enclosing_circle(points)
88         expected = reference_circle(points)
89         max_mec_error = max(max_mec_error, abs(actual[1] - expected[1]))
90         assert abs(actual[1] - expected[1]) < 1e-6, (points, actual, expected)
91         assert all(math.dist(actual[0], p) <= actual[1] for p in points)
92     triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
93     polygon = initial_polygon()
94     length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
95     stations = []
96     for i, (a, b) in enumerate(zip(triangle, triangle[1:] + triangle[:1])):
97         unit = ((b[0] - a[0]) / 36, (b[1] - a[1]) / 36)
98         s = (a[0] - length * unit[0], a[1] - length * unit[1])
99         stations.append(s)
100     polygon = clip_bearing(polygon, s, 1 + 120 * i, error_deg=1.0)
101     counterexample = {
102         "stations": stations,
103         "bearings_deg": [1, 121, 241],
104         "diameter_m": diameter(polygon),
105         "mec_radius_m": enclosing_circle(polygon)[1],
106     }
107     assert abs(counterexample["diameter_m"] - 36) < 1e-6
108     assert counterexample["mec_radius_m"] > 20
109     # True-source containment including boundary source, wraparound and +/-1 noise.
110     containment_trials = 2000
111     optical_trials = 0
112     for index in range(containment_trials):
113         angle = rng.uniform(0, math.tau)
114         radius = 1800 if index % 5 == 0 else 1800 * math.sqrt(rng.random())
115         source = (radius * math.cos(angle), radius * math.sin(angle))
116         poly = initial_polygon()
117         for j in range(4):
118             r, phi = rng.uniform(6, 1500), rng.uniform(0, math.tau)
119             station = (source[0] + r * math.cos(phi), source[1] + r * math.sin(phi))
120             error = rng.choice([-1.0, 1.0]) if index % 2 else rng.uniform(-1, 1)
121             bearing = round((math.degrees(phi) + 180 + error) % 360, 2) % 360
122             poly = clip_bearing(poly, station, bearing)
123             assert contains(poly, source), (index, source, poly)
124         if index < 200:
125             centers = optical_cover(poly)
126             for _ in range(20):
127                 weights = [rng.random() for _ in poly]
128                 total = sum(weights)
129                 p = tuple(
130                     sum(w * vertex[k] for w, vertex in zip(weights, poly)) / total
131                     for k in (0, 1)
132                 )
133                 assert min(math.dist(p, c) for c in centers) <= 19.9 + 1e-7

```

```

134         optical_trials += 1
135     omni, directional = omni_waypoints(), directional_waypoints()
136     max_omni_distance, min_directional_slack = 0.0, float("inf")
137     for i in range(2000):
138         angle = rng.uniform(0, math.tau)
139         radius = 1800 if i % 2 else 1800 * math.sqrt(rng.random())
140         p = (radius * math.cos(angle), radius * math.sin(angle))
141         max_omni_distance = max(max_omni_distance, min(math.dist(p, s) for s in omni))
142         nearby = [s for s in directional if math.dist(p, s) <= 990 + 1e-7]
143         # Independent convex-hull feasibility, all orientations at once for p.
144         points = np.asarray(nearby)
145         solved = linprog(
146             np.zeros(len(points)),
147             A_eq=np.vstack([points.T, np.ones(len(points))]),
148             b_eq=np.array([*p, 1.0]),
149             bounds=(0, None),
150             method="highs",
151         )
152         assert solved.success, p
153         for _ in range(5):
154             phi = rng.uniform(0, math.tau)
155             seen = [
156                 math.dist(p, s)
157                 for s in directional
158                 if (s[0] - p[0]) * math.cos(phi) + (s[1] - p[1]) * math.sin(phi) >= 0
159             ]
160             min_directional_slack = min(min_directional_slack, 1000 - min(seen))
161     assert max_omni_distance <= 900 + 1e-7 and min_directional_slack >= 0
162     # Exact triple-lens safe region sampled as a separate numerical sanity check.
163     alpha = math.radians(1.005)
164     max_reception_excess = -float("inf")
165     for _ in range(10000):
166         t = rng.uniform(1, 999)
167         bmax = min(
168             math.sqrt(1e6 - t * t),
169             math.sqrt(1e6 - (t - 1000 * math.cos(alpha)) ** 2) - 1000 * math.sin(alpha),
170         )
171         b = rng.uniform(-1, 1) * max(0.0, bmax)
172         r, phi = rng.uniform(0, 1500), rng.uniform(-alpha, alpha)
173         excess = math.dist((t, b), (r * math.cos(phi), r * math.sin(phi))) - max(
174             1000, r
175         )
176         max_reception_excess = max(max_reception_excess, excess)
177     assert excess <= 1e-6
178     result = {
179         "evidence": "independent_numerical_checks_not_a_substitute_for_analytic_proofs",
180         "exact_forward_ray_regression": True,
181         "unbounded_refinement_rejected": True,
182         "mec_reference_cases": len(sets),
183         "maximum_mec_radius_error_m": max_mec_error,
184         "realizable_counterexample": counterexample,
185         "containment_cases": containment_trials,
186         "bearings_per_case": 4,
187         "optical_interior_samples": optical_trials,
188         "coverage_source_samples": 2000,
189         "maximum_sampled_omni_distance_m": max_omni_distance,
190         "minimum_sampled_directional_radio_slack_m": min_directional_slack,
191         "triple_lens_samples": 10000,
192         "maximum_sampled_reception_excess_m": max_reception_excess,
193         "coverage_certificate": coverage_certificate(),
194     }
195     Path("results").mkdir(exist_ok=True)
196     Path("results/geometry_verification.json").write_text(
197         json.dumps(result, ensure_ascii=False, indent=2) + "\n"
198     )
199     # The default enhanced policy uses the 1200m Q3 ring and rolling Q4
200     # skeleton. The fixed-probe/local-recovery bound is deliberately separate

```

```

201 # from the optional adaptive candidate. Reflection is about the last
202 # positive station and contracts its station distance by one half.
203 station_probe_bound = math.hypot(1501, 120)
204 source_probe_bound = 1500 + station_probe_bound
205 assert station_probe_bound < 1506 and source_probe_bound < 3006
206 max_move = 31 * 5240 + 16 * (7500 + 5 * 6020 + 4520 + 4520 + 7570)
207 max_time = max_move / 5 + 1212 * 6 + 3584 * 5
208 assert max_time < 100 * 3600 <= 360000
209 bound = {
210     "evidence": "arithmetic_of_analytic_bound_not_simulation",
211     "scope": "enhanced policy: Q3 ring1200, Q4 rolling skeleton, fixed probes, six probes",
212     "skeleton_edge_upper_m": 5240,
213     "skeleton_move_upper_m": 31 * 5240,
214     "positive_source_distance_upper_m": 1500,
215     "strip_axial_upper_m": 1501,
216     "strip_width_upper_m": 53,
217     "station_probe_bound_m": station_probe_bound,
218     "source_probe_bound_m": source_probe_bound,
219     "local_transition_upper_m": 6020,
220     "local_task_entry_upper_m": 7500,
221     "optional_center_clear_access_upper_m": 4520,
222     "optical_access_upper_m": 4520,
223     "optical_cover_centers_upper": 216,
224     "optical_snake_length_upper_m": 7570,
225     "distance_upper_m": max_move,
226     "measure_upper": 1212,
227     "clear_attempt_upper": 3584,
228     "virtual_time_upper_s": max_time,
229     "virtual_time_upper_h": max_time / 3600,
230     "proof": "report/bound_proof.tex; no guarantee for real network deadline",
231 }
232 Path("results/operation_bound.json").write_text(json.dumps(bound, indent=2) + "\n")
233 print(json.dumps(result, ensure_ascii=False, indent=2))
234
235
236 if __name__ == "__main__":
237     main()

```

Listing 6: 问题一原始交会区域分类与直径

```

1 """Q1: actual bearing-halfplane intersection, without an artificial bounding box.
2
3 Feasibility and four coordinate LPs distinguish empty/unbounded/bounded sets.
4 For a bounded set all feasible pairwise boundary intersections are enumerated.
5 This optional Q1 analysis uses SciPy; the online search policy uses stdlib only.
6 """
7
8 from __future__ import annotations
9
10 import itertools
11 import json
12 import math
13 from pathlib import Path
14
15 import numpy as np
16 from scipy.optimize import linprog
17
18 from geometry import diameter, enclosing_circle
19
20
21 def solve_intersection(stations, bearings_deg, error_deg=1.0):
22     if len(stations) != len(bearings_deg) or not 0 <= error_deg < 90:
23         raise ValueError("Match stations/readings and use an error in [0,90) degrees")
24     if not stations:
25         return {"status": "unbounded", "vertices": [], "diameter_m": None}
26     rows, rhs = [], []
27     alpha = math.radians(error_deg)

```

```

28     for (x, y), angle in zip(stations, bearings_deg):
29         if not all(math.isfinite(value) for value in (x, y, angle)):
30             raise ValueError("Finite coordinates and bearings are required")
31         theta = math.radians(angle % 360)
32         for normal in (
33             (math.sin(theta - alpha), -math.cos(theta - alpha)),
34             (-math.sin(theta + alpha), math.cos(theta + alpha)),
35             (-math.cos(theta), -math.sin(theta)),
36         ):
37             rows.append(normal)
38             rhs.append(normal[0] * x + normal[1] * y)
39     a, b = np.asarray(rows), np.asarray(rhs)
40     feasible = linprog(
41         np.zeros(2), A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
42     )
43     if feasible.status == 2:
44         return {"status": "infeasible", "vertices": [], "diameter_m": None}
45     if not feasible.success:
46         raise RuntimeError(f"Halfplane feasibility LP failed: {feasible.message}")
47     for objective in ((1.0, 0.0), (-1.0, 0.0), (0.0, 1.0), (0.0, -1.0)):
48         bound = linprog(
49             objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
50         )
51         if bound.status == 3:
52             return {"status": "unbounded", "vertices": [], "diameter_m": None}
53         if not bound.success:
54             raise RuntimeError(f"Halfplane boundedness LP failed: {bound.message}")
55     vertices = []
56     for i, j in itertools.combinations(range(len(a)), 2):
57         determinant = a[i, 0] * a[j, 1] - a[i, 1] * a[j, 0]
58         if determinant == 0:
59             continue
60         point = (
61             (b[i] * a[j, 1] - a[i, 1] * b[j]) / determinant,
62             (a[i, 0] * b[j] - b[i] * a[j, 0]) / determinant,
63         )
64         if np.max(a @ point - b) <= 1e-7 and all(
65             math.dist(point, old) > 1e-7 for old in vertices
66         ):
67             vertices.append(point)
68     if not vertices:
69         raise ArithmeticError(
70             "Bounded nonempty intersection had no numerical vertex; inspect conditioning"
71         )
72     center = tuple(np.mean(vertices, axis=0))
73     vertices.sort(
74         key=lambda point: math.atan2(point[1] - center[1], point[0] - center[0])
75     )
76     circle = enclosing_circle(vertices)
77     return {
78         "status": "bounded",
79         "vertices": vertices,
80         "diameter_m": diameter(vertices),
81         "mec_center": circle[0],
82         "mec_radius_m": circle[1],
83         "max_halfplane_residual_m": max(float(np.max(a @ p - b)) for p in vertices),
84     }
85
86
87 def main():
88     triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
89     length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
90     stations = []
91     for first, second in zip(triangle, triangle[1:] + triangle[:1]):
92         stations.append(
93             (
94                 first[0] - length * (second[0] - first[0]) / 36,

```

```

95         first[1] - length * (second[1] - first[1]) / 36,
96     )
97 )
98 bounded = solve_intersection(stations, [1, 121, 241])
99 unbounded = solve_intersection([(0.0, 0.0)], [0.0])
100 infeasible = solve_intersection([(0.0, 0.0), (-10.0, 0.0)], [0.0, 180.0])
101 assert bounded["status"] == "bounded" and abs(bounded["diameter_m"] - 36) < 1e-6
102 assert unbounded["status"] == "unbounded" and infeasible["status"] == "infeasible"
103 result = {
104     "evidence": "Q1_direct_halfplane_algorithm_examples",
105     "bounded": bounded,
106     "unbounded": unbounded,
107     "infeasible": infeasible,
108 }
109 Path("results").mkdir(exist_ok=True)
110 Path("results/intersection_verification.json").write_text(
111     json.dumps(result, indent=2) + "\n"
112 )
113 print(json.dumps(result, indent=2))
114
115
116 if __name__ == "__main__":
117     main()

```

Listing 7: 覆盖与采样工具

```

1  """Finite, analytically justified discovery waypoint sets for CUMCM B Q3/Q4.
2
3  These functions prescribe measurement locations, not a route or a stopping rule.
4  Every unresolved channel needs its own completed coverage record. In particular,
5  Q4 waypoints outside the source arena must not be projected back into the arena.
6  """
7
8  from __future__ import annotations
9
10 import math
11
12 Point = tuple[float, float]
13
14 ARENA_RADIUS = 1800.0
15 MIN_RADIO_RADIUS = 1000.0
16 OMNI_COVER_RADIUS = 900.0
17 DEFAULT_SPACING = 990.0
18
19
20 def omni_waypoints(ring_radius: float | None = None) -> list[Point]:
21     """Return origin plus six ring points; the legacy radius certifies 900 m.
22
23     The 1200-m contracted ring instead has exact cover radius given by
24     ring_cover_radius, still below the guaranteed 1000-m reception distance.
25     No unspecified source-position prior is used by this geometric guarantee.
26     """
27     ring_radius = OMNI_COVER_RADIUS * math.sqrt(3.0) if ring_radius is None else float(ring_radius)
28     if not math.isfinite(ring_radius) or not 1200.0 <= ring_radius <= OMNI_COVER_RADIUS * math.sqrt(3.0):
29         raise ValueError("Supported ring radius is [1200, 900*sqrt(3)] m")
30     return [(0.0, 0.0)] + [
31         (
32             ring_radius * math.cos(k * math.pi / 3.0),
33             ring_radius * math.sin(k * math.pi / 3.0),
34         )
35         for k in range(6)
36     ]
37
38
39 def ring_cover_radius(ring_radius: float) -> float:
40     """Exact covering radius of origin plus six equally spaced ring points."""
41     omni_waypoints(ring_radius)

```

```

42     return max(ring_radius / math.sqrt(3.0),
43                math.sqrt(ARENA_RADIUS**2 + ring_radius**2
44                          - math.sqrt(3.0) * ARENA_RADIUS * ring_radius))
45
46
47 def directional_waypoints(spacing: float = DEFAULT_SPACING) -> list[Point]:
48     """Return a finite triangular lattice with a full source-domain guard.
49
50     For a source g inside the arena, the elementary triangle containing g has
51     vertices within spacing of g and norm <= ARENA_RADIUS + spacing.
52     If g is a lattice vertex, all six nearest neighbours satisfy that same
53     norm bound, so every emission direction has a nonzero forward neighbour.
54     No such six-neighbour claim is made for vertices outside the source arena.
55     Support 900 <= spacing <= 999; ordering is origin then row and column.
56     """
57     spacing = float(spacing)
58     if not math.isfinite(spacing) or not 900.0 <= spacing <= MIN_RADIO_RADIUS - 1.0:
59         raise ValueError("spacing must be finite and in [900, 999] m")
60
61     # With a=(s,0), b=(s/2,s*sqrt(3)/2), the squared norm of i*a+j*b
62     # is s*s*(i*i+i*j+j*j). Compare the radial cutoff using integers, so
63     # floating-point rounding cannot accidentally remove a boundary vertex.
64     numerator, denominator = spacing.as_integer_ratio()
65     guard_numerator = 1800 * denominator + numerator
66     numerator_squared = numerator * numerator
67     guard_squared = guard_numerator * guard_numerator
68     # q=i*i+i*j+j*j >= 3*i*i/4 and >= 3*j*j/4. The simpler bound
69     # |i|,|j| <= ceil(2*(1800+s)/s) safely contains the radial selection.
70     bound = (2 * guard_numerator + numerator - 1) // numerator
71     row_height = spacing * math.sqrt(3.0) / 2.0
72     points: list[Point] = [(0.0, 0.0)]
73     for j in range(-bound, bound + 1):
74         for i in range(-bound, bound + 1):
75             if i == 0 and j == 0:
76                 continue
77             norm_squared_units = i * i + i * j + j * j
78             if norm_squared_units * numerator_squared <= guard_squared:
79                 points.append((spacing * (i + j / 2.0), row_height * j))
80     return points
81
82
83 def coverage_certificate() -> dict:
84     """Return analytic constants/counts, not a sampled or experimental result.
85
86     Counts describe the default 990 m lattice. Norm shells are exact integer
87     values of i*i+i*j+j*j; their multiplicities can be checked independently.
88     No simulator actions, numerical coverage sampling, or routing occur here.
89     """
90     return {
91         "certificate_kind": "analytic_geometric_cover",
92         "arena_radius_m": ARENA_RADIUS,
93         "minimum_radio_radius_m": MIN_RADIO_RADIUS,
94         "channels": 20,
95         "omni_legacy": {
96             "waypoint_count": 7,
97             "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
98             "cover_radius_m": OMNI_COVER_RADIUS,
99             "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
100             "proof": (
101                 "r<=900 is covered by the origin. For 900<=r<=1800, "
102                 "choose a ring center with angular gap <=pi/6. Then "
103                 "distance^2<=r^2-2700*r+2430000<=810000 by convexity "
104                 "and equality of the two endpoint values."
105             ),
106         },
107         "omni_enhanced": {
108             "waypoint_count": 7,

```

```

109         "ring_radius_m": 1200.0,
110         "cover_radius_m": ring_cover_radius(1200.0),
111         "radio_slack_m": MIN_RADIO_RADIUS - ring_cover_radius(1200.0),
112         "proof": "Exact envelope formula ring_cover_radius(1200); no point or channel obligation is removed
113     },
114     "omni": {
115         "waypoint_count": 7,
116         "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
117         "cover_radius_m": OMNI_COVER_RADIUS,
118         "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
119         "proof": "Legacy 900m certificate; enhanced 1200m certificate is in omni_enhanced.",
120     },
121     "directional": {
122         "spacing_m": DEFAULT_SPACING,
123         "basis_m": [
124             [DEFAULT_SPACING, 0.0],
125             [DEFAULT_SPACING / 2.0, DEFAULT_SPACING * math.sqrt(3.0) / 2.0],
126         ],
127         "selection_radius_m": ARENA_RADIUS + DEFAULT_SPACING,
128         "maximum_source_vertex_distance_m": DEFAULT_SPACING,
129         "radio_slack_m": MIN_RADIO_RADIUS - DEFAULT_SPACING,
130         "waypoint_count": 31,
131         "inside_or_on_arena_count": 13,
132         "outside_arena_count": 18,
133         "norm_squared_units_counts": {0: 1, 1: 6, 3: 6, 4: 6, 7: 12},
134         "maximum_waypoint_radius_m": DEFAULT_SPACING * math.sqrt(7.0),
135         "closed_halfplane_required": True,
136         "boundary_guard_required": True,
137         "proof": (
138             "An elementary triangular-lattice triangle containing g has "
139             "three vertices within spacing of g. Their norms are at most "
140             "1800+spacing, hence all are selected. Write "
141             "g=sum(lambda_i*v_i), lambda_i>=0, sum(lambda_i)=1. "
142             "For any emission normal n, sum(lambda_i*n dot (v_i-g))=0, "
143             "so at least one vertex is in the closed emission halfplane. "
144             "If g is a lattice vertex inside the source arena, all six "
145             "nearest neighbors have norm <=1800+spacing and are selected; "
146             "one makes angle <=pi/6 with any emission normal."
147         ),
148         "count_proof": (
149             "At spacing 990 the cutoff is i^2+i*j+j^2<=961/121. "
150             "The possible shells are 0,1,3,4,7 with multiplicities "
151             "1,6,6,6,12. The arena cutoff is 400/121, leaving shells "
152             "0,1,3 inside and shells 4,7 outside."
153         ),
154     },
155     "termination": (
156         "For each of 20 channels, retain either a confirmed successful "
157         "clear or a complete all-no_signal covering scan for that channel. "
158         "A detected but uncleared channel remains unresolved. With at "
159         "most one source per channel, all 20 certificates imply no "
160         "remaining source. Independently, 16 distinct confirmed clears "
161         "reach the stated global upper bound. Fewer clears alone do not."
162     ),
163     "no_signal": {
164         "q3": "Excludes every source at distance <=1000 from that detector.",
165         "q4": (
166             "Does not exclude that disk: a source may emit away from the "
167             "detector. Only the completed per-channel halfplane cover "
168             "excludes every source position and orientation."
169         ),
170     },
171     "assumptions": [
172         "Sources are stationary, persist until cleared, and use distinct channels.",
173         "Directional emission is a closed 180-degree halfplane with fixed orientation.",
174         "Reception reaches at least 1000 m throughout the emission halfplane.",

```

```

175         "All designated measurements actually complete on the indicated channel.",
176         "Near-source responses count as detections, not as no_signal.",
177         "Robot coordinates outside the radius-1800 source arena are permitted.",
178         "Geometric proofs use exact real coordinates; default float coordinates have 10 m range slack.",
179     ],
180 }

```

Listing 8: 环境与模拟器接口

```

1  """Synthetic, not official, CUMCM 2026 B environment (standard library only).
2
3  Generation assumptions are deliberately explicit, not claims about official data:
4  * random: N is discrete uniform on 10..16; N distinct channels sampled uniformly
5    without replacement; positions iid uniform in the radius-1800 disk (sqrt(U)
6    radial law); radio radii iid uniform [1000,1500]; orientations uniform [0,2*pi).
7    Q3 is all omni. Q4 chooses a uniform count in 1..N-1 and a uniform subset
8    of directional sources, guaranteeing both types.
9  * minimum_radius: random geometry, but every radio radius is exactly 1000.
10 * outward_boundary: 16 equally spaced sources on the arena boundary, a seeded
11   common rotation, all radii 1000. Q4 has 15 outward-facing directional sources
12   and one omni, so inward-only search is deliberately inadequate.
13 * clustered: 16 sources uniform in a 35m disk around a seeded center at radius
14   1650; all radio radii 1000, with the random mixed-type rule for Q4.
15
16 Errors are deterministic functions of (seed, channel, exact coordinate): smooth
17 is a bounded two-sinusoid field; extreme is spatial-hashed +/-1 degree;
18 iid_location is spatial-hashed uniform [-1,1], NOT independent repeat readings.
19 The latter is a synthetic hash field, not an assertion of official IID noise.
20 Sources and their parameters are private. Only post-exit summary reveals counts.
21 There are no virtual-duration sleeps; movement time is rounded to microseconds.
22 """
23
24 from __future__ import annotations
25
26 import hashlib
27 import math
28 import random
29 import struct
30 import threading
31 import time
32 from dataclasses import dataclass
33
34 Point = tuple[float, float]
35 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
36 ERROR_MODES = ("smooth", "extreme", "iid_location")
37
38
39 def _action_args(path: str, position: Point | None, channel: int | None):
40     """Validate before any state mutation; match protocol coordinate limits."""
41     if path not in ("/enter", "/exit", "/measure", "/clear"):
42         raise ValueError("Unknown action path (paths must be exact)")
43     if path in ("/enter", "/exit"):
44         if position is not None or channel is not None:
45             raise ValueError("enter/exit do not accept position or channel")
46         return None, None
47     if not isinstance(position, (tuple, list)) or len(position) != 2:
48         raise ValueError("position must contain exactly two finite coordinates")
49     if any(isinstance(v, bool) or not isinstance(v, (int, float)) for v in position):
50         raise ValueError("Coordinates must be JSON numbers")
51     try:
52         point = (float(position[0]), float(position[1]))
53     except OverflowError as exc:
54         raise ValueError("Coordinate outside protocol range") from exc
55     if any(not math.isfinite(v) or abs(v) > 2_000_000 for v in point):
56         raise ValueError("Coordinates must be finite and within +/-2000000m")
57     if (
58         isinstance(channel, bool)

```

```

59         or not isinstance(channel, (int, float))
60         or not 1 <= channel <= 20
61         or int(channel) != channel
62     ):
63         raise ValueError("channel must be an integer in 1..20")
64     return point, int(channel)
65
66
67 @dataclass(slots=True)
68 class _Source:
69     position: Point
70     radius: float
71     normal: Point | None
72     phase: float
73     cleared: bool = False
74
75
76 class LocalSimulator:
77     """One local test session; /enter initializes state, duplicate entry rejects.
78
79     log contains only observable request/response/timing data. After termination,
80     construct a new instance for another session, just as the official interface
81     requires a new GUI test. Automatic deadline termination closes this session;
82     no fabricated /exit response is generated in that case.
83     """
84
85     def __init__(
86         self,
87         seed: int,
88         problem: int = 3,
89         error_mode: str = "smooth",
90         scenario: str = "random",
91     ):
92         if isinstance(seed, bool) or not isinstance(seed, int):
93             raise TypeError("seed must be an integer")
94         if problem not in (3, 4):
95             raise ValueError("problem must be 3 or 4")
96         if error_mode not in ERROR_MODES or scenario not in SCENARIOS:
97             raise ValueError(f"Use scenarios {SCENARIOS} and modes {ERROR_MODES}")
98         self._seed, self._problem = seed, problem
99         self._error_mode, self._scenario = error_mode, scenario
100         self._lock = threading.Lock()
101         self._status = "ready"
102         self._sources: dict[int, _Source] = {}
103         self._position: Point = (0.0, 0.0)
104         self._radio_channel = 1
105         self._virtual_us = 0
106         self._distance = 0.0
107         self._measures = self._switches = self._clear_attempts = self._failed_clears = 0
108         self._started: float | None = None
109         self._finished: float | None = None
110         self._exit_reason: str | None = None
111         self.log: list[dict] = []
112
113     def _generate(self):
114         rng = random.Random(self._seed)
115         n = (
116             16
117             if self._scenario in ("outward_boundary", "clustered")
118             else rng.randint(10, 16)
119         )
120         channels = rng.sample(range(1, 21), n)
121         if self._problem == 3:
122             directional = set()
123         elif self._scenario == "outward_boundary":
124             directional = set(channels[:-1])
125         else:

```

```

126         directional = set(rng.sample(channels, rng.randint(1, n - 1)))
127     rotation = rng.uniform(0, math.tau)
128     center = (1650 * math.cos(rotation), 1650 * math.sin(rotation))
129     sources = {}
130     for i, channel in enumerate(channels):
131         theta = rng.uniform(0, math.tau)
132         if self._scenario == "outward_boundary":
133             theta = rotation + math.tau * i / n
134             position = (1800 * math.cos(theta), 1800 * math.sin(theta))
135         elif self._scenario == "clustered":
136             r = 35 * math.sqrt(rng.random())
137             position = (
138                 center[0] + r * math.cos(theta),
139                 center[1] + r * math.sin(theta),
140             )
141         else:
142             r = 1800 * math.sqrt(rng.random())
143             position = (r * math.cos(theta), r * math.sin(theta))
144             radius = rng.uniform(1000, 1500) if self._scenario == "random" else 1000.0
145             orientation = (
146                 theta
147                 if self._scenario == "outward_boundary"
148                 else rng.uniform(0, math.tau)
149             )
150             normal = (
151                 (math.cos(orientation), math.sin(orientation))
152                 if channel in directional
153                 else None
154             )
155             sources[channel] = _Source(
156                 position, radius, normal, rng.uniform(0, math.tau)
157             )
158     self._sources = sources
159
160     def _bearing_error(self, point: Point, channel: int, source: _Source) -> float:
161         x, y = point
162         if self._error_mode == "smooth":
163             return 0.6 * math.sin(x / 173 + y / 251 + source.phase) + 0.4 * math.sin(
164                 x / 419 - y / 137 + 2 * source.phase
165             )
166         # Canonicalize signed zero: it is the same physical coordinate.
167         packed = struct.pack("!ddB", x or 0.0, y or 0.0, channel)
168         salt = str(self._seed).encode("ascii") + b":"
169         value = int.from_bytes(
170             hashlib.blake2b(salt + packed, digest_size=8).digest(), "big"
171         )
172         if self._error_mode == "extreme":
173             return 1.0 if value & 1 else -1.0
174         return 2 * (value / ((1 << 64) - 1)) - 1
175
176     def _visible(self, source: _Source, point: Point) -> bool:
177         dx, dy = point[0] - source.position[0], point[1] - source.position[1]
178         if math.hypot(dx, dy) > source.radius:
179             return False
180         if source.normal is None:
181             return True
182         a, b = dx * source.normal[0], dy * source.normal[1]
183         # Include the closed halfplane boundary despite floating-point dot error;
184         # tolerance is only a handful of machine ulps, not an angular margin.
185         return a + b >= -8 * math.ulp(max(abs(a), abs(b), 1.0))
186
187     def _response(self, accepted: bool = True, **fields) -> dict:
188         return {
189             "accepted": accepted,
190             "real_timestamp_ms": int(time.time() * 1000),
191             "virtual_time_s": self._virtual_us / 1_000_000 if accepted else 0,
192             **fields,

```

```

193     }
194
195     def _finish(self, reason: str):
196         self._status = "exited"
197         self._finished = time.monotonic()
198         self._exit_reason = reason
199
200     def act(
201         self, path: str, position: Point | None = None, channel: int | None = None
202     ) -> dict:
203         position, channel = _action_args(path, position, channel)
204         with self._lock:
205             if self._status == "exited":
206                 raise ConnectionError(
207                     "Local session interface closed after exit/timeout"
208                 )
209             if self._started is not None and time.monotonic() >= self._started + 1200:
210                 self._finish("real_timeout")
211                 raise ConnectionError("Local session reached its real deadline")
212             request = {"path": path}
213             if position is not None:
214                 request.update(
215                     position={"x": position[0], "y": position[1]}, channel=channel
216                 )
217             before = self._virtual_us
218             radio_before = self._radio_channel
219             distance = 0.0
220             switched = False
221             action_seconds = 0
222             if path == "/enter":
223                 if self._status != "ready":
224                     response = self._response(False)
225                 else:
226                     self._generate()
227                     self._position, self._radio_channel = (0.0, 0.0), 1
228                     self._virtual_us = 0
229                     self._started = time.monotonic()
230                     self._status = "active"
231                     response = self._response(
232                         max_virtual_duration_s=360000,
233                         max_real_duration_s=1200,
234                         remaining_real_duration_s=1200,
235                     )
236                 elif self._status != "active":
237                     response = self._response(False)
238                 elif path == "/exit":
239                     self._finish("user_exit")
240                     response = self._response(exit_reason="user_exit")
241                 else:
242                     distance = math.dist(self._position, position)
243                     source = self._sources.get(channel)
244                     live = source is not None and not source.cleared
245                     if path == "/measure":
246                         switched = channel != self._radio_channel
247                         self._radio_channel = channel
248                         self._switches += int(switched)
249                         self._measures += 1
250                         action_seconds = 5
251                         fields = {"measure_result": "no_signal"}
252                         if live and self._visible(source, position):
253                             if math.dist(position, source.position) <= 5:
254                                 fields = {"measure_result": "near"}
255                             else:
256                                 angle = math.degrees(
257                                     math.atan2(
258                                         source.position[1] - position[1],
259                                         source.position[0] - position[0],

```

```

260         )
261     )
262     bearing = (
263         round(
264             (
265                 angle
266                 + self._bearing_error(position, channel, source)
267             )
268             % 360,
269             2,
270         )
271         % 360
272     )
273     fields = {"measure_result": "direction", "svd_deg": bearing}
274 else:
275     self._clear_attempts += 1
276     success = live and math.dist(position, source.position) <= 20
277     if success:
278         source.cleared = True
279     else:
280         self._failed_clears += 1
281         action_seconds = 5 if success else 3
282         fields = {
283             "clear_result": "success" if success else "no_target_in_range"
284         }
285     self._position = position
286     self._distance += distance
287     self._virtual_us += (
288         round(distance / 5 * 1_000_000)
289         + (action_seconds + int(switched)) * 1_000_000
290     )
291     response = self._response(**fields)
292     if self._virtual_us >= 360000 * 1_000_000:
293         # An action registered before the deadline is allowed to finish.
294         self._finish("virtual_timeout")
295     self.log.append(
296         {
297             "request": request,
298             "response": dict(response),
299             "distance_m": distance,
300             "move_time_s": round(distance / 5, 6),
301             "switch_time_s": int(switched),
302             "action_time_s": action_seconds,
303             "radio_channel_before": radio_before,
304             "radio_channel_after": self._radio_channel,
305             "action_virtual_time_s": (self._virtual_us - before) / 1_000_000,
306         }
307     )
308     return response
309
310 def summary(self) -> dict:
311     """Evaluation only: inaccessible during a run; never consumed by planner."""
312     with self._lock:
313         if self._status != "exited":
314             raise RuntimeError("summary is available only after session exit")
315         n = len(self._sources)
316         cleared = sum(source.cleared for source in self._sources.values())
317         virtual_time = self._virtual_us / 1_000_000
318         directional = sum(
319             source.normal is not None for source in self._sources.values()
320         )
321     return {
322         "evidence": "local_synthetic_not_official",
323         "seed": self._seed,
324         "problem": self._problem,
325         "scenario": self._scenario,
326         "error_mode": self._error_mode,

```

```

327         "n_sources": n,
328         "n_cleared": cleared,
329         "n_omnidirectional": n - directional,
330         "n_directional": directional,
331         "fraction_cleared": cleared / n,
332         "virtual_time_s": virtual_time,
333         "average_time_s": virtual_time / cleared if cleared else None,
334         "distance_m": self._distance,
335         "measures": self._measures,
336         "switches": self._switches,
337         "clear_attempts": self._clear_attempts,
338         "failed_clears": self._failed_clears,
339         "real_duration_s": self._finished - self._started,
340         "exit_reason": self._exit_reason,
341     }

```

Listing 9: 第二交点计算

```

1  """Q2 robust reception lens and explicitly sampled min-max geometry scoring."""
2
3  from __future__ import annotations
4
5  import json
6  import math
7  from pathlib import Path
8
9  from geometry import clip_bearing, enclosing_circle, initial_polygon
10
11
12  def safe_candidates(station=(0.0, 0.0), bearing_deg=0.0, alpha_deg=1.005):
13      alpha, angle = math.radians(alpha_deg), math.radians(bearing_deg)
14      u, n = (math.cos(angle), math.sin(angle)), (-math.sin(angle), math.cos(angle))
15      choices = []
16      for t in (125.0, 250.0, 375.0, 500.0, 625.0, 750.0, 875.0):
17          bound = min(
18              math.sqrt(1000**2 - t * t),
19              math.sqrt(1000**2 - (t - 1000 * math.cos(alpha)) ** 2)
20              - 1000 * math.sin(alpha),
21          )
22          for proportion in (0.25, 0.5, 0.75):
23              for sign in (-1, 1):
24                  b = sign * proportion * bound
25                  point = (
26                      station[0] + t * u[0] + b * n[0],
27                      station[1] + t * u[1] + b * n[1],
28                  )
29                  choices.append({"point": point, "t_m": t, "b_m": b})
30      return choices
31
32
33  def example():
34      """Normalized illustrative station/readout, not a supplied official case.
35
36      Safe reception follows the analytic lens theorem; sampled ranking does not
37      certify continuous worst-case optimality. No prior distribution is assumed.
38      """
39      first = clip_bearing(initial_polygon(), (0.0, 0.0), 0.0)
40      source_samples = [
41          (r * math.cos(math.radians(phi)), r * math.sin(math.radians(phi)))
42          for r in range(25, 1501, 25)
43          for phi in (-1.0, 0.0, 1.0)
44      ]
45      output = []
46      choices = safe_candidates() + [{"point": (750.0, 0.0), "t_m": 750.0, "b_m": 0.0}]
47      for candidate in choices:
48          q = candidate["point"]
49          worst_radius = 0.0

```

```

50     for source in source_samples:
51         assert math.dist(q, source) <= max(1000, math.dist((0, 0), source)) + 1e-7
52         if math.dist(q, source) <= 5:
53             radius = 5.0
54         else:
55             truth = math.degrees(math.atan2(source[1] - q[1], source[0] - q[0]))
56             radius = 0.0
57             for error in (-1.0, 0.0, 1.0):
58                 reading = round((truth + error) % 360, 2) % 360
59                 poly = clip_bearing(first, q, reading)
60                 radius = max(radius, enclosing_circle(poly)[1])
61             worst_radius = max(worst_radius, radius)
62         output.append(
63             {
64                 **candidate,
65                 "sampled_worst_mec_radius_m": worst_radius,
66                 "first_move_and_measure_s": math.hypot(*q) / 5 + 5,
67             }
68         )
69     best = min(
70         output,
71         key=lambda row: (
72             row["sampled_worst_mec_radius_m"],
73             row["first_move_and_measure_s"],
74         ),
75     )
76     result = {
77         "evidence": "illustrative_Q2_grid_score_not_global_optimum",
78         "first_station": [0, 0],
79         "first_reading_deg": 0,
80         "source_samples": len(source_samples),
81         "second_error_values_deg": [-1, 0, 1],
82         "best_sampled_geometry": best,
83         "collinear_control": output[-1],
84         "candidates": output,
85     }
86     Path("results").mkdir(exist_ok=True)
87     Path("results/second_point_example.json").write_text(
88         json.dumps(result, indent=2) + "\n"
89     )
90     print(json.dumps({k: v for k, v in result.items() if k != "candidates"}, indent=2))
91
92
93 if __name__ == "__main__":
94     example()

```

Listing 10: 恢复策略比较

```

1 import json
2 import statistics
3 from pathlib import Path
4
5 from environment import LocalSimulator
6 from planner import Planner
7
8 rows, summary = [], {}
9 for scenario in ("random", "minimum_radius", "outward_boundary", "clustered"):
10     for enabled in (False, True):
11         name = "recovery" if enabled else "two_flanks"
12         group = []
13         for seed in range(500000, 500100):
14             env = LocalSimulator(seed, 4, "smooth", scenario)
15             result = Planner(env, 4, strategy="integrated", loss_recovery=enabled).run()
16             actual = env.summary()
17             assert actual["fraction_cleared"] == 1
18             group.append(actual)
19             rows.append(**actual, "policy": name)

```

```

20     summary[f"{scenario}_{name}"] = {
21         "runs": len(group),
22         "mean_average_time_s": statistics.mean(r["average_time_s"] for r in group),
23         "mean_failed_clears": statistics.mean(r["failed_clears"] for r in group),
24         "worst_virtual_time_s": max(r["virtual_time_s"] for r in group),
25     }
26 result = {
27     "evidence": "synthetic_development_comparison_not_final_holdout",
28     "summary": summary,
29     "runs": rows,
30 }
31 Path("results/recovery_development.json").write_text(
32     json.dumps(result, indent=2) + "\n"
33 )
34 print(json.dumps(summary, indent=2))

```

Listing 11: HTTP 客户端

```

1  """Serial, append-only-logged HTTP adapter for an already-open simulator test.
2
3  This module does NOT authenticate, launch a GUI, or start a formal test. The user
4  must first start the intended test in the official GUI and wait for readiness.
5  Only explicit act calls cause simulator actions; close never sends /exit.
6
7  A new logical action receives a UUID. Transient retries preserve exactly the
8  same bytes, path, and request ID. Exhausted ambiguous actions fail closed: no
9  new action can be submitted on this client because the previous action may have
10 executed. Logs are ordinary plaintext observations, NOT official encrypted logs.
11 Real sleeps are used solely for short transport-retry backoff, never virtual time.
12 """
13
14 from __future__ import annotations
15
16 import http.client
17 import json
18 import math
19 import queue
20 import threading
21 import time
22 import unicodedata
23 import uuid
24 from pathlib import Path
25 from urllib.error import HTTPError, URLError
26 from urllib.parse import urlsplit
27 from urllib.request import HTTPRedirectHandler, ProxyHandler, Request, build_opener
28
29 from environment import Point, _action_args
30
31
32 class ClientError(RuntimeError):
33     """An action did not produce a verified accepted response."""
34
35
36 class RejectedActionError(ClientError):
37     def __init__(self, status: int, response: dict):
38         self.status, self.response = status, response
39         super().__init__(
40             f"Simulator rejected action: HTTP {status}, accepted={response.get('accepted')!r}"
41         )
42
43
44 class AmbiguousActionError(ClientError):
45     """The action may have executed; consult the log and simulator GUI."""
46
47
48 class ClientProtocolError(ClientError):
49     pass

```

```

50
51
52 class DeadlineExceeded(ClientError):
53     pass
54
55
56 class _NoRedirect(HTTPRedirectHandler):
57     def redirect_request(self, req, fp, code, msg, headers, newurl):
58         # Never let urllib turn a POST into a GET or send identifiers elsewhere.
59         return None
60
61
62 def _number(value) -> bool:
63     return (
64         not isinstance(value, bool)
65         and isinstance(value, (int, float))
66         and math.isfinite(value)
67     )
68
69
70 def _unique_object(pairs):
71     result = {}
72     for key, value in pairs:
73         if key in result:
74             raise ValueError(f"Duplicate response key: {key}")
75         result[key] = value
76     return result
77
78
79 class HTTPClient:
80     """Official wire schema, no truth API, one in-flight logical action at a time.
81
82     Cached public position/radio_channel/virtual_time_s describe only confirmed
83     accepted actions. The real deadline uses /enter.remaining_real_duration_s,
84     conservatively subtracting all elapsed time since its first send, including
85     transport retries. Before entry, a bounded 20-second connection budget applies
86     because the actual GUI window deadline is not available through the API.
87     """
88
89     _MAX_ATTEMPTS = 4
90     _REQUEST_TIMEOUT_S = 5.0
91     _ACTION_RETRY_BUDGET_S = 20.0
92     _MAX_RESPONSE_BYTES = 65536
93     _TRANSIENT_HTTP = frozenset((429, 500, 502, 503, 504))
94
95     def __init__(self, base_url: str, robot_id: str, log_path: str):
96         parsed = urlsplit(base_url)
97         if (
98             parsed.scheme not in ("http", "https")
99             or not parsed.hostname
100             or parsed.path not in ("", "/")
101             or parsed.query
102             or parsed.fragment
103             or parsed.username is not None
104             or parsed.password is not None
105         ):
106             raise ValueError(
107                 "base_url must be an HTTP(S) origin, without credentials/path/query"
108             )
109         # Force port validation before sending anything.
110         _ = parsed.port
111         if (
112             not isinstance(robot_id, str)
113             or not 1 <= len(robot_id.encode("utf-8")) <= 64
114         ):
115             raise ValueError("robot_id must contain 1..64 UTF-8 bytes")
116         if any(unicodedata.category(c) in ("Cc", "Cf") for c in robot_id):

```

```

117         raise ValueError("robot_id cannot contain control or format characters")
118     self._base_url, self._robot_id = base_url.rstrip("/"), robot_id
119     self._lock = threading.Lock()
120     self._opener = build_opener(ProxyHandler({}), _NoRedirect())
121     self._closed = False
122     self._uncertain = False
123     self._entered = self._exited = False
124     self._deadline: float | None = None
125     self._max_virtual_duration_s: float | None = None
126     self.position: Point = (0.0, 0.0)
127     self.radio_channel = 1
128     self.virtual_time_s = 0.0
129     self._log = Path(log_path).open("a", encoding="utf-8", buffering=1) # noqa: SIM115 - lifecycle closed
    by close()

130
131 @property
132 def remaining_real_duration_s(self) -> float | None:
133     """Locally conservative seconds left; unknown until accepted /enter."""
134     return (
135         None
136         if self._deadline is None
137         else max(0.0, self._deadline - time.monotonic())
138     )
139
140 def _record(self, event: dict):
141     self._log.write(
142         json.dumps(
143             {"logged_at_ms": int(time.time() * 1000), **event},
144             ensure_ascii=False,
145             allow_nan=False,
146             separators=(",", ":"),
147         )
148         + "\n"
149     )
150     self._log.flush()
151
152 def _decode(self, raw: bytes) -> dict:
153     if len(raw) > self._MAX_RESPONSE_BYTES:
154         raise ClientProtocolError("Response exceeds bounded JSON body size")
155     try:
156         response = json.loads(raw.decode("utf-8"), object_pairs_hook=_unique_object)
157     except (UnicodeDecodeError, ValueError) as exc:
158         raise ClientProtocolError(
159             "Response is not valid unambiguous UTF-8 JSON"
160         ) from exc
161     if not isinstance(response, dict) or type(response.get("accepted")) is not bool:
162         raise ClientProtocolError("Response lacks a boolean accepted field")
163     if (
164         not _number(response.get("real_timestamp_ms"))
165         or not _number(response.get("virtual_time_s"))
166         or response["virtual_time_s"] < 0
167     ):
168         raise ClientProtocolError("Response lacks valid numeric timestamps")
169     return response
170
171 def _validate_accepted(self, path: str, response: dict):
172     virtual_time = response["virtual_time_s"]
173     if path == "/enter":
174         remaining = response.get("remaining_real_duration_s")
175         if (
176             not _number(remaining)
177             or not 0 <= remaining <= 1200
178             or int(remaining) != remaining
179         ):
180             raise ClientProtocolError("Invalid actual remaining real duration")
181         if (
182             not _number(response.get("max_virtual_duration_s"))

```

```

183         or response["max_virtual_duration_s"] <= 0
184         or not _number(response.get("max_real_duration_s"))
185         or response["max_real_duration_s"] <= 0
186         or virtual_time != 0
187     ):
188         raise ClientProtocolError(
189             "Invalid accepted entry limits or initial time"
190         )
191     elif virtual_time < self.virtual_time_s:
192         raise ClientProtocolError("Accepted action moved virtual time backwards")
193     if path == "/measure":
194         result = response.get("measure_result")
195         if result not in ("near", "no_signal", "direction"):
196             raise ClientProtocolError("Unknown measure result")
197         if result == "direction":
198             angle = response.get("svd_deg")
199             if not _number(angle) or not 0 <= angle < 360:
200                 raise ClientProtocolError("Invalid direction angle")
201         elif "svd_deg" in response:
202             raise ClientProtocolError("Bearing supplied without direction result")
203     elif path == "/clear" and response.get("clear_result") not in (
204         "success",
205         "no_target_in_range",
206     ):
207         raise ClientProtocolError("Unknown clear result")
208     elif path == "/exit" and response.get("exit_reason") != "user_exit":
209         raise ClientProtocolError("Unexpected successful exit reason")
210
211     def _commit(
212         self,
213         path: str,
214         position: Point | None,
215         channel: int | None,
216         response: dict,
217         first_send: float,
218     ):
219         if path == "/enter":
220             self._entered = True
221             self.position, self.radio_channel = (0.0, 0.0), 1
222             self._deadline = first_send + response["remaining_real_duration_s"]
223             self._max_virtual_duration_s = response["max_virtual_duration_s"]
224         elif path == "/exit":
225             self._exited = True
226         else:
227             self.position = position
228             if path == "/measure":
229                 self.radio_channel = channel
230             self.virtual_time_s = response["virtual_time_s"]
231             self._uncertain = False
232
233     def _exchange(self, request: Request, deadline: float) -> tuple[int, bytes]:
234         """Bound connection, headers and body together, not just idle sockets.
235
236         On expiry the daemon may finish the same in-flight request, but cannot
237         mutate client state or send another action. act remains fail-closed.
238         """
239         completed = queue.Queue(maxsize=1)
240
241         def receive():
242             try:
243                 available = deadline - time.monotonic()
244                 if available <= 0:
245                     raise TimeoutError("Absolute HTTP deadline elapsed")
246             try:
247                 stream = self._opener.open(
248                     request, timeout=min(self._REQUEST_TIMEOUT_S, available)
249                 )

```

```

250         except HTTPError as exc:
251             stream = exc
252         with stream:
253             status = stream.code
254             expected_length = stream.headers.get("Content-Length")
255             if expected_length is not None:
256                 try:
257                     expected_length = int(expected_length)
258                 except ValueError as exc:
259                     raise ClientProtocolError("Invalid Content-Length") from exc
260                 if not 0 <= expected_length <= self._MAX_RESPONSE_BYTES:
261                     raise ClientProtocolError(
262                         "Response exceeds bounded JSON body size"
263                     )
264             raw = stream.read(self._MAX_RESPONSE_BYTES + 1)
265             if expected_length is not None and len(raw) < expected_length:
266                 raise http.client.IncompleteRead(
267                     raw, expected_length - len(raw)
268                 )
269             completed.put((True, (status, raw)))
270         except (
271             ClientError,
272             OSError,
273             ValueError,
274             TypeError,
275             TimeoutError,
276             http.client.HTTPException,
277         ) as exc:
278             completed.put((False, exc))
279
280     threading.Thread(target=receive, daemon=True, name="simulator-response").start()
281     try:
282         succeeded, value = completed.get(
283             timeout=max(0.0, deadline - time.monotonic())
284         )
285     except queue.Empty as exc:
286         raise TimeoutError(
287             "Absolute HTTP deadline elapsed; outcome remains unknown"
288         ) from exc
289     if not succeeded:
290         raise value
291     return value
292
293     def act(
294         self, path: str, position: Point | None = None, channel: int | None = None
295     ) -> dict:
296         position, channel = _action_args(path, position, channel)
297         with self._lock:
298             if self._closed:
299                 raise ClientError("Client is closed")
300             if self._uncertain:
301                 raise AmbiguousActionError(
302                     "Previous outcome is unresolved; no different action may be sent"
303                 )
304             if self._exited:
305                 raise ClientError("Session already exited; do not query /exit again")
306             if self._deadline is not None and time.monotonic() >= self._deadline:
307                 raise DeadlineExceeded(
308                     "Actual remaining real duration has elapsed; interface may be closed"
309                 )
310             if (
311                 self._max_virtual_duration_s is not None
312                 and self.virtual_time_s >= self._max_virtual_duration_s
313             ):
314                 raise DeadlineExceeded(
315                     "Virtual deadline reached; no post-timeout /exit query is legal"
316                 )

```

```

317     if path == "/enter" and self._entered:
318         raise ClientError("Cannot enter the same test twice")
319     if path != "/enter" and not self._entered:
320         raise ClientError(
321             "Explicit successful /enter is required before other actions"
322         )
323     request_id = uuid.uuid4().hex
324     payload = {
325         "arena_id": "default",
326         "robot_id": self._robot_id,
327         "request_id": request_id,
328     }
329     if position is not None:
330         payload.update(
331             position={"x": position[0], "y": position[1]}, channel=channel
332         )
333     body = json.dumps(
334         payload, ensure_ascii=False, allow_nan=False, separators=(",", ":")
335     ).encode("utf-8")
336     first_send = time.monotonic()
337     stop = first_send + self._ACTION_RETRY_BUDGET_S
338     if self._deadline is not None:
339         stop = min(stop, self._deadline)
340     last_error = "No response"
341     for attempt in range(1, self._MAX_ATTEMPTS + 1):
342         available = stop - time.monotonic()
343         if available <= 0:
344             break
345         self._record(
346             {
347                 "event": "request",
348                 "path": path,
349                 "request_id": request_id,
350                 "attempt": attempt,
351                 "body": payload,
352             }
353         )
354         request = Request(
355             self._base_url + path,
356             data=body,
357             headers={"Content-Type": "application/json"},
358             method="POST",
359         )
360         # From this point until a valid outcome, execution is uncertain.
361         self._uncertain = True
362         try:
363             status, raw = self._exchange(request, stop)
364             self._record(
365                 {
366                     "event": "response",
367                     "path": path,
368                     "request_id": request_id,
369                     "attempt": attempt,
370                     "http_status": status,
371                     "body_utf8": raw.decode("utf-8", errors="replace"),
372                 }
373             )
374         except (URLError, OSError, http.client.HTTPException) as exc:
375             # Includes connection refusal, broken connections, timeouts,
376             # truncated HTTP bodies. No state is inferred from any of these.
377             last_error = f"{type(exc).__name__}: {exc}"
378             self._record(
379                 {
380                     "event": "transport_error",
381                     "path": path,
382                     "request_id": request_id,
383                     "attempt": attempt,

```

```

384         "error": last_error,
385     }
386 )
387 else:
388     if status in self._TRANSIENT_HTTP:
389         last_error = f"Transient HTTP {status}"
390     else:
391         response = self._decode(raw)
392         if status != 200 or response["accepted"] is not True:
393             if response["accepted"] is True:
394                 raise ClientProtocolError(
395                     "accepted=true with non-200 HTTP status"
396                 )
397             self._uncertain = False
398             raise RejectedActionError(status, response)
399         self._validate_accepted(path, response)
400         radio_before = self.radio_channel
401         distance = (
402             math.dist(self.position, position)
403             if position is not None
404             else 0.0
405         )
406         switch = int(path == "/measure" and channel != radio_before)
407         self._record(
408             {
409                 "event": "accepted",
410                 "path": path,
411                 "request_id": request_id,
412                 "distance_m": distance,
413                 "radio_channel_before": radio_before,
414                 "radio_channel_after": channel
415                 if path == "/measure"
416                 else radio_before,
417                 "switch_time_s": switch,
418                 "action_virtual_time_s": response["virtual_time_s"]
419                 - self.virtual_time_s,
420             }
421         )
422         self._commit(path, position, channel, response, first_send)
423         return response
424     if attempt < self._MAX_ATTEMPTS:
425         delay = min(
426             0.2 * 2 ** (attempt - 1), max(0.0, stop - time.monotonic())
427         )
428         if delay > 0:
429             time.sleep(delay)
430     raise AmbiguousActionError(
431         f"Action {request_id} has no confirmed outcome after bounded retries: {last_error}"
432     )
433
434 def close(self):
435     """Close only this adapter's log. Never start/end a simulator session."""
436     with self._lock:
437         if not self._closed:
438             self._log.close()
439             self._closed = True

```

Listing 12: HTTP 验证

```

1  """HTTP fault-injection regression. Local test server, never the official simulator.
2
3  Start --server with hub; run without --server as a finite client verification.
4  The server executes the first measure, truncates its HTTP body, then answers its
5  same-ID retry from cache. Passing requires exactly-once virtual-time effects.
6  """
7
8  from __future__ import annotations

```

```

9
10 import argparse
11 import json
12 import tempfile
13 import time
14 from http.server import BaseHTTPRequestHandler, HTTPServer
15 from pathlib import Path
16
17 from client import AmbiguousActionError, HTTPClient
18 from environment import LocalSimulator
19
20
21 def serve(port, case):
22     environment = LocalSimulator(20260910)
23     cache = {}
24     truncated = False
25
26     class Handler(BaseHTTPRequestHandler):
27         def log_message(self, format, *args):
28             return
29
30         def do_POST(self):
31             nonlocal truncated
32             raw = self.rfile.read(int(self.headers["Content-Length"]))
33             body = json.loads(raw)
34             identity = body["request_id"]
35             key = (self.path, raw)
36             if identity in cache:
37                 old_key, response = cache[identity]
38                 if key != old_key:
39                     self.send_error(409)
40                     return
41             else:
42                 point = body.get("position")
43                 point = None if point is None else (point["x"], point["y"])
44                 response = environment.act(self.path, point, body.get("channel"))
45                 cache[identity] = (key, response)
46             encoded = json.dumps(response).encode()
47             self.send_response(200)
48             self.send_header("Content-Type", "application/json")
49             self.send_header("Content-Length", str(len(encoded)))
50             self.end_headers()
51             if self.path == "/measure" and not truncated:
52                 truncated = True
53                 if case == "deadline":
54                     size = max(1, len(encoded) // 8)
55                     for start in range(0, len(encoded), size):
56                         self.wfile.write(encoded[start : start + size])
57                         self.wfile.flush()
58                         time.sleep(0.3)
59                 else:
60                     self.wfile.write(encoded[: len(encoded) // 2])
61                     self.wfile.flush()
62                     self.close_connection = True
63             else:
64                 self.wfile.write(encoded)
65
66     server = HTTPServer(("127.0.0.1", port), Handler)
67     print(f"HTTP fault fixture ready on {port}", flush=True)
68     server.serve_forever()
69
70
71 def verify(port, case):
72     Path("results").mkdir(exist_ok=True)
73     with tempfile.TemporaryDirectory(prefix="b-http-check-") as directory:
74         path = Path(directory) / "requests.jsonl"
75         client = HTTPClient(f"http://127.0.0.1:{port}", "local-regression", str(path))

```

```

76     try:
77         client.act("/enter")
78     if case == "deadline":
79         client._ACTION_RETRY_BUDGET_S = 0.5
80         started = time.monotonic()
81         try:
82             client.act("/measure", (300.0, 400.0), 1)
83         except AmbiguousActionError:
84             elapsed = time.monotonic() - started
85         else:
86             raise AssertionError("Slow response bypassed the absolute deadline")
87         assert elapsed < 1.5, elapsed
88         assert client.virtual_time_s == 0
89         try:
90             client.act("/measure", (0.0, 0.0), 2)
91         except AmbiguousActionError:
92             pass
93         else:
94             raise AssertionError(
95                 "Different action sent after an ambiguous timeout"
96             )
97         result = {
98             "evidence": "local_HTTP_fault_fixture_not_official",
99             "absolute_deadline_fail_closed": True,
100             "deadline_s": 0.5,
101             "observed_elapsed_s": elapsed,
102             "no_unconfirmed_state_advance": True,
103         }
104         output = Path("results/http_deadline_verification.json")
105     else:
106         assert (
107             client.act("/measure", (300.0, 400.0), 1)["virtual_time_s"] == 105
108         )
109         assert (
110             client.act("/measure", (300.0, 400.0), 2)["virtual_time_s"] == 111
111         )
112         clear = client.act("/clear", (300.0, 0.0), 3)
113         expected = 196 if clear["clear_result"] == "success" else 194
114         assert clear["virtual_time_s"] == expected
115         assert client.radio_channel == 2
116         assert (
117             client.act("/measure", (300.0, 0.0), 2)["virtual_time_s"]
118             == expected + 5
119         )
120         assert client.act("/exit")["virtual_time_s"] == expected + 5
121         result = {
122             "evidence": "local_HTTP_fault_fixture_not_official",
123             "truncated_response_recovered": True,
124             "retry_reused_identical_request": True,
125             "virtual_time_exactly_once": True,
126             "clear_kept_radio_channel": True,
127             "final_virtual_time_s": expected + 5,
128         }
129         output = Path("results/http_verification.json")
130     finally:
131         client.close()
132     events = [json.loads(line) for line in path.read_text().splitlines()]
133     requests = [
134         row
135         for row in events
136         if row["event"] == "request" and row["path"] == "/measure"
137     ]
138     if case == "deadline":
139         assert len(requests) == 1
140     else:
141         assert requests[0]["request_id"] == requests[1]["request_id"]
142         assert requests[0]["body"] == requests[1]["body"]

```

```

143         output.write_text(json.dumps(result, indent=2) + "\n")
144         output.with_suffix(".jsonl").write_text(path.read_text())
145     print(json.dumps(result, indent=2))
146
147
148 if __name__ == "__main__":
149     parser = argparse.ArgumentParser()
150     parser.add_argument("--server", action="store_true")
151     parser.add_argument("--port", type=int, default=12026)
152     parser.add_argument(
153         "--case", choices=("truncated", "deadline"), default="truncated"
154     )
155     arguments = parser.parse_args()
156     serve(arguments.port, arguments.case) if arguments.server else verify(
157         arguments.port, arguments.case
158     )

```

Listing 13: 官方运行入口（仅获授权后使用）

```

1  """Connect the policy to an already started OFFICIAL simulator session.
2
3  Never starts a formal test or supplies team credentials automatically. Use the
4  real GUI to choose the intended practice/formal module, then invoke this CLI.
5  Plaintext request logs include robot_id and are private working files: do not
6  put them unredacted in anonymous submission. Official encrypted logs are exported
7  separately from the simulator with unchanged filenames.
8  """
9
10 from __future__ import annotations
11
12 import argparse
13 import json
14 from pathlib import Path
15 from time import monotonic
16
17 from client import HTTPClient
18 from planner import Planner
19
20
21 def main():
22     parser = argparse.ArgumentParser()
23     parser.add_argument("--robot-id", required=True)
24     parser.add_argument("--problem", required=True, type=int, choices=(3, 4))
25     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
26     parser.add_argument("--output-dir", type=Path, required=True)
27     args = parser.parse_args()
28     if args.output_dir.exists():
29         parser.error("Output directory must be new to keep separate session evidence")
30     args.output_dir.mkdir(parents=True)
31     client = HTTPClient(
32         args.base_url, args.robot_id, str(args.output_dir / "private_requests.jsonl")
33     )
34     started = monotonic()
35     try:
36         result = Planner(client, problem=args.problem).run()
37         result.update(
38             evidence="official_HTTP_observations_not_encrypted_export",
39             local_wall_duration_s=monotonic() - started,
40             case_code="Obtain actual case code from simulator GUI; not returned by HTTP",
41             total_source_count="Not disclosed by official formal interface",
42         )
43         (args.output_dir / "observed_result.json").write_text(
44             json.dumps(result, ensure_ascii=False, indent=2) + "\n"
45         )
46         print(json.dumps(result, ensure_ascii=False, indent=2))
47     finally:
48         client.close()

```

```

49
50
51 if __name__ == "__main__":
52     main()

```

Listing 14: 数据驱动图表与轨迹生成

```

1  """Deterministic local-evidence assets; run with conda run -n py314 python make_assets.py.
2
3  No official service is accessed. Replay logs contain public observations, not source
4  coordinates. Wall-clock durations in raw replay evidence are inherently variable;
5  figures, table values, and virtual-time paths do not depend on those durations.
6  """
7
8  from __future__ import annotations
9
10 import hashlib
11 import json
12 import math
13 import os
14 import statistics
15 import tempfile
16 from pathlib import Path
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23 from matplotlib.patches import Circle, Polygon
24
25 from coverage import directional_waypoints, omni_waypoints
26 from environment import LocalSimulator
27 from geometry import BEARING_ERROR_DEG, clip_bearing, enclosing_circle, initial_polygon
28 from planner import Planner
29
30 ROOT = Path(__file__).resolve().parent
31 GENERATED = ROOT / "report" / "generated"
32 FIGURES = ROOT / "report" / "figures"
33 POLICIES = ("baseline", "batched", "integrated")
34 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
35 SCENARIO_LABELS = ("Random", "Min. radius", "Outward boundary", "Clustered")
36 COLORS = ("0072B2", "E69F00", "009E73")
37 WIDTH = 16 / 2.54
38
39
40 def require(condition, message):
41     if not condition:
42         raise ValueError(message)
43
44
45 def atomic_bytes(path, content):
46     """Replace only our named outputs; never remove/rewrite source evidence."""
47     require(path.parent in (GENERATED, FIGURES), "Output outside owned directories")
48     path.parent.mkdir(parents=True, exist_ok=True)
49     fd, temporary = tempfile.mkstemp(prefix="." + path.name + ".", dir=path.parent)
50     try:
51         with os.fdopen(fd, "wb") as stream:
52             stream.write(content)
53             os.replace(temporary, path)
54     finally:
55         Path(temporary).unlink(missing_ok=True)
56
57
58 def write_json(name, value):
59     atomic_bytes(
60         GENERATED / name,

```

```

61         (
62             json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
63         ).encode("utf-8"),
64     )
65
66
67 def save_figure(fig, name):
68     # Fixed physical width, not bbox_inches=tight (which changes final font scale).
69     from io import BytesIO
70
71     buffer = BytesIO()
72     fig.savefig(
73         buffer,
74         format="pdf",
75         metadata={"CreationDate": None, "ModDate": None, "Creator": "make_assets.py"},
76     )
77     atomic_bytes(FIGURES / name, buffer.getvalue())
78     plt.close(fig)
79
80
81 def tex_escape(value):
82     return str(value).replace("_", r"\_").replace("%", r"\%")
83
84
85 def table(name, headers, rows, alignment=None):
86     alignment = alignment or "l" + "r" * (len(headers) - 1)
87     lines = [
88         r"\begin{tabular}{%s + alignment + "}",
89         r"\toprule",
90         " & ".join(headers) + r" \\",
91         r"\midrule",
92     ]
93     for row in rows:
94         require(len(row) == len(headers), "Ragged table")
95         lines.append(" & ".join(str(v) for v in row) + r" \\\")
96     lines += [r"\bottomrule", r"\end{tabular}"]
97     atomic_bytes(GENERATED / name, ("\n".join(lines) + "\n").encode("utf-8"))
98
99
100 def number(value):
101     require(
102         isinstance(value, (int, float)) and math.isfinite(value), "Nonfinite metric"
103     )
104     return f"{value:.2f}"
105
106
107 def metrics(rows):
108     values = sorted(row["average_time_s"] for row in rows)
109     require(bool(values), "Empty evidence group")
110     return {
111         "cases": len(rows),
112         "all_cleared_cases": sum(r["fraction_cleared"] == 1 for r in rows),
113         "mean_average_time_s": statistics.mean(values),
114         "median_average_time_s": statistics.median(values),
115         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
116         "mean_failed_clears": statistics.mean(r["failed_clears"] for r in rows),
117     }
118
119
120 def validate_group(rows, summary, name):
121     actual = metrics(rows)
122     for key, value in actual.items():
123         require(
124             key in summary
125             and math.isclose(value, summary[key], rel_tol=1e-10, abs_tol=1e-8),
126             f"{name}: summary mismatch for {key}",
127         )

```

```

128     return rows
129
130
131 def load_inputs():
132     names = (
133         "final_experiments.json",
134         "geometry_verification.json",
135         "http_verification.json",
136         "second_point_example.json",
137     )
138     data, hashes = {}, {}
139     for name in names:
140         raw = (ROOT / "results" / name).read_bytes()
141         data[name] = json.loads(raw)
142         hashes["results/" + name] = hashlib.sha256(raw).hexdigest()
143     final = data[names[0]]
144     require(
145         final["evidence"] == "local_synthetic_not_official", "Not synthetic evidence"
146     )
147     require(final["total_runs"] == len(final["runs"]), "Run denominator mismatch")
148     for row in final["runs"]:
149         require(
150             row["evidence"] == "local_synthetic_not_official"
151             and row["fraction_cleared"] == 1,
152             "Incomplete or wrongly attributed run",
153         )
154         require(row["n_sources"] == row["n_cleared"] > 0, "Invalid clearing count")
155         require(
156             math.isclose(
157                 row["average_time_s"],
158                 row["virtual_time_s"] / row["n_sources"],
159                 rel_tol=1e-10,
160             ),
161             "Metric is not per-world T/N",
162         )
163         require(row["timing_independently_verified"] is True, "Missing timing evidence")
164         require(row["certificate_complete"] is True, "Missing complete certificate")
165         require(row["loss_recovery"] is (row["problem"] == 4 and row.get("loss_recovery_requested", True)),
166                 "Effective recovery mismatch")
167     main, recovery = {}, {}
168     n = final["paired_random_cases_per_problem"]
169     for problem in (3, 4):
170         for strategy in POLICIES:
171             key = f"Q{problem}_{strategy}"
172             rows = sorted(
173                 (
174                     r
175                     for r in final["runs"]
176                     if r["problem"] == problem
177                     and r["strategy"] == strategy
178                     and 600000 <= r["seed"] < 600000 + n
179                 ),
180                 key=lambda r: r["seed"],
181             )
182             require(
183                 [r["seed"] for r in rows] == list(range(600000, 600000 + n)),
184                 "Missing/duplicate paired seed",
185             )
186             main[key] = validate_group(rows, final["summary"][key], key)
187     paired = final["paired_comparison"][f"Q{problem}"]
188     saved = statistics.mean(
189         a["average_time_s"] - b["average_time_s"]
190         for a, b in zip(
191             main[f"Q{problem}_baseline"], main[f"Q{problem}_integrated"]
192         )
193     )
194     require(

```

```

195     paired["paired_cases"] == n
196     and math.isclose(saved, paired["mean_saved_s_per_source"], rel_tol=1e-10),
197     "Pairing mismatch",
198 )
199 require(
200     math.isclose(
201         paired["mean_time_reduction_pct"],
202         100
203         * saved
204         / final["summary"][f"Q{problem}_baseline"]["mean_average_time_s"],
205         rel_tol=1e-10,
206     ),
207     "Gain mismatch",
208 )
209 for scenario in SCENARIOS[1:]:
210     for mode in ("smooth", "extreme", "iid_location"):
211         key = f"Q{problem}_{scenario}_{mode}"
212         rows = [
213             r
214             for r in final["runs"]
215             if r["problem"] == problem
216             and r["scenario"] == scenario
217             and r["error_mode"] == mode
218             and 700000 <= r["seed"] < 700020
219         ]
220         validate_group(rows, final["stress"][key], key)
221 for scale in (0.12, 0.22, 0.40):
222     key = f"Q{problem}_probe_{scale}"
223     rows = [
224         r
225         for r in final["runs"]
226         if r["problem"] == problem
227         and r["probe_scale"] == scale
228         and 800000 <= r["seed"] < 800030
229     ]
230     validate_group(rows, final["sensitivity"][key], key)
231 for spacing in (900.0, 950.0, 990.0):
232     key = f"Q4_spacing_{spacing}"
233     rows = [
234         r
235         for r in final["runs"]
236         if r["problem"] == 4
237         and r["spacing_m"] == spacing
238         and 900000 <= r["seed"] < 900030
239     ]
240     validate_group(rows, final["sensitivity"][key], key)
241 for scenario in SCENARIOS:
242     for enabled in (False, True):
243         key = f"{scenario}_recovery_{enabled}"
244         rows = sorted(
245             (
246                 r
247                 for r in final["runs"]
248                 if r["problem"] == 4
249                 and r["scenario"] == scenario
250                 and r["loss_recovery"] is enabled
251                 and 610000 <= r["seed"] < 610100
252             ),
253             key=lambda r: r["seed"],
254         )
255         require(
256             [r["seed"] for r in rows] == list(range(610000, 610100)),
257             "Recovery pairing mismatch",
258         )
259         recovery[key] = validate_group(rows, final["recovery_ablation"][key], key)
260 geometry, http, second = (data[name] for name in names[1:])
261 require(

```

```

262     geometry["evidence"]
263     == "independent_numerical_checks_not_a_substitute_for_analytic_proofs",
264     "Geometry evidence mismatch",
265 )
266 require(
267     http["evidence"] == "local_HTTP_fault_fixture_not_official",
268     "HTTP evidence mismatch",
269 )
270 require(
271     second["evidence"] == "illustrative_Q2_grid_score_not_global_optimum",
272     "Q2 evidence mismatch",
273 )
274 for key in (
275     "truncated_response_recovered",
276     "retry_reused_identical_request",
277     "virtual_time_exactly_once",
278     "clear_kept_radio_channel",
279 ):
280     require(http[key] is True, f"Failed fixture: {key}")
281 return final, geometry, http, second, main, recovery, hashes
282
283
284 def generate_tables(final, geometry, http):
285     summary = final["summary"]
286     macros = {"ExperimentRuns": str(final["total_runs"])}
287     for problem, word in ((3, "Three"), (4, "Four")):
288         macros[f"Q{word}Time"] = number(
289             summary[f"Q{problem}_integrated"]["mean_average_time_s"]
290         )
291         macros[f"Q{word}Baseline"] = number(
292             summary[f"Q{problem}_baseline"]["mean_average_time_s"]
293         )
294         # Numeric macro: paper supplies its own percent sign and unit.
295         macros[f"Q{word}Gain"] = number(
296             final["paired_comparison"][f"Q{problem}"]["mean_time_reduction_pct"]
297         )
298     atomic_bytes(
299         GENERATED / "numbers.tex",
300         (
301             "\n".join(
302                 r"\newcommand{" + "\\\" + k + "-{" + v + "}" for k, v in macros.items()
303             )
304             + "\n"
305         ).encode(),
306     )
307     rows = []
308     for problem in (3, 4):
309         for policy in POLICIES:
310             s = summary[f"Q{problem}_{policy}"]
311             rows.append(
312                 (
313                     f"Q{problem} {policy}",
314                     number(s["mean_average_time_s"]),
315                     number(s["median_average_time_s"]),
316                     number(s["p90_average_time_s"]),
317                     f"{s['all_cleared_cases']}/{s['cases']}",
318                 )
319             )
320     table(
321         "comparison_table.tex",
322         ("Policy", r"Mean  $T/N$  (s)", "Median (s)", " $p_{90}$  (s)", "Complete"),
323         rows,
324     )
325     rows = []
326     for scenario, label in zip(SCENARIOS[1:], SCENARIO_LABELS[1:]):
327         for mode, mode_label in (
328             ("smooth", "Smooth"),

```

```

329         ("extreme", r"$\pm 1^\circ$ hash"),
330         ("iid_location", "Uniform hash"),
331     ):
332         s3, s4 = (final["stress"][f"Q{p}_{scenario}_{mode}"] for p in (3, 4))
333         require(s3["cases"] == s4["cases"], "Stress denominator mismatch")
334         rows.append(
335             (
336                 label,
337                 mode_label,
338                 number(s3["mean_average_time_s"]),
339                 number(s4["mean_average_time_s"]),
340                 str(s3["cases"]),
341             )
342         )
343     table(
344         "stress_table.tex",
345         ("Geometry", "Error field", "Q3 mean (s)", "Q4 mean (s)", "$n$/Q"),
346         rows,
347         "llrrr",
348     )
349     rows = []
350     for key, s in final["sensitivity"].items():
351         problem, parameter, value = key.split("_")
352         label = (
353             r"$\lambda$=" + value + "$"
354             if parameter == "probe"
355             else "$s$ = " + f"{float(value):.0f}" + r"\, \mathrm{m}$"
356         )
357         rows.append(
358             (
359                 problem,
360                 label,
361                 number(s["mean_average_time_s"]),
362                 number(s["p90_average_time_s"]),
363                 str(s["cases"]),
364             )
365         )
366     table(
367         "sensitivity_table.tex",
368         ("Problem", "Setting", "Mean (s)", "$p_{90}$ (s)", "$n$"),
369         rows,
370         "llrrr",
371     )
372     rows = []
373     for scenario, label in zip(SCENARIOS, SCENARIO_LABELS):
374         off, on = (
375             final["recovery_ablation"][f"{scenario}_recovery_{enabled}"]
376             for enabled in (False, True)
377         )
378         rows.append(
379             (
380                 label,
381                 number(off["mean_average_time_s"]),
382                 number(on["mean_average_time_s"]),
383                 number(off["mean_failed_clears"]),
384                 number(on["mean_failed_clears"]),
385                 str(on["cases"]),
386             )
387         )
388     table(
389         "recovery_table.tex",
390         ("Scenario", "$T/N$ off", "$T/N$ on", "Fails off", "Fails on", "Pairs"),
391         rows,
392     )
393     rows = [
394         (
395             "MEC reference",

```

```

396         str(geometry["mec_reference_cases"]),
397         f"{geometry['maximum_mec_radius_error_m']:.2e} m",
398     ),
399     (
400         "Bearing containment",
401         str(geometry["containment_cases"]),
402         f"{geometry['bearings_per_case']} readings/case",
403     ),
404     (
405         "Optical cover samples",
406         str(geometry["optical_interior_samples"]),
407         "Contained",
408     ),
409     (
410         "Discovery samples",
411         str(geometry["coverage_source_samples"]),
412         "Both geometries",
413     ),
414     (
415         "Triple-lens samples",
416         str(geometry["triple_lens_samples"]),
417         "Reception checked",
418     ),
419     ("Local HTTP fixture", "1", "Four checks passed"),
420 ]
421 table(
422     "verification_table.tex",
423     ("Numerical / local check", "Cases / points", "Observed result"),
424     rows,
425     "lrl",
426 )
427
428
429 def spatial_axis(ax, limit, unit="km"):
430     ax.set(
431         xlim=(-limit, limit),
432         ylim=(-limit, limit),
433         xlabel=f"x ({unit})",
434         ylabel=f"y ({unit})",
435         aspect="equal",
436     )
437     ax.grid(alpha=0.18)
438
439
440 def coverage_figure(geometry):
441     cert = geometry["coverage_certificate"]
442     omni, lattice = np.array(omni_waypoints()), np.array(directional_waypoints())
443     require(
444         len(omni) == cert["omni"]["waypoint_count"]
445         and len(lattice) == cert["directional"]["waypoint_count"],
446         "Coverage count mismatch",
447     )
448     fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 3.7))
449     fig.subplots_adjust(left=0.08, right=0.98, bottom=0.30, top=0.88, wspace=0.4)
450     for ax in axes:
451         spatial_axis(ax, 3.0)
452         ax.add_patch(
453             Circle(
454                 (0, 0), cert["arena_radius_m"] / 1000, fill=False, color="black", lw=1.3
455             )
456         )
457     for point in omni / 1000:
458         axes[0].add_patch(
459             Circle(
460                 point,
461                 cert["omni"]["cover_radius_m"] / 1000,
462                 color=COLORS[0],

```

```

463         fill=False,
464         alpha=0.6,
465         lw=0.8,
466     )
467 )
468 axes[0].scatter(
469     *(omni / 1000).T, s=20, color=COLORS[0], label=f"{len(omni)} detectors"
470 )
471 inside = np.linalg.norm(lattice, axis=1) <= cert["arena_radius_m"]
472 require(
473     int((~inside).sum()) == cert["directional"]["outside_arena_count"],
474     "Outside-node count mismatch",
475 )
476 # Draw only nearest-neighbour lattice edges, including exterior nodes.
477 for i, p in enumerate(lattice):
478     for q in lattice[:i]:
479         if math.isclose(
480             math.dist(p, q), cert["directional"]["spacing_m"], abs_tol=1e-6
481         ):
482             axes[1].plot(
483                 [p[0] / 1000, q[0] / 1000],
484                 [p[1] / 1000, q[1] / 1000],
485                 color="0.78",
486                 lw=0.6,
487                 zorder=0,
488             )
489 axes[1].scatter(
490     *(lattice[inside] / 1000).T,
491     color=COLORS[0],
492     s=22,
493     label=f"Inside: {int(inside.sum())}",
494 )
495 axes[1].scatter(
496     *(lattice[~inside] / 1000).T,
497     color="#D55E00",
498     marker="^",
499     s=30,
500     label=f"Outside: {int((~inside).sum())}",
501 )
502 axes[0].set_title("Q3: 900 m covering disks")
503 axes[1].set_title(f"Q4: {len(lattice)} lattice nodes")
504 for ax in axes:
505     box = ax.get_position()
506     ax.legend(loc="lower center", bbox_to_anchor=(box.x0 + box.x1) / 2, 0.01),
507     bbox_transform=fig.transFigure, frameon=False)
508 save_figure(fig, "coverage.pdf")
509
510
511 def geometry_figure(geometry):
512     example = geometry["realizable_counterexample"]
513     polygon = initial_polygon()
514     for station, bearing in zip(example["stations"], example["bearings_deg"]):
515         polygon = clip_bearing(polygon, tuple(station), bearing, error_deg=1.0)
516     center, radius = enclosing_circle(polygon)
517     require(
518         math.isclose(radius, example["mec_radius_m"], abs_tol=1e-6),
519         "Triangle reconstruction mismatch",
520     )
521     a, b = max(
522         ((a, b) for a in polygon for b in polygon), key=lambda pair: math.dist(*pair)
523     )
524     midpoint = tuple((x + y) / 2 for x, y in zip(a, b))
525     fig, ax = plt.subplots(figsize=(WIDTH, 3.6), layout="constrained")
526     ax.add_patch(Polygon(polygon, facecolor="#56B4E9", alpha=0.35, edgecolor=COLORS[0]))
527     ax.plot(
528         *zip(*(polygon + [polygon[0]])),
529         color=COLORS[0],

```

```

530     label=f"Feasible region: D={example['diameter_m']:.0f} m",
531 )
532 ax.add_patch(
533     Circle(
534         midpoint,
535         example["diameter_m"] / 2,
536         fill=False,
537         color="#D55E00",
538         ls="--",
539         label=f"Diameter circle: {example['diameter_m'] / 2:.2f} m",
540     )
541 )
542 ax.add_patch(
543     Circle(
544         center,
545         radius,
546         fill=False,
547         color=COLORS[2],
548         lw=1.5,
549         label=f"MEC: {radius:.2f} m > 20 m",
550     )
551 )
552 ax.scatter(*center, color=COLORS[2], marker="+", s=60)
553 ax.set(
554     xlim=(-9, 49), ylim=(-22, 40), xlabel="x (m)", ylabel="y (m)", aspect="equal"
555 )
556 ax.legend(loc="center left", bbox_to_anchor=(1.0, 0.5), frameon=False)
557 ax.grid(alpha=0.2)
558 save_figure(fig, "geometry.pdf")
559
560
561 def second_point_figure(second):
562     require(
563         second["first_station"] == [0, 0] and second["first_reading_deg"] == 0,
564         "Q2 figure expects normalized illustrative case",
565     )
566     alpha = math.radians(BEARING_ERROR_DEG)
567     centers = np.array(
568         [
569             (0, 0),
570             (1000 * math.cos(alpha), 1000 * math.sin(alpha)),
571             (1000 * math.cos(alpha), -1000 * math.sin(alpha)),
572         ]
573     )
574     t = np.linspace(0, 1000, 1001)
575     lower = np.full_like(t, -np.inf)
576     upper = np.full_like(t, np.inf)
577     for x, y in centers:
578         h = np.sqrt(np.maximum(0, 1000**2 - (t - x) ** 2))
579         lower, upper = np.maximum(lower, y - h), np.minimum(upper, y + h)
580     mask = lower <= upper
581     fig, ax = plt.subplots(figsize=(WIDTH, 3.6), layout="constrained")
582     ax.fill_between(
583         t,
584         lower,
585         upper,
586         where=mask,
587         color="#56B4E9",
588         alpha=0.35,
589         label="Triple-disk reception lens",
590     )
591     ax.plot(t[mask], lower[mask], color=COLORS[0])
592     ax.plot(t[mask], upper[mask], color=COLORS[0])
593     candidates = second["candidates"]
594     ax.scatter(
595         [c["t_m"] for c in candidates],
596         [c["b_m"] for c in candidates],

```

```

597         color="0.55",
598         marker=".",
599         s=12,
600         label="Finite candidate grid",
601     )
602     for key, label, marker, color in (
603         ("best_sampled_geometry", "Sampled choice", "*", COLORS[2]),
604         ("collinear_control", "Collinear control", "s", "#D55E00"),
605     ):
606         c = second[key]
607         require(
608             all(math.dist(c["point"], p) <= 1000 + 1e-6 for p in centers),
609             "Candidate outside lens",
610         )
611         ax.scatter(
612             c["t_m"],
613             c["b_m"],
614             s=90,
615             marker=marker,
616             color=color,
617             label=f"{label}\nSampled max R={c['sampled_worst_mec_radius_m']:.2f} m",
618         )
619     ax.scatter(0, 0, color="black", s=20)
620     ax.axhline(0, color="0.6", lw=0.6, ls="--")
621     ax.set(
622         xlabel="Along first bearing t (m)",
623         ylabel="Transverse b (m)",
624         xlim=(-40, 1040),
625         ylim=(-920, 920),
626     )
627     ax.legend(loc="center left", bbox_to_anchor=(1.0, 0.5), frameon=False)
628     save_figure(fig, "q2_lens.pdf")
629
630
631 def performance_figure(main):
632     fig, axes = plt.subplots(
633         1, 2, figsize=(WIDTH, 3.7), layout="constrained", sharey=True
634     )
635     for problem, ax in zip((3, 4), axes):
636         for policy, color, style in zip(POLICIES, COLORS, ("-", "--", "-.")):
637             rows = main[f"Q{problem}_{policy}"]
638             values = sorted(r["average_time_s"] for r in rows)
639             ax.step(
640                 values,
641                 np.arange(1, len(values) + 1) / len(values),
642                 where="post",
643                 color=color,
644                 ls=style,
645                 label=policy.capitalize(),
646             )
647             ax.set(
648                 title=f"Q{problem}: {len(rows)} matched worlds",
649                 xlabel="Per-world T/N (s)",
650                 ylim=(0, 1.02),
651             )
652             ax.grid(alpha=0.2)
653             ax.legend(loc="lower right", frameon=False)
654     axes[0].set_ylabel("Empirical cumulative fraction")
655     save_figure(fig, "performance.pdf")
656
657
658 def replay_trajectories():
659     traces = []
660     for problem in (3, 4):
661         env = LocalSimulator(
662             20260910, problem=problem, error_mode="smooth", scenario="random"
663         )

```

```

664     planner = Planner(
665         env, problem=problem, strategy="enhanced", loss_recovery=True
666     )
667     result = planner.run()
668     summary = env.summary() # Evaluation after termination; never supplied to Planner.
669     require(summary["fraction_cleared"] == 1, "Trajectory replay incomplete")
670     require(result["certificate_complete"] is True, "Trajectory certificate incomplete")
671     points, clears = [[0.0, 0.0]], []
672     for event in env.log:
673         request, response = event["request"], event["response"]
674         if not response["accepted"] or "position" not in request:
675             continue
676         point = [request["position"]["x"], request["position"]["y"]]
677         if point != points[-1]:
678             points.append(point)
679         if response.get("clear_result") == "success":
680             clears.append(
681                 {
682                     "channel": request["channel"],
683                     "point": point,
684                     "virtual_time_s": response["virtual_time_s"],
685                 }
686             )
687     require(
688         len(clears) == summary["n_cleared"] == result["n_cleared"],
689         "Replay clear count mismatch",
690     )
691     trace = {
692         "evidence": "local_synthetic_not_official",
693         "seed": 20260910,
694         "problem": problem,
695         "strategy": "enhanced",
696         "scenario": "random",
697         "error_mode": "smooth",
698         "requested_loss_recovery": True,
699         "effective_loss_recovery": problem == 4,
700         "point_semantics": "Accepted robot coordinates; successful clear within 20 m, not source
701 coordinates",
702         "path": points,
703         "successful_clears": clears,
704         "planner": result,
705         "post_exit_summary": summary,
706         "observable_log": env.log,
707     }
708     write_json(f"trajectory_q{problem}.json", trace)
709     traces.append(trace)
710     write_json(
711         "trajectory_summary.json",
712         [
713             {k: t[k] for k in ("problem", "seed", "planner", "post_exit_summary")}
714             for t in traces
715         ],
716     )
717     fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 3.8), layout="constrained")
718     limit = max(
719         2.0, max(abs(v) / 1000 for t in traces for p in t["path"] for v in p) + 0.15
720     )
721     for ax, trace in zip(axes, traces):
722         path = np.array(trace["path"]) / 1000
723         clear = np.array([c["point"] for c in trace["successful_clears"]]) / 1000
724         spatial_axis(ax, limit)
725         ax.add_patch(Circle((0, 0), 1.8, color="0.5", fill=False, ls="--"))
726         ax.plot(*path.T, color=COLORS[0], lw=0.8, label="Accepted path")
727         ax.scatter(
728             *clear.T,
729             color="#D55E00",
730             marker="x",

```

```

730         s=26,
731         label="Successful clear",
732         zorder=3,
733     )
734     ax.scatter(0, 0, color="black", marker="s", s=22, label="Start", zorder=4)
735     ax.set_title(
736         f"Q{trace['problem']}: {len(clear)} clears\N/T/N={trace['post_exit_summary']['average_time_s']:.2f}
s"
737     )
738     handles, labels = axes[0].get_legend_handles_labels()
739     fig.legend(handles, labels, loc="outside lower center", ncol=3, frameon=False)
740     save_figure(fig, "trajectories.pdf")
741     return traces
742
743
744 def recovery_figure(final):
745     fig, axes = plt.subplots(
746         2, 1, figsize=(WIDTH, 5.2), layout="constrained", sharex=True
747     )
748     x = np.arange(len(SCENARIOS))
749     for enabled, offset, color, hatch, label in (
750         (False, -0.18, COLORS[0], "/", "Recovery off"),
751         (True, 0.18, COLORS[2], "", "Recovery on"),
752     ):
753         summaries = [
754             final["recovery_ablation"][f"{s}_recovery_{enabled}"] for s in SCENARIOS
755         ]
756         for ax, metric in zip(axes, ("mean_average_time_s", "mean_failed_clears")):
757             ax.bar(
758                 x + offset,
759                 [s[metric] for s in summaries],
760                 width=0.34,
761                 color=color,
762                 hatch=hatch,
763                 label=label,
764             )
765             ax.grid(axis="y", alpha=0.2)
766             ax.set_axisbelow(True)
767         axes[0].set_ylabel("Mean T/N (s)")
768         axes[1].set_ylabel("Mean failed clears")
769         axes[1].set_xticks(x, SCENARIO_LABELS, rotation=12, ha="right")
770         axes[0].legend(frameon=False, ncol=2, loc="upper left")
771         counts = sorted({v["cases"] for v in final["recovery_ablation"].values()})
772         require(len(counts) == 1, "Recovery counts differ")
773         axes[0].set_title(f"Q4: {counts[0]} matched worlds per scenario")
774         save_figure(fig, "recovery.pdf")
775
776
777 def main():
778     plt.rcParams.update(
779         {
780             "font.family": "DejaVu Sans",
781             "font.size": 10,
782             "axes.titlesize": 10,
783             "axes.labelsize": 10,
784             "xtick.labelsize": 10,
785             "ytick.labelsize": 10,
786             "legend.fontsize": 10,
787             "pdf.fonttype": 42,
788             "savefig.transparent": False,
789             "figure.facecolor": "white",
790         }
791     )
792     final, geometry, http, second, main_rows, _recovery_rows, hashes = load_inputs()
793     generate_tables(final, geometry, http)
794     coverage_figure(geometry)
795     geometry_figure(geometry)

```

```

796 second_point_figure(second)
797 performance_figure(main_rows)
798 traces = replay_trajectories()
799 recovery_figure(final)
800 for name in (
801     "make_assets.py",
802     "planner.py",
803     "environment.py",
804     "geometry.py",
805     "coverage.py",
806     "second_point.py",
807 ):
808     hashes[name] = hashlib.sha256((ROOT / name).read_bytes()).hexdigest()
809 enhancement_input = ROOT / "results" / "enhancement_experiments.json"
810 if enhancement_input.exists():
811     hashes["results/enhancement_experiments.json"] = hashlib.sha256(enhancement_input.read_bytes()).
812     hexdigest()
813 ring_sweep_input = ROOT / "results" / "ring_sweep.json"
814 if ring_sweep_input.exists():
815     hashes["results/ring_sweep.json"] = hashlib.sha256(ring_sweep_input.read_bytes()).hexdigest()
816 write_json(
817     "metadata.json",
818     {
819         "evidence": "local_synthetic_not_official",
820         "official_calls": 0,
821         "asset_set": "base_comparison_with_enhanced_trajectory",
822         "input_sha256": hashes,
823         "matplotlib_version": matplotlib.__version__,
824         "figures": [
825             "coverage.pdf", "geometry.pdf", "q2_lens.pdf", "performance.pdf",
826             "trajectories.pdf", "recovery.pdf", "enhancement_geometry.pdf",
827             "enhancement_performance.pdf",
828         ],
829         "tables": [
830             "comparison_table.tex", "stress_table.tex", "sensitivity_table.tex",
831             "recovery_table.tex", "verification_table.tex", "enhancement_table.tex",
832             "enhancement_costs.tex", "enhancement_stress.tex", "candidate_table.tex",
833             "coverage_limits_table.tex",
834         ],
835         "enhancement_metadata": "enhancement_metadata.json",
836         "layout": {
837             "width_cm": 16,
838             "minimum_font_pt": 10,
839             "backend": "Agg",
840             "format": "vector PDF",
841         },
842         "metric": "Mean, median and nearest-rank p90 of per-world T/N; never pooled sum(T)/sum(N). Gains
843         are reduction of group means; ECDFs use the same seeds per policy.",
844         "denominators": {
845             "all_runs_including_paired_repeats": final["total_runs"],
846             "main_worlds_per_problem": final["paired_random_cases_per_problem"],
847             "trajectory_replays_additional_to_experiments": len(traces),
848         },
849         "seed_ranges": {
850             "main": [600000, 600000 + final["paired_random_cases_per_problem"] - 1],
851             "stress": [700000, 700019],
852             "probe_sensitivity": [800000, 800029],
853             "spacing_sensitivity": [900000, 900029],
854             "recovery": [610000, 610099],
855             "trajectory": 20260910,
856         },
857         "parameters": {
858             "strategy": "enhanced",
859             "anchor_policy_Q3": "nearest",
860             "anchor_policy_Q4": "rolling",
861             "ring_radius_Q3_m": 1200,
862             "spacing_Q4_m": 990,

```

```

861         "probe_scale": 0.22,
862         "max_probes": 6,
863         "loss_recovery_Q4": True,
864         "bearing_halfwidth_deg": BEARING_ERROR_DEG,
865         "Q2_source_samples": second["source_samples"],
866         "Q2_second_errors_deg": second["second_error_values_deg"],
867         "Q2_optimality": "Finite candidate/sample comparison only, no global or continuous minimax
certificate",
868     },
869     "distributions": {
870         "random": "N uniform integer 10..16; distinct channels uniformly sampled; positions iid area-
uniform radius-1800 disk; radii uniform [1000,1500]; orientations uniform [0,2pi). Q3 all omni; Q4 uniform
directional count 1..N-1 and uniform subset.",
871         "minimum_radius": "Random geometry, radio radius exactly 1000 m.",
872         "outward_boundary": "16 equally spaced boundary sources with seeded rotation, radius 1000; Q4
15 outward directional and one omni.",
873         "clustered": "16 area-uniform sources in radius-35 disk with center at radius1650; radio
radius1000.",
874     },
875     "error_fields": {
876         "smooth": "Deterministic bounded two-sinusoid field.",
877         "extreme": "Coordinate/seed/channel hash selecting +/-1 degree.",
878         "iid_location": "Coordinate/seed/channel hash uniform [-1,1], not independent repeat
measurements.",
879     },
880     "trajectory_evidence": "trajectory_q3.json and trajectory_q4.json preserve complete observable logs
, accepted paths, successful robot clearing locations and post-exit aggregate summaries. No hidden source
coordinates inspected.",
881     "determinism": "PDF creation timestamps suppressed; no random plotting jitter. Raw real-duration/
timing observations may vary between replays; figures use virtual time only.",
882 },
883 )
884 print(
885     "Generated six local-evidence PDFs, five tabular fragments, numbers.tex, metadata.json and two complete
observable replay logs."
886 )
887
888
889 if __name__ == "__main__":
890     main()

```

Listing 15: PDF 逐页渲染与结构审计

```

1  """Render every PDF page and record structural/layout evidence for visual review."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import json
8  import re
9  from pathlib import Path
10
11  import pymupdf
12  from PIL import Image, ImageDraw
13
14
15  def audit(pdf_path: Path, prefix: str, dpi: int = 120):
16      directory = Path(__file__).resolve().parent / "report" / "visual"
17      directory.mkdir(parents=True, exist_ok=True)
18      document = pymupdf.open(pdf_path)
19      pages, outside, thin, replacements = [], [], [], 0
20      for number, page in enumerate(document, 1):
21          text = page.get_text()
22          replacements += text.count("\ufffd")
23          if len(text.strip()) < 15:
24              thin.append(number)

```

```

25     spans = [
26         span
27         for block in page.get_text("dict")["blocks"]
28         if "lines" in block
29         for line in block["lines"]
30         for span in line["spans"]
31         if span["text"].strip()
32     ]
33     for span in spans:
34         rect = pymupdf.Rect(span["bbox"])
35         if (
36             rect.x0 < -0.5
37             or rect.y0 < -0.5
38             or rect.x1 > page.rect.width + 0.5
39             or rect.y1 > page.rect.height + 0.5
40         ):
41             outside.append(
42                 {"page": number, "text": span["text"], "bbox": list(rect)}
43             )
44     image = directory / f"{prefix}_{number:03d}.png"
45     page.get_pixmap(dpi=dpi).save(image)
46     pages.append(
47         {
48             "page": number,
49             "characters": len(text),
50             "image": image.name,
51             "smallest_text_pt": min((s["size"] for s in spans), default=None),
52             "largest_text_pt": max((s["size"] for s in spans), default=None),
53         }
54     )
55     sheets = []
56     for start in range(0, len(pages), 6):
57         width, height, gutter = 600, 880, 15
58         sheet = Image.new(
59             "RGB", (3 * width + 4 * gutter, 2 * height + 3 * gutter), "#d4d4d4"
60         )
61         draw = ImageDraw.Draw(sheet)
62         for offset, record in enumerate(pages[start : start + 6]):
63             picture = Image.open(directory / record["image"]).convert("RGB")
64             picture.thumbnail((width, height - 28), Image.Resampling.LANCZOS)
65             x = gutter + (offset % 3) * (width + gutter)
66             y = gutter + (offset // 3) * (height + gutter)
67             draw.text((x + 8, y + 4), f"Page {record['page']}", fill="black")
68             sheet.paste(picture, (x + (width - picture.width) // 2, y + 27))
69             name = f"{prefix}_sheet_{start // 6 + 1:02d}.png"
70             sheet.save(directory / name)
71             sheets.append(name)
72     endbody = None
73     auxiliary = pdf_path.with_suffix(".aux")
74     if auxiliary.exists():
75         match = re.search(
76             r"\newlabel{endbody}\{[^\}]*\}(\d+)\}", auxiliary.read_text()
77         )
78         if match:
79             endbody = int(match[1])
80     result = {
81         "pdf": str(pdf_path),
82         "sha256": hashlib.sha256(pdf_path.read_bytes()).hexdigest(),
83         "pages": len(document),
84         "size_mib": pdf_path.stat().st_size / 2**20,
85         "body_end_page": endbody,
86         "body_pages_excluding_abstract": endbody - 1 if endbody else None,
87         "abstract_keywords_on_first_page": "关键词" in document[0].get_text(),
88         "out_of_page_spans": outside,
89         "replacement_characters": replacements,
90         "near_empty_pages": thin,
91         "rendered_all_pages": True,

```

```

92     "dpi": dpi,
93     "contact_sheets": sheets,
94     "page_records": pages,
95     "visual_review_status": "rendered_for_inspection_not_automatically_approved",
96 }
97 output = directory / f"{prefix}_audit.json"
98 output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n")
99 if outside or replacements:
100     raise RuntimeError(f"PDF has structural text issues; inspect {output}")
101 if prefix == "paper" and (
102     endbody is None
103     or endbody - 1 > 30
104     or not result["abstract_keywords_on_first_page"]
105 ):
106     raise RuntimeError(
107         f"Paper abstract/body page requirement failed; inspect {output}"
108     )
109 print(
110     json.dumps(
111         {k: v for k, v in result.items() if k != "page_records"},
112         ensure_ascii=False,
113         indent=2,
114     )
115 )
116 return result
117
118
119 if __name__ == "__main__":
120     parser = argparse.ArgumentParser()
121     parser.add_argument("--pdf", type=Path, default=Path("report/paper.pdf"))
122     parser.add_argument("--prefix", default="paper")
123     parser.add_argument("--dpi", type=int, default=120)
124     args = parser.parse_args()
125     audit(args.pdf, args.prefix, args.dpi)

```

Listing 16: 匿名材料打包与解压复现验证

```

1  """Build an anonymous, reproducible RESEARCH archive; official runs are deferred."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import importlib.metadata
8  import json
9  import os
10 import re
11 import shutil
12 import subprocess
13 import sys
14 import tempfile
15 import time
16 import zipfile
17 from pathlib import Path, PurePosixPath
18
19 import pymupdf
20
21 ROOT = Path(__file__).resolve().parent
22 SOURCES = (
23     "geometry.py",
24     "coverage.py",
25     "planner.py",
26     "environment.py",
27     "client.py",
28     "run.py",
29     "official_run.py",
30     "intersection.py",

```

```

31     "second_point.py",
32     "experiments.py",
33     "compare_recovery.py",
34     "compare_routes.py",
35     "compare_probes.py",
36     "verify_geometry.py",
37     "verify_coverage_limits.py",
38     "verify_refinements.py",
39     "verify_terminal.py",
40     "verify_http.py",
41     "make_enhancement_assets.py",
42     "make_assets.py",
43     "audit_pdf.py",
44     "package.py",
45     "active_policy_candidate.py",
46     "route_policy_candidate.py",
47     "active_policy_notes.txt",
48     "route_policy_notes.txt",
49     "polar_coverage_candidate.py",
50     "polar_coverage_notes.txt",
51     "joint_dispatch_candidate.py",
52     "joint_dispatch_notes.txt",
53     "optical_policy_candidate.py",
54     "optical_policy_notes.txt",
55     "refinement_study.py",
56     "research_experiments.py",
57     "verify_research.py",
58     "make_research_assets.py",
59     "contracts.json",
60     "report_contracts.json",
61     "enhancements.py",
62     "ring_sweep.py",
63     "develop_anchors.py",
64     "theory_additions.txt",
65     "paper_expansion.txt",
66     "coverage_notes.txt",
67     "coverage_limits.tex",
68     "requirements.txt",
69     "reproduce.txt",
70     "official_status.json",
71 )
72 RESULTS = (
73     "final_experiments.json",
74     "geometry_verification.json",
75     "intersection_verification.json",
76     "second_point_example.json",
77     "recovery_development.json",
78     "route_candidate_comparison.json",
79     "adaptive_candidate_comparison.json",
80     "terminal_verification.json",
81     "ring_sweep.json",
82     "enhancement_experiments.json",
83     "anchor_development.json",
84     "enhancement_review.json",
85     "coverage_limits.json",
86     "refinement_verification.json",
87     "refinement_development.json",
88     "research_experiments.json",
89     "research_verification.json",
90     "research_review.json",
91     "http_verification.json",
92     "http_deadline_verification.json",
93     "review_resolutions.json",
94     "operation_bound.json",
95     "paper_review.json",
96     "visual_review.json",
97     "final_review.json",

```

```

98     "environment_manifest.json",
99 )
100
101
102 def verify_asset_inputs(root: Path):
103     for metadata_name in ("metadata.json", "enhancement_metadata.json", "research_metadata.json"):
104         metadata = json.loads((root / "report/generated" / metadata_name).read_text())
105         for name, expected in metadata["input_sha256"].items():
106             actual = hashlib.sha256((root / name).read_bytes()).hexdigest()
107             if actual != expected:
108                 raise ValueError(f"Stale generated asset input: {name}")
109
110
111 def deterministic_content(value):
112     """Exclude measured wall clocks and their maximum, not behavioral statistics."""
113     if isinstance(value, dict):
114         return {key: deterministic_content(item) for key, item in value.items()
115                 if key not in ("real_duration_s", "max_real_duration_s")}
116     if isinstance(value, list):
117         return [deterministic_content(item) for item in value]
118     return value
119
120
121 def verify_archive(archive: Path):
122     records = []
123     with tempfile.TemporaryDirectory(prefix="b-reproduce-") as directory:
124         extracted = Path(directory)
125         with zipfile.ZipFile(archive) as bundle:
126             for item in bundle.infolist():
127                 relative = PurePosixPath(item.filename)
128                 if relative.is_absolute() or ".." in relative.parts:
129                     raise ValueError("Unsafe member in generated archive")
130                 bundle.extractall(extracted)
131             manifest = json.loads((extracted / "MANIFEST.json").read_text())
132             for name, information in manifest["files"].items():
133                 raw = (extracted / name).read_bytes()
134                 assert hashlib.sha256(raw).hexdigest() == information["sha256"], name
135             verify_asset_inputs(extracted)
136             expected_runs = {
137                 "final_experiments.json": 4430,
138                 "enhancement_experiments.json": 3080,
139                 "ring_sweep.json": 2200,
140                 "refinement_development.json": 2550,
141                 "research_experiments.json": 10700,
142             }
143             baselines = {
144                 name: json.loads((extracted / "results" / name).read_text())
145                 for name in (*expected_runs, "research_verification.json")
146             }
147             commands = [
148                 [sys.executable, "verify_geometry.py"],
149                 [sys.executable, "intersection.py"],
150                 [sys.executable, "second_point.py"],
151                 [
152                     sys.executable,
153                     "experiments.py",
154                     "--cases",
155                     "500",
156                     "--output",
157                     "results/final_experiments.json",
158                 ],
159                 [sys.executable, "enhancements.py"],
160                 [sys.executable, "ring_sweep.py"],
161                 [sys.executable, "verify_terminal.py"],
162                 [sys.executable, "verify_refinements.py"],
163                 [sys.executable, "verify_coverage_limits.py"],
164                 [sys.executable, "compare_routes.py"],

```

```

165     [sys.executable, "compare_probes.py"],
166     [sys.executable, "verify_research.py"],
167     [sys.executable, "refinement_study.py", "--cases", "150", "--seed", "3100000"],
168     [sys.executable, "research_experiments.py"],
169     [sys.executable, "make_assets.py"],
170     [sys.executable, "make_enhancement_assets.py"],
171     [sys.executable, "make_research_assets.py"],
172     [
173         "latexmk",
174         "-xelatex",
175         "-interaction=nonstopmode",
176         "-halt-on-error",
177         "paper.tex",
178     ],
179     [
180         "latexmk",
181         "-xelatex",
182         "-interaction=nonstopmode",
183         "-halt-on-error",
184         "ai_usage.tex",
185     ],
186 ]
187 for command in commands:
188     started = time.monotonic()
189     cwd = extracted / "report" if command[0] == "latexmk" else extracted
190     completed = subprocess.run(
191         command,
192         cwd=cwd,
193         text=True,
194         stdout=subprocess.PIPE,
195         stderr=subprocess.STDOUT,
196         timeout=1800,
197         check=False,
198         env={**os.environ, "MPLBACKEND": "Agg"},
199     )
200     records.append(
201         {
202             "command": [Path(command[0]).name, *command[1:]],
203             "returncode": completed.returncode,
204             "seconds": time.monotonic() - started,
205         }
206     )
207     if completed.returncode:
208         raise RuntimeError(
209             f"Extracted archive command failed: {command}\n{completed.stdout[-12000:]}"
210         )
211 reproduced_runs = {}
212 for name, baseline in baselines.items():
213     replay = json.loads((extracted / "results" / name).read_text())
214     if name in expected_runs:
215         expected = expected_runs[name]
216         assert baseline["total_runs"] == replay["total_runs"] == expected, name
217         if name == "ring_sweep.json":
218             count = sum(len(group["runs"]) for phase in ("development", "held_out")
219                          for group in replay[phase].values())
220         else:
221             count = len(replay["runs"])
222             for old, new in zip(baseline["runs"], replay["runs"], strict=True):
223                 assert deterministic_content(old) == deterministic_content(new), (name, old["seed"])
224             assert count == expected, (name, count)
225         reproduced_runs[name] = count
226     assert deterministic_content(baseline) == deterministic_content(replay), name
227 verify_asset_inputs(extracted)
228 for filename in ("paper", "ai_usage"):
229     log = (extracted / "report" / f"{filename}.log").read_text(errors="replace")
230     assert "Missing character:" not in log and "Overfull" not in log
231     assert (

```

```

232         "undefined references" not in log and "undefined citations" not in log
233     )
234     assert len(pymupdf.open(extracted / "report" / f"{filename}.pdf")) > 0
235 return {
236     "status": "passed",
237     "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
238     "manifest_hashes_verified": True,
239     "all_4430_behavioral_runs_reproduced_exactly": True,
240     "all_3080_enhancement_runs_reproduced_exactly": True,
241     "ring_sweep_reproduced_exactly": True,
242     "all_2200_ring_runs_reproduced_exactly": True,
243     "all_10700_research_runs_reproduced_exactly": True,
244     "all_2550_development_runs_reproduced_exactly": True,
245     "research_verification_reproduced_exactly": True,
246     "generated_asset_input_hashes_verified": True,
247     "experiment_runs_reproduced_by_dataset": reproduced_runs,
248     "experiment_runs_reproduced_total": sum(reproduced_runs.values()),
249     "excluded_nondeterminism": "measured wall-clock real_duration_s and its summary maximum
max_real_duration_s only",
250     "commands": records,
251 }
252
253
254 def main():
255     parser = argparse.ArgumentParser()
256     parser.add_argument("--verify", action="store_true")
257     args = parser.parse_args()
258     verify_asset_inputs(ROOT)
259     for prefix, name in (("paper", "paper"), ("ai", "ai_usage")):
260         audit = json.loads((ROOT / "report/visual" / f"{prefix}_audit.json").read_text())
261         actual = hashlib.sha256((ROOT / "report" / f"{name}.pdf").read_bytes()).hexdigest()
262         if audit["sha256"] != actual or audit["out_of_page_spans"] or audit["replacement_characters"]:
263             raise ValueError(f"Missing or stale PDF audit: {name}")
264         if prefix == "paper" and not (audit["abstract_keywords_on_first_page"]
265                                     and 0 < audit["body_pages_excluding_abstract"] <= 30):
266             raise ValueError("Paper abstract/body page rule violated")
267     environment = {
268         "python": sys.version,
269         "environment": "conda py314",
270         "packages": {
271             name: importlib.metadata.version(name)
272             for name in ("numpy", "scipy", "matplotlib", "pymupdf")
273         },
274     }
275     (ROOT / "results/environment_manifest.json").write_text(
276         json.dumps(environment, indent=2) + "\n"
277     )
278     files = {name: ROOT / name for name in SOURCES}
279     files.update({"results/" + name: ROOT / "results" / name for name in RESULTS})
280     files["AI工具使用详情.pdf"] = ROOT / "report/ai_usage.pdf"
281     for path in (ROOT / "report").iterdir():
282         if path.suffix in (".tex", ".pdf"):
283             files[path.relative_to(ROOT).as_posix()] = path
284     for folder in ("generated", "figures"):
285         for path in (ROOT / "report" / folder).iterdir():
286             if path.is_file() and not path.name.startswith("."):
287                 files[path.relative_to(ROOT).as_posix()] = path
288     for filename in ("paper_audit.json", "ai_audit.json"):
289         path = ROOT / "report" / "visual" / filename
290         files[path.relative_to(ROOT).as_posix()] = path
291     missing = [name for name, path in files.items() if not path.is_file()]
292     if missing:
293         raise RuntimeError(f"Deliverable proof is missing: {missing}")
294     anonymous_pattern = re.compile(r"/Users/[A-Za-z0-9._-]+/")
295     for name, path in files.items():
296         if path.suffix in (".py", ".txt", ".tex", ".json") and anonymous_pattern.search(
297             path.read_text(encoding="utf-8")

```

```

298     ):
299         raise ValueError(f"Personal absolute path in anonymous material: {name}")
300     if path.suffix == ".pdf":
301         document = pymupdf.open(path)
302         if anonymous_pattern.search(
303             "\n".join(page.get_text() for page in document)
304         ):
305             raise ValueError(f"Personal absolute path in PDF: {name}")
306     manifest = {
307         "status": "VERIFIED_RESEARCH_VERSION_OFFICIAL_TESTS_DEFERRED",
308         "not_ready_for_contest_submission": True,
309         "deferred_by_user": "Official simulator practice/formal runs and six encrypted logs",
310         "human_review": "No actual contestant review record; AI review is not human review",
311         "python_environment": "conda py314",
312         "python_version": sys.version.split()[0],
313         "files": {
314             name: {
315                 "sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
316                 "bytes": path.stat().st_size,
317             }
318             for name, path in sorted(files.items())
319         },
320     }
321     destination = ROOT / "delivery"
322     destination.mkdir(exist_ok=True)
323     archive = destination / "B题研究支撑材料.zip"
324     with zipfile.ZipFile(
325         archive, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9
326     ) as bundle:
327         for name, path in sorted(files.items()):
328             bundle.write(path, name)
329         bundle.writestr(
330             "MANIFEST.json", json.dumps(manifest, ensure_ascii=False, indent=2) + "\n"
331         )
332     with zipfile.ZipFile(archive) as bundle:
333         assert bundle.testzip() is None
334     shutil.copy2(ROOT / "report/paper.pdf", destination / "B题研究论文.pdf")
335     shutil.copy2(ROOT / "report/ai_usage.pdf", destination / "AI工具使用详情.pdf")
336     for path in (
337         archive,
338         destination / "B题研究论文.pdf",
339         destination / "AI工具使用详情.pdf",
340     ):
341         assert path.stat().st_size <= 20 * 2**20, path.name
342     status = {
343         "status": manifest["status"],
344         "official_tests": "DEFERRED_BY_USER",
345         "human_review": "OUTSTANDING",
346         "source_files": len(files),
347         "archive_bytes": archive.stat().st_size,
348         "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
349     }
350     if args.verify:
351         proof = verify_archive(archive)
352         (destination / "复现验证.json").write_text(
353             json.dumps(proof, ensure_ascii=False, indent=2) + "\n"
354         )
355         status["extracted_archive_reproduction"] = "passed"
356     (destination / "交付状态.json").write_text(
357         json.dumps(status, ensure_ascii=False, indent=2) + "\n"
358     )
359     print(json.dumps(status, ensure_ascii=False, indent=2))
360
361
362 if __name__ == "__main__":
363     main()

```

Listing 17: 区间评分自适应测点候选

```

1  """Unexecuted research candidate: observation-only, bounded active probing.
2
3  No action or belief update occurs here. ``None`` means use the unchanged
4  localizer/optical cover; a selected point is NOT a clearing certificate.
5  """
6
7  from __future__ import annotations
8
9  from math import acos, cos, dist, hypot, isfinite, nextafter, pi, radians, sin
10
11  from geometry import (
12      BEARING_ERROR_DEG,
13      MAX_RADIO_RADIUS,
14      Point,
15      _clip_halfplane,
16      _padding,
17      enclosing_circle,
18  )
19  from second_point import safe_candidates
20
21
22  def _bearing_bound(polygon, point, radius, bins, error_deg):
23      """Upper-bound every positive-bearing posterior using overlapping wedges.
24
25      Each angular bin is enlarged by the measurement error; unlike clip_bearing,
26      the scoring relaxation has NO projected range cap. A bounding-box circle
27      encloses each relaxed polygon. The original MEC is also an enclosing disk.
28      """
29      half_bin = 180.0 / bins
30      width = radians(error_deg + half_bin)
31      scale = max(
32          MAX_RADIO_RADIUS,
33          abs(point[0]),
34          abs(point[1]),
35          *(max(abs(x), abs(y)) for x, y in polygon),
36      )
37      # A padded narrow wedge is a translated exact wedge. Its apex shift is
38      # epsilon/sin(alpha); this larger pad also covers that displacement.
39      padding = 4.0 * _padding(scale) / sin(radians(error_deg))
40      worst, nonempty = 0.0, 0
41      for index in range(bins):
42          angle = 2.0 * pi * index / bins
43          low, high = angle - width, angle + width
44          outer = _clip_halfplane(
45              polygon, point, (sin(low), -cos(low)), 0.0, padding
46          )
47          outer = _clip_halfplane(
48              outer, point, (-sin(high), cos(high)), 0.0, padding
49          )
50          if not outer:
51              continue
52          nonempty += 1
53          xmin = min(p[0] for p in outer)
54          xmax = max(p[0] for p in outer)
55          ymin = min(p[1] for p in outer)
56          ymax = max(p[1] for p in outer)
57          box_radius = nextafter(
58              0.5 * hypot(xmax - xmin, ymax - ymin) + padding, float("inf")
59          )
60          worst = max(worst, min(radius, box_radius))
61      # An unexpected empty partition must not look like perfect information.
62      return (worst if nonempty else radius), nonempty
63
64
65  def _loss_risk(samples, readings, point, problem):
66      """Geometric risk index, NOT an identified no-signal probability."""

```

```

67     total = 0.0
68     station = readings[-1][0]
69     for source in samples:
70         query_distance = dist(point, source)
71         # A positive station implies range >= its distance, even in Q4.
72         lower_range = max(1000.0, *(dist(s, source) for s, _ in readings))
73         if query_distance > lower_range:
74             total += 1.0
75             continue
76         if problem == 3:
77             continue
78         old_distance = dist(station, source)
79         if old_distance <= 1e-9 or query_distance <= 1e-9:
80             # Direction is undefined here; do not manufacture guaranteed signal.
81             total += 1.0
82             continue
83         cosine = sum(
84             (station[i] - source[i]) * (point[i] - source[i]) for i in (0, 1)
85         ) / (old_distance * query_distance)
86         total += acos(max(-1.0, min(1.0, cosine))) / pi
87     return total / len(samples)
88
89
90 def score_probes(
91     polygon,
92     readings,
93     current_position,
94     problem,
95     *,
96     probes_remaining=1,
97     rejected_points=(),
98     bearing_bins=72,
99     error_deg=BEARING_ERROR_DEG,
100     probe_scale=0.22,
101     movement_weight=0.15,
102     loss_weight=1.0,
103     include_lens=True,
104 ):
105     """Return a deterministic score table; consume only public observations.
106
107     polygon: nonempty ordered convex conservative vertices, as Track.polygon.
108     readings: positive (station, svd_deg) pairs for ONE known channel only.
109     rejected_points: previous unsuccessful probes/clears; excludes locations,
110         never source hypotheses. The caller retains all underlying history.
111     probes_remaining: caller-owned remaining bound; this function cannot debit it.
112
113     Legal coordinates are finite and within +/-2,000,000 m per protocol, NOT
114     restricted to the radius-1800 source arena. Scores are dimensionless.
115     Empty output signals fallback. Malformed/contradictory inputs raise, rather
116     than resetting a belief or silently substituting a made-up polygon.
117     """
118     if problem not in (3, 4):
119         raise ValueError("Only Q3 and Q4 localization is supported")
120     if not isinstance(probes_remaining, int) or probes_remaining < 0:
121         raise ValueError("probes_remaining must be a nonnegative integer")
122     if not isinstance(bearing_bins, int) or bearing_bins < 4:
123         raise ValueError("bearing_bins must be an integer >= 4")
124     if not isfinite(error_deg) or not BEARING_ERROR_DEG <= error_deg < 90:
125         raise ValueError("error_deg must include the public rounding allowance")
126     if error_deg + 180.0 / bearing_bins >= 90.0:
127         raise ValueError("Expanded bearing bins must be narrower than 180 degrees")
128     if not isfinite(probe_scale) or not 0 < probe_scale <= 1:
129         raise ValueError("probe_scale must lie in (0, 1]")
130     if any(not isfinite(w) or w < 0 for w in (movement_weight, loss_weight)):
131         raise ValueError("Score weights must be finite and nonnegative")
132     poly = [tuple(p) for p in polygon]
133     history = [(tuple(s), float(a)) for s, a in readings]

```

```

134 position = tuple(current_position)
135 rejected = [tuple(p) for p in rejected_points]
136 points = poly + [position] + rejected + [s for s, _ in history]
137 if any(len(p) != 2 or not all(isfinite(v) for v in p) for p in points):
138     raise ValueError("All points must contain two finite coordinates")
139 if any(not isfinite(a) for _, a in history):
140     raise ValueError("Readings must have finite public bearing angles")
141 center, radius = enclosing_circle(poly) # Empty belief must raise.
142 if not history or probes_remaining == 0 or radius <= 19.9:
143     return []
144
145 station, bearing = history[-1]
146 dx, dy = center[0] - station[0], center[1] - station[1]
147 length = hypot(dx, dy)
148 axis = (dx / length, dy / length) if length > 1e-8 else (
149     cos(radians(bearing)), sin(radians(bearing))
150 )
151 normal = (-axis[1], axis[0])
152 offset = min(120.0, max(25.0, probe_scale * radius))
153 candidates = [("mec_center", center), ("positive_station", station)]
154 for name, direction in (("normal", normal), ("axis", axis)):
155     for sign in (-1, 1):
156         candidates.append((
157             f"mec_{name}_{sign:+d}",
158             tuple(center[i] + sign * offset * direction[i] for i in (0, 1)),
159         ))
160 if include_lens:
161     candidates.extend(
162         (f"q2_lens_{index}", tuple(row["point"]))
163         for index, row in enumerate(safe_candidates(station, bearing, error_deg))
164     )
165
166 samples = list(dict.fromkeys(poly + [center]))
167 exclusions = [s for s, _ in history] + rejected
168 table, seen = [], set()
169 for label, point in candidates:
170     if point in seen:
171         continue
172     seen.add(point)
173     reason = None
174     if any(not isfinite(v) or abs(v) > 2_000_000 for v in point):
175         reason = "outside_protocol_coordinates"
176     elif any(dist(point, old) <= 0.5 for old in exclusions):
177         reason = "previous_observation_or_rejected_location"
178     row = {
179         "candidate": label,
180         "point": point,
181         "eligible": reason is None,
182         "rejection": reason,
183         "score": None,
184     }
185     if reason is None:
186         bound, bins_used = _bearing_bound(
187             poly, point, radius, bearing_bins, error_deg
188         )
189         movement = dist(position, point)
190         risk = _loss_risk(samples, history, point, problem)
191         row.update({
192             "prior_mec_radius_m": radius,
193             "positive_bearing_radius_upper_m": bound,
194             "movement_m": movement,
195             "no_signal_risk_index": risk,
196             "radius_term": bound / radius,
197             "movement_term": movement_weight * movement / 1000.0,
198             "loss_term": loss_weight * risk,
199             "worst_including_uninformative_no_signal_m": radius,
200             "nonempty_bearing_bins": bins_used,

```

```

201         "bearing_bins": bearing_bins,
202         "error_deg": error_deg,
203         "risk_samples": len(samples),
204         "score": (
205             bound / radius
206             + movement_weight * movement / 1000.0
207             + loss_weight * risk
208         ),
209     })
210     table.append(row)
211     return table
212
213
214 def choose_probe(
215     polygon, readings, current_position, problem, **options
216 ) -> Point | None:
217     """Choose minimum score, then minimum travel, then generation-order tie.
218
219     The caller MUST debit its existing finite probe budget, apply only actual
220     positive bearings via Planner.measure, and retain the original fallback.
221     Use score_probes with identical arguments to obtain the full score table.
222     """
223     table = score_probes(polygon, readings, current_position, problem, **options)
224     eligible = [row for row in table if row["eligible"]]
225     if not eligible:
226         return None
227     best = min(eligible, key=lambda row: (row["score"], row["movement_m"]))
228     return best["point"]

```

Listing 18: 骨架路由候选

```

1  """Deterministic routing and task-priority candidates.
2
3  These helpers are deliberately separate from :mod:`planner`: they can improve
4  travel order without changing the certified waypoint set or its discovery
5  obligations. No simulator/source-truth state is read here.
6  """
7
8  from __future__ import annotations
9
10 import math
11 from collections.abc import Sequence
12 from numbers import Real
13
14
15 def _point(value: object) -> tuple[float, float] | None:
16     if isinstance(value, (str, bytes)) or not isinstance(value, Sequence) or len(value) != 2:
17         return None
18     x, y = value
19     if not isinstance(x, Real) or not isinstance(y, Real):
20         return None
21     x, y = float(x), float(y)
22     return (x, y) if math.isfinite(x) and math.isfinite(y) else None
23
24
25 def optimize_waypoints(points: Sequence[object], start: tuple[float, float] = (0.0, 0.0)) -> list[int]:
26     """Return a deterministic nearest-neighbour + 2-opt index order.
27
28     Every valid input point occurs exactly once. Invalid point data, or an
29     invalid start, uses the exact identity order as a safe, obligation-
30     preserving fallback; it never silently removes or fabricates a waypoint.
31     The 2-opt pass only reverses contiguous order segments, so it changes
32     travel length but cannot change the waypoint certificate.
33     """
34     try:
35         n = len(points)
36     except (TypeError, AttributeError):

```

```

37     return []
38 fallback = list(range(n))
39 origin = _point(start)
40 if origin is None:
41     return fallback
42 coords = [_point(p) for p in points]
43 if any(p is None for p in coords):
44     return fallback
45 xy = [p for p in coords if p is not None]
46 remaining = set(range(n))
47 route: list[int] = []
48 current = origin
49 while remaining:
50     index = min(remaining, key=lambda i: ((xy[i][0] - current[0]) ** 2 + (xy[i][1] - current[1]) ** 2, i))
51     route.append(index)
52     remaining.remove(index)
53     current = xy[index]
54
55 def edge(a: tuple[float, float], b: tuple[float, float]) -> float:
56     return math.hypot(a[0] - b[0], a[1] - b[1])
57
58 # First-improvement, lexicographically scanned, strict 2-opt is stable.
59 # Repeat until no improving exchange remains.
60 changed = True
61 while changed:
62     changed = False
63     for i in range(-1, n - 2): # edge from start (i=-1) or route[i]
64         a = origin if i == -1 else xy[route[i]]
65         ai = i + 1
66         for j in range(ai + 1, n):
67             b = xy[route[ai]]
68             c = xy[route[j]]
69             d = xy[route[j + 1]] if j + 1 < n else None
70             old = edge(a, b) + (edge(c, d) if d is not None else 0.0)
71             new = edge(a, c) + (edge(b, d) if d is not None else 0.0)
72             if new + 1e-12 < old:
73                 route[ai : j + 1] = reversed(route[ai : j + 1])
74                 changed = True
75                 break
76         if changed:
77             break
78     return route

```

Listing 19: 路由候选对照实验

```

1  """Paired route-order candidate comparison on declared synthetic worlds."""
2  from __future__ import annotations
3
4  import json
5  import statistics
6  from pathlib import Path
7
8  from environment import LocalSimulator
9  from planner import Planner
10
11
12 def run(seed, problem, anchor_policy):
13     env = LocalSimulator(seed, problem=problem, scenario="random", error_mode="smooth")
14     planner = Planner(env, problem=problem, strategy="integrated", anchor_policy=anchor_policy)
15     planner.run()
16     result = env.summary()
17     assert result["fraction_cleared"] == 1
18     return result
19
20
21 def main():
22     output = {}

```

```

23 records = []
24 for problem in (3, 4):
25     pairs = [(run(950000 + k, problem, "nearest"), run(950000 + k, problem, "two_opt")) for k in range(200)
26 ]
27     records.extend({"problem": problem, "seed": 950000+k,
28                     "nearest": a, "two_opt": b} for k, (a, b) in enumerate(pairs))
29     deltas = [a["average_time_s"] - b["average_time_s"] for a, b in pairs]
30     output[f"Q{problem}"] = {
31         "cases": len(pairs),
32         "all_nearest_cleared": sum(a["fraction_cleared"] == 1 for a, _ in pairs),
33         "all_two_opt_cleared": sum(b["fraction_cleared"] == 1 for _, b in pairs),
34         "nearest_mean_s_per_source": statistics.mean(a["average_time_s"] for a, _ in pairs),
35         "two_opt_mean_s_per_source": statistics.mean(b["average_time_s"] for _, b in pairs),
36         "mean_saved_s_per_source": statistics.mean(deltas),
37         "median_saved_s_per_source": statistics.median(deltas),
38         "faster_cases": sum(d > 0 for d in deltas),
39         "mean_distance_saved_m": statistics.mean(a["distance_m"] - b["distance_m"] for a, b in pairs),
40     }
41     result = {"evidence": "local_synthetic_route_candidate_not_official", "seed_range": [950000, 950199], "
42             summary": output, "runs": records}
43     Path("results/route_candidate_comparison.json").write_text(json.dumps(result, ensure_ascii=False, indent=2)
44         + "\n")
45     print(json.dumps(result, ensure_ascii=False, indent=2))
46
47 if __name__ == "__main__":
48     main()

```

Listing 20: 自适应测点对照实验

```

1  """Paired comparison for the interval-scored adaptive probe candidate."""
2  from __future__ import annotations
3
4  import json
5  import statistics
6  from pathlib import Path
7
8  from environment import LocalSimulator
9  from planner import Planner
10
11
12 def run(seed, problem, probe_policy):
13     env = LocalSimulator(seed, problem=problem, scenario="random", error_mode="smooth")
14     planner = Planner(env, problem=problem, strategy="integrated", probe_policy=probe_policy)
15     planner.run()
16     summary = env.summary()
17     assert summary["fraction_cleared"] == 1
18     return {**summary, "adaptive_probes": planner.adaptive_probes}
19
20
21 def aggregate(rows):
22     delta = [a["average_time_s"] - b["average_time_s"] for a, b in rows]
23     return {
24         "cases": len(rows),
25         "fixed_all_cleared": sum(a["fraction_cleared"] == 1 for a, _ in rows),
26         "adaptive_all_cleared": sum(b["fraction_cleared"] == 1 for _, b in rows),
27         "fixed_mean_s_per_source": statistics.mean(a["average_time_s"] for a, _ in rows),
28         "adaptive_mean_s_per_source": statistics.mean(b["average_time_s"] for _, b in rows),
29         "mean_saved_s_per_source": statistics.mean(delta),
30         "median_saved_s_per_source": statistics.median(delta),
31         "faster_cases": sum(d > 0 for d in delta),
32         "fixed_mean_failed_clears": statistics.mean(a["failed_clears"] for a, _ in rows),
33         "adaptive_mean_failed_clears": statistics.mean(b["failed_clears"] for _, b in rows),
34         "mean_adaptive_probes": statistics.mean(b["adaptive_probes"] for _, b in rows),
35     }
36
37

```

```

38 def main():
39     summary = {}
40     records = []
41     for problem in (3, 4):
42         pairs = [(run(960000 + k, problem, "fixed"), run(960000 + k, problem, "adaptive")) for k in range(200)]
43         summary[f"Q{problem}"] = aggregate(pairs)
44         records.extend({"problem": problem, "seed": 960000 + k, "fixed": fixed, "adaptive": adaptive}
45                        for k, (fixed, adaptive) in enumerate(pairs))
46     result = {"evidence": "local_synthetic_adaptive_candidate_not_official", "seed_range": [960000, 960199], "
47              summary": summary, "runs": records}
48     Path("results/adaptive_candidate_comparison.json").write_text(json.dumps(result, ensure_ascii=False, indent
49                             =2) + "\n")
50     print(json.dumps({"evidence": result["evidence"], "summary": summary}, ensure_ascii=False, indent=2))
51
52 if __name__ == "__main__":
53     main()

```

Listing 21: 超时终端状态回归

```

1  """Finite Planner terminal/deadline regression using a protocol-shaped fake adapter."""
2  from __future__ import annotations
3
4  import json
5  from pathlib import Path
6
7  from planner import Planner
8
9
10 class DeadlineAdapter:
11     def __init__(self, remaining=1200, max_virtual=10, near_channel=None):
12         self.remaining = remaining
13         self.max_virtual = max_virtual
14         self.virtual = 0.0
15         self.channel = 1
16         self.paths = []
17         self.near_channel = near_channel
18
19     def act(self, path, position=None, channel=None):
20         self.paths.append(path)
21         if path == "/enter":
22             return {"accepted": True, "virtual_time_s": 0.0,
23                    "remaining_real_duration_s": self.remaining,
24                    "max_virtual_duration_s": self.max_virtual}
25         if path == "/exit":
26             if self.virtual >= self.max_virtual:
27                 raise AssertionError("Planner sent /exit after automatic virtual close")
28             return {"accepted": True, "virtual_time_s": self.virtual,
29                    "exit_reason": "user_exit"}
30         if path == "/clear":
31             self.virtual += 5
32             return {"accepted": True, "virtual_time_s": self.virtual, "clear_result": "success"}
33         switch = int(path == "/measure" and channel != self.channel)
34         self.virtual += 5 + switch
35         self.channel = channel
36         return {"accepted": True, "virtual_time_s": self.virtual,
37                "measure_result": "near" if channel == self.near_channel else "no_signal"}
38
39
40 def main():
41     automatic = DeadlineAdapter(remaining=1200, max_virtual=10)
42     result_auto = Planner(automatic, problem=3).run()
43     assert result_auto["terminal_reason"] == "virtual_timeout"
44     assert result_auto["certificate_complete"] is False
45     assert "/exit" not in automatic.paths
46
47     real = DeadlineAdapter(remaining=1, max_virtual=360000)

```

```

48 result_real = Planner(real, problem=3).run()
49 assert result_real["terminal_reason"] == "TimeoutError"
50 assert result_real["certificate_complete"] is False
51 assert "/exit" not in real.paths
52 near = DeadlineAdapter(max_virtual=119, near_channel=20)
53 policy = Planner(near, problem=3)
54 for record in policy.scan_records.values():
55     record.update(range(1,7))
56 near_result = policy.run()
57 assert near_result["channel_certificates"][20] == "detected"
58 assert not near_result["certificate_complete"] and "/clear" not in near.paths
59 assert "/exit" not in near.paths
60 completed = DeadlineAdapter(max_virtual=119)
61 policy = Planner(completed, problem=3)
62 for record in policy.scan_records.values():
63     record.update(range(1,7))
64 completed_result = policy.run()
65 assert completed_result["certificate_complete"]
66 assert completed_result["termination"] == "complete_after_virtual_timeout"
67 assert "/exit" not in completed.paths
68 final_clear = DeadlineAdapter(max_virtual=5)
69 policy = Planner(final_clear, problem=3)
70 policy.action("/enter")
71 policy.cleared.update(range(1,16))
72 policy.clear(16, (0.0,0.0))
73 assert policy.result()["certificate_complete"] and "/exit" not in final_clear.paths
74 result = {"evidence": "local_protocol_shaped_terminal_regression_not_official",
75           "automatic_virtual_timeout": result_auto,
76           "preflight_real_deadline": result_real,
77           "never_exited_after_terminal": True,
78           "near_at_limit_not_absent": True,
79           "final_absence_certificate_at_limit": True,
80           "sixteenth_clear_at_limit_complete": True,
81           "never_claimed_certificate": True}
82 Path("results/terminal_verification.json").write_text(json.dumps(result, indent=2) + "\n")
83 print(json.dumps(result, indent=2))
84
85
86 if __name__ == "__main__":
87     main()

```

Listing 22: 环半径与滚动调度开发比较

```

1  """Development-only anchor geometry and rolling route screen, no official calls."""
2  from __future__ import annotations
3
4  import json
5  import statistics
6  from pathlib import Path
7
8  from environment import LocalSimulator
9  from planner import Planner
10
11
12 def case(seed, problem, route, ring_radius=None):
13     env = LocalSimulator(seed, problem=problem)
14     policy = Planner(env, problem=problem, strategy="integrated", anchor_policy=route,
15                     ring_radius=ring_radius)
16     result = policy.run()
17     actual = env.summary()
18     assert result['certificate_complete'] and actual['fraction_cleared'] == 1
19     return actual
20
21
22 def main():
23     results = {}
24     for problem in (3, 4):

```

```

25     configs = [('nearest', None), ('rolling', None)]
26     if problem == 3:
27         configs += [('nearest', 1200.), ('rolling', 1200.)]
28     baseline = None
29     for route, radius in configs:
30         rows = [case(1000000+k, problem, route, radius) for k in range(150)]
31         if baseline is None:
32             baseline = rows
33         differences = [a['average_time_s']-b['average_time_s'] for a,b in zip(baseline,rows)]
34         results[f'Q{problem}_{route}_{radius}'] = {
35             'cases':len(rows), 'mean_s_per_source':statistics.mean(r['average_time_s'] for r in rows),
36             'mean_distance_m':statistics.mean(r['distance_m'] for r in rows),
37             'mean_measures':statistics.mean(r['measures'] for r in rows),
38             'mean_saved_s_per_source':statistics.mean(differences),
39             'faster_cases':sum(d>0 for d in differences), 'all_cleared':True,
40             'runs':rows}
41     Path('results/anchor_development.json').write_text(json.dumps(results,indent=2)+'\n')
42     print(json.dumps({k:f'v for f,v in row.items() if f!='runs'} for k,row in results.items()),indent=2))
43
44
45 if __name__ == '__main__':
46     main()

```

Listing 23: 增强策略样本外实验

```

1  """Frozen held-out comparison of accepted geometric/rolling enhancements."""
2  from __future__ import annotations
3
4  import json
5  import math
6  import statistics
7  from pathlib import Path
8
9  from coverage import ring_cover_radius
10 from experiments import aggregate, run_case
11
12
13 def paired_summary(control, enhanced):
14     assert [r['seed'] for r in control] == [r['seed'] for r in enhanced]
15     differences = [a['average_time_s']-b['average_time_s'] for a,b in zip(control, enhanced)]
16     average = statistics.mean(differences)
17     stderr = statistics.stdev(differences)/math.sqrt(len(differences))
18     return {'cases':len(differences), 'mean_saved_s_per_source':average,
19             'mean_reduction_pct':100*average/statistics.mean(r['average_time_s'] for r in control),
20             'approx_95pct_ci_s':[average-1.96*stderr, average+1.96*stderr],
21             'faster_cases':sum(d>0 for d in differences),
22             'mean_distance_saved_m':statistics.mean(a['distance_m']-b['distance_m'] for a,b in zip(control,
23             enhanced))}
24
25 def main():
26     rows, summary, comparison, stress = [], {}, {}, {}
27     for problem in (3,4):
28         paired = []
29         for policy in ('integrated','enhanced'):
30             group = [run_case(1100000+k,problem,policy) for k in range(500)]
31             rows.extend(group)
32             summary[f'Q{problem}_{policy}'] = aggregate(group)
33             paired.append(group)
34         comparison[f'Q{problem}'] = paired_summary(*paired)
35     for scenario in ('minimum_radius','outward_boundary','clustered'):
36         for mode in ('smooth','extreme','iid_location'):
37             groups = []
38             for policy in ('integrated','enhanced'):
39                 group = [run_case(1110000+k,problem,policy,scenario,mode) for k in range(30)]
40                 rows.extend(group)
41                 groups.append(group)

```

```

42         key = f'Q{problem}_{scenario}_{mode}'
43         stress[key] = {'integrated':aggregate(groups[0]), 'enhanced':aggregate(groups[1]),
44                       'paired':paired_summary(*groups)}
45     result = {'evidence':'held_out_local_synthetic_not_official',
46             'selection':'Only development seeds1000000..1000149 used; final seeds excluded from choice',
47             'main_seed_range':[1100000,1100499], 'stress_seed_range':[1110000,1110029],
48             'total_runs':len(rows), 'summary':summary, 'comparison':comparison, 'stress':stress,
49             'contracted_ring':{'radius_m':1200., 'exact_cover_radius_m':ring_cover_radius(1200.),
50                               'radio_margin_m':1000-ring_cover_radius(1200.)}, 'runs':rows}
51     Path('results/enhancement_experiments.json').write_text(json.dumps(result,indent=2)+'\n')
52     print(json.dumps({k:v for k,v in result.items() if k not in ('runs','stress')},indent=2))
53
54
55 if __name__ == '__main__':
56     main()

```

Listing 24: Q3 环半径筛选与新世界确认

```

1  """Development and held-out sweep for the safe Q3 ring radius."""
2  from __future__ import annotations
3
4  import json
5  import math
6  import statistics
7  from pathlib import Path
8
9  from coverage import ring_cover_radius
10 from experiments import run_case
11
12 CANDIDATES = (1200.0, 1250.0, 1300.0, 1350.0, 1400.0,
13              1450.0, 1500.0, 1550.0)
14 DEV_SEEDS = range(1000000, 1000150)
15 HELD_OUT_SEEDS = range(2300000, 2300500)
16
17
18 def summarize(rows):
19     values = sorted(row["average_time_s"] for row in rows)
20     return {
21         "cases": len(rows),
22         "all_cleared_cases": sum(row["fraction_cleared"] == 1 for row in rows),
23         "mean_average_time_s": statistics.mean(values),
24         "median_average_time_s": statistics.median(values),
25         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
26         "mean_distance_m": statistics.mean(row["distance_m"] for row in rows),
27         "mean_measures": statistics.mean(row["measures"] for row in rows),
28     }
29
30
31 def run_radius(radius, seeds):
32     cover = ring_cover_radius(radius)
33     if cover > 1000.0:
34         raise ValueError(f"unsafe candidate radius {radius}: cover={cover}")
35     rows = []
36     for seed in seeds:
37         row = run_case(seed, 3, "enhanced", ring_radius=radius,
38                       anchor_policy="nearest")
39         row.pop("real_duration_s") # Exclude wall-clock noise from geometry comparison.
40         rows.append(row)
41     return rows
42
43
44 def paired_summary(control, candidate):
45     assert [row["seed"] for row in control] == [row["seed"] for row in candidate]
46     deltas = [a["average_time_s"] - b["average_time_s"]
47              for a, b in zip(control, candidate, strict=True)]
48     mean = statistics.mean(deltas)
49     standard_error = statistics.stdev(deltas) / math.sqrt(len(deltas))

```

```

50     return {
51         "cases": len(deltas),
52         "mean_saved_s_per_source": mean,
53         "approx_95pct_ci_s": [mean - 1.96 * standard_error,
54                               mean + 1.96 * standard_error],
55         "faster_candidate_cases": sum(delta > 0 for delta in deltas),
56         "all_control_cleared_cases": sum(row["fraction_cleared"] == 1 for row in control),
57         "all_candidate_cleared_cases": sum(row["fraction_cleared"] == 1 for row in candidate),
58         "deltas_s_per_source": deltas,
59     }
60
61
62 def main():
63     development = {}
64     for radius in CANDIDATES:
65         rows = run_radius(radius, DEV_SEEDS)
66         development[str(radius)] = {
67             "ring_radius_m": radius,
68             "exact_cover_radius_m": ring_cover_radius(radius),
69             "summary": summarize(rows),
70             "runs": rows,
71         }
72     ordered = sorted(development,
73                      key=lambda key: development[key]["summary"]["mean_average_time_s"])
74     winner, runner_up = ordered[0], ordered[1]
75     held_out = {}
76     held_out_rows = {}
77     for radius in sorted({1200.0, float(winner), float(runner_up)}):
78         rows = run_radius(radius, HELD_OUT_SEEDS)
79         held_out_rows[str(radius)] = rows
80         held_out[str(radius)] = {
81             "ring_radius_m": radius,
82             "exact_cover_radius_m": ring_cover_radius(radius),
83             "summary": summarize(rows),
84             "runs": rows,
85         }
86     control = "1200.0" if winner != "1200.0" else runner_up
87     comparison = paired_summary(held_out_rows[control], held_out_rows[winner])
88     comparison.update({"control_ring_radius_m": float(control),
89                      "candidate_ring_radius_m": float(winner),
90                      "control_role": "incumbent" if control == "1200.0" else "development_runner_up"})
91     result = {
92         "evidence": "held_out_local_synthetic_not_official",
93         "selection": "Eight radii compared on existing development seeds; fresh confirmation seeds replace
exploratory reuse of 1100000..1100499. No retuning after confirmation.",
94         "total_runs": len(CANDIDATES) * len(DEV_SEEDS) + len(held_out_rows) * len(HELD_OUT_SEEDS),
95         "candidate_radii_m": list(CANDIDATES),
96         "development_seed_range": [DEV_SEEDS.start, DEV_SEEDS.stop - 1],
97         "held_out_seed_range": [HELD_OUT_SEEDS.start, HELD_OUT_SEEDS.stop - 1],
98         "development": development,
99         "development_winner_m": float(winner),
100        "development_runner_up_m": float(runner_up),
101        "held_out": held_out,
102        "held_out_comparison": comparison,
103    }
104    Path("results/ring_sweep.json").write_text(
105        json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
106    )
107    print(json.dumps({
108        "total_runs": result["total_runs"],
109        "development_winner_m": float(winner),
110        "development_runner_up_m": float(runner_up),
111        "held_out_seed_range": result["held_out_seed_range"],
112        "held_out": {key: value["summary"] for key, value in held_out.items()},
113        "comparison": {key: value for key, value in comparison.items()
114                       if key != "deltas_s_per_source"},
115    }, indent=2))

```

```

116
117
118 if __name__ == "__main__":
119     main()

```

Listing 25: 增强几何及区间评分交叉验证

```

1 """Independent checks for ring contraction, 2-opt and interval probe bounds."""
2 from __future__ import annotations
3
4 import itertools
5 import json
6 import math
7 import random
8 from pathlib import Path
9
10 from active_policy_candidate import score_probes
11 from coverage import directional_waypoints, omni_waypoints, ring_cover_radius
12 from geometry import clip_bearing, enclosing_circle, initial_polygon
13 from route_policy_candidate import optimize_waypoints
14
15
16 def length(points, order, start):
17     sequence = [start] + [points[i] for i in order]
18     return sum(math.dist(a, b) for a, b in itertools.pairwise(sequence))
19
20
21 def main():
22     rng = random.Random(417072)
23     route_cases, interval_cases = 0, 0
24     for size in (0,1,2,7,31):
25         for _ in range(10):
26             points = [(rng.uniform(-2500,2500), rng.uniform(-2500,2500)) for _ in range(size)]
27             start = (rng.uniform(-1000,1000),rng.uniform(-1000,1000))
28             route = optimize_waypoints(points,start)
29             assert sorted(route) == list(range(size))
30             current, pending, greedy = start, set(range(size)), []
31             while pending:
32                 i = min(pending,key=lambda i:(math.dist(current,points[i]),i))
33                 greedy.append(i); pending.remove(i); current = points[i]
34                 assert length(points,route,start) <= length(points,greedy,start)+1e-7
35                 route_cases += 1
36             ring = omni_waypoints(1200.)
37             rho = ring_cover_radius(1200.)
38             for _ in range(2000):
39                 r, phi = 1800*math.sqrt(rng.random()), rng.uniform(0,math.tau)
40                 source = (r*math.cos(phi), r*math.sin(phi))
41                 assert min(math.dist(source,p) for p in ring) <= rho + 1e-7
42             attained = min(math.dist((1800*math.cos(math.pi/6),1800*math.sin(math.pi/6)),p) for p in ring)
43             assert abs(attained-rho)<1e-7 and rho<1000
44             ring_tours = []
45             for d in (1200.0, 1550.0, 900.0 * math.sqrt(3.0)):
46                 centers = omni_waypoints(d)[1:]
47                 minimum = min(length(centers, order, (0.0, 0.0))
48                             for order in itertools.permutations(range(6)))
49                 assert abs(minimum - 6 * d) < 1e-7
50                 ring_tours.append({"radius_m": d, "permutations": 720,
51                                 "minimum_length_m": minimum, "analytic_length_m": 6 * d})
52             vertex_cases = 0
53             for spacing in (900.0, 950.0, 990.0, 999.0):
54                 vertices = directional_waypoints(spacing)
55                 for source in vertices:
56                     if math.hypot(*source) > 1800.0 + 1e-8:
57                         continue
58                     for k in range(6):
59                         neighbor = (source[0] + spacing * math.cos(k * math.pi / 3),
60                                   source[1] + spacing * math.sin(k * math.pi / 3))

```

```

61         assert min(math.dist(neighbor, v) for v in vertices) < 1e-7
62         vertex_cases += 1
63     max_violation = -float('inf')
64     for bearing in (0., 89., 179., 359.99):
65         poly = clip_bearing(initial_polygon(), (0., 0.), bearing)
66         table = score_probes(poly, [(0., 0.), bearing], (0., 0.), 3, include_lens=False)
67         for row in table:
68             if not row['eligible']:
69                 continue
70             for angle in range(0, 360, 2):
71                 post = clip_bearing(poly, row['point'], angle)
72                 if not post:
73                     continue
74                 violation = enclosing_circle(post)[1] - row['positive_bearing_radius_upper_m']
75                 max_violation = max(max_violation, violation)
76                 assert violation <= 1e-6
77                 interval_cases += 1
78     result = {'evidence': 'numerical_sanity_plus_analytic_geometry_not_official',
79              'route_cases': route_cases, 'ring_samples': 2000,
80              'ring_radius_m': 1200., 'exact_ring_cover_radius_m': rho,
81              'attained_boundary_radius_m': attained, 'radio_margin_m': 1000 - rho,
82              'ring_tour_exhaustive': ring_tours,
83              'source_vertex_six_neighbor_cases': vertex_cases,
84              'candidate_posterior_cases': interval_cases, 'max_interval_bound_violation_m': max_violation}
85     Path('results/refinement_verification.json').write_text(json.dumps(result, indent=2) + '\n')
86     print(json.dumps(result, indent=2))
87
88
89 if __name__ == '__main__':
90     main()

```

Listing 26: 31 格点不可删反例验证

```

1  """Generate exact, auditable coverage-limit witnesses (Python 3.14).
2
3  No simulator is used: every assertion is a direct Euclidean calculation on the
4  31 points returned by ``coverage.directional_waypoints``. The JSON file is a
5  reproducible calculation artifact, not a hand-entered table.
6  """
7  from __future__ import annotations
8
9  import json
10 import math
11 import sys
12 from pathlib import Path
13
14 ROOT = Path(__file__).resolve().parent
15 sys.path.insert(0, str(ROOT))
16 import coverage
17
18 R = coverage.ARENA_RADIUS
19 Q = coverage.MIN_RADIO_RADIUS
20 S = coverage.DEFAULT_SPACING
21 EPS_CANDIDATES = (1.0,)
22
23 # Six nearest-neighbour directions in axial (i,j) coordinates.
24 DIRS = ((1, 0), (0, 1), (-1, 1), (-1, 0), (0, -1), (1, -1))
25
26 def xy(ij: tuple[int, int]) -> tuple[float, float]:
27     i, j = ij
28     return (S * (i + j / 2.0), S * math.sqrt(3.0) * j / 2.0)
29
30 def norm(p: tuple[float, float]) -> float:
31     return math.hypot(*p)
32
33 def sub(a: tuple[float, float], b: tuple[float, float]) -> tuple[float, float]:
34     return (a[0] - b[0], a[1] - b[1])

```

```

35
36 def dot(a: tuple[float, float], b: tuple[float, float]) -> float:
37     return a[0] * b[0] + a[1] * b[1]
38
39 def add(a: tuple[float, float], b: tuple[float, float]) -> tuple[float, float]:
40     return (a[0] + b[0], a[1] + b[1])
41
42 def scale(c: float, a: tuple[float, float]) -> tuple[float, float]:
43     return (c * a[0], c * a[1])
44
45
46 def actual_lattice() -> dict[tuple[int, int], tuple[float, float]]:
47     pts = coverage.directional_waypoints(S)
48     axial: dict[tuple[int, int], tuple[float, float]] = {}
49     for p in pts:
50         # Recover integer coordinates and verify the reconstruction, rather than
51         # silently substituting a separately hard-coded 31-point set.
52         j = round(p[1] / (S * math.sqrt(3.0) / 2.0))
53         i = round(p[0] / S - j / 2.0)
54         ij = (i, j)
55         assert math.dist(p, xy(ij)) < 1e-8
56         axial[ij] = p
57     assert len(axial) == 31
58     return axial
59
60 def witness_for(vij: tuple[int, int], points: dict[tuple[int, int], tuple[float, float]]) -> dict:
61     v = points[vij]
62     candidates = []
63     for k in range(6):
64         d1, d2 = DIRS[k], DIRS[(k + 1) % 6]
65         uij = (vij[0] + d1[0], vij[1] + d1[1])
66         wij = (vij[0] + d2[0], vij[1] + d2[1])
67         if uij not in points or wij not in points:
68             continue
69         m = scale(0.5, add(points[uij], points[wij]))
70         nvec = sub(v, m)
71         h = norm(nvec)
72         n = scale(1.0 / h, nvec)
73         for eps in EPS_CANDIDATES:
74             g = add(m, scale(eps, n))
75             if norm(g) >= R:
76                 continue
77             rows = []
78             for ij, p in points.items():
79                 distance = norm(sub(p, g))
80                 forward = dot(sub(p, g), n)
81                 rows.append((ij, distance, forward))
82             reachable = [row for row in rows if row[1] <= Q + 1e-8 and row[2] >= -1e-8]
83             if len(reachable) == 1 and reachable[0][0] == vij:
84                 candidates.append((eps, m, n, g, rows, uij, wij))
85             break
86     assert candidates, f"no elementary witness for {vij}"
87     eps, m, n, g, rows, uij, wij = min(candidates, key=lambda z: (z[0], z[1]))
88     target_distance = norm(sub(v, g))
89     second = min(norm(sub(p, g)) for ij, p in points.items() if ij != vij)
90     reachable = sorted((dist, ij) for ij, dist, fw in rows if dist <= Q + 1e-8 and fw >= -1e-8)
91     assert reachable[0][1] == vij and len(reachable) == 1
92     return {"solevertex": list(vij), "target": list(vij), "edge_neighbors": [list(uij), list(wij)],
93           "midpoint": list(m), "epsilon": eps, "source": list(g), "n": list(n),
94           "normal": list(n), "source_radius": norm(g),
95           "target_distance": target_distance, "second_detector_distance": second,
96           "reachable": [[list(ij), dist] for dist, ij in reachable],
97           "min_other_forward": min(fw for ij, dist, fw in rows if ij != vij),
98           "midpoint_to_nearest_other_positive": min(norm(sub(p, m)) for ij, p in points.items()
99               if ij not in (uij, wij, vij) and dot(sub(p, m), n) > 1e
100               -8)}

```

```

101 def main() -> None:
102     points = actual_lattice()
103     witnesses = [witness_for(ij, points) for ij in sorted(points)]
104     for w in witnesses:
105         assert w["source_radius"] < R and w["target_distance"] <= Q
106         assert len(w["reachable"]) == 1
107         assert w["second_detector_distance"] > 5.0
108     optimal_d = math.hypot(R, Q)
109     half_angle = math.atan(Q / R)
110     attained_angle = math.acos(R / optimal_d)
111     assert abs(attained_angle - half_angle) < 1e-12
112     for i in range(1001):
113         d = R + i * Q / 1000
114         cos_limit = max(R / d, (d*d + R*R - Q*Q) / (2*d*R))
115         assert math.acos(min(1.0, cos_limit)) <= half_angle + 1e-12
116     result = {"parameters": {"arena_radius": R, "receive_radius": Q,
117                             "spacing": S, "point_count": len(points)},
118              "q3_boundary_angle_lower_bound": {"q": 900.0, "minimum_points": 7,
119                                                  "half_angle_radians": math.asin(900.0 / R),
120                                                  "reason": "six arcs have total angular capacity 2*pi only at equality; equality ring misses
121              origin"},
122              "q1000_static_seven_irreducible": {"minimum_fixed_subset": 7,
123                                                  "boundary_arc_capacity_of_five": 10.0 * math.asin(Q / R),
124                                                  "origin_to_ring": 900.0 * math.sqrt(3.0)},
125              "q4_global_outer_boundary_lower_bound": {"minimum_outer_detectors": 7,
126                                                        "half_angle_radians": math.atan(Q / R),
127                                                        "optimal_detector_radius": optimal_d,
128                                                        "attained_half_angle_radians": attained_angle,
129                                                        "checked_detector_radii": 1001,
130                                                        "statement": "lower bound for arbitrary placements at an outward boundary source; not a 31-
131              point global optimum claim"},
132              "witnesses": witnesses}
133     out = ROOT / "results" / "coverage_limits.json"
134     out.parent.mkdir(exist_ok=True)
135     out.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n", encoding="utf-8")
136
137 if __name__ == "__main__":
138     main()

```

Listing 27: 增强策略图表生成

```

1 """Generate enhancement-only paper assets from saved local experiment evidence.
2
3 Run from any directory with conda run -n py314 python <path>/make_enhancement_assets.py.
4 No planner, simulator, hidden world, or source file is loaded or executed.
5 """
6 from __future__ import annotations
7
8 import hashlib
9 import json
10 import math
11 import os
12 import statistics
13 import tempfile
14 from collections import defaultdict
15 from io import BytesIO
16 from pathlib import Path
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 from matplotlib.lines import Line2D
23 from matplotlib.patches import Circle
24
25 ROOT = Path(__file__).resolve().parent
26 GENERATED = ROOT / "report" / "generated"

```

```

27 FIGURES = ROOT / "report" / "figures"
28 WIDTH = 16 / 2.54
29 POLICIES = ("integrated", "enhanced")
30 SCENARIOS = ("minimum_radius", "outward_boundary", "clustered")
31 MODES = ("smooth", "extreme", "iid_location")
32 COLORS = ("0072B2", "D55E00")
33
34
35 def require(condition, message):
36     if not condition:
37         raise ValueError(message)
38
39
40 def close(actual, expected, name, tolerance=1e-8):
41     require(math.isfinite(actual) and math.isfinite(expected), f"{name}: nonfinite value")
42     require(math.isclose(actual, expected, rel_tol=1e-10, abs_tol=tolerance),
43             f"{name}: {actual!r} != {expected!r}")
44
45
46 def number(value):
47     require(math.isfinite(value), "Nonfinite table value")
48     return f"{value:.2f}"
49
50
51 def load(name):
52     return json.loads((ROOT / "results" / name).read_text(encoding="utf-8"))
53
54
55 def atomic_bytes(path, content):
56     require(path.parent in (GENERATED, FIGURES), "Output outside owned directories")
57     path.parent.mkdir(parents=True, exist_ok=True)
58     fd, temporary = tempfile.mkstemp(prefix="." + path.name + ".", dir=path.parent)
59     try:
60         with os.fdopen(fd, "wb") as stream:
61             stream.write(content)
62         os.replace(temporary, path)
63     finally:
64         Path(temporary).unlink(missing_ok=True)
65
66
67 def table(name, headers, rows, widths, left_columns=(0, 1)):
68     """Fixed-width booktabs tabular only; the parent paper owns all captions."""
69     require(len(headers) == len(widths), "Invalid table widths")
70     require(sum(widths) + 0.14 * (len(widths) - 1) <= 15.9, "Table exceeds 16 cm")
71     columns = []
72     for index, width in enumerate(widths):
73         align = r"\raggedright" if index in left_columns else r"\raggedleft"
74         columns.append(">{" + align + r"\arraybackslash}p{" + str(width) + "cm}")
75     specification = "@{" + r"@{\hspace{0.14cm}}".join(columns) + "@{"
76     lines = [r"\begin{tabular}{" + specification + "}", r"\toprule",
77             " & ".join(headers) + r" \\", r"\midrule"]
78     for row in rows:
79         require(len(row) == len(headers), "Ragged table")
80         lines.append(" & ".join(str(item) for item in row) + r" \\")
81     lines.extend((r"\bottomrule", r"\end{tabular}"))
82     atomic_bytes(GENERATED / name, ("\n".join(lines) + "\n").encode("utf-8"))
83
84
85 def save_figure(fig, name):
86     buffer = BytesIO()
87     fig.savefig(buffer, format="pdf", metadata={"CreationDate": None, "ModDate": None,
88                                             "Creator": "make_enhancement_assets.py"})
89     atomic_bytes(FIGURES / name, buffer.getvalue())
90     plt.close(fig)
91
92
93 def check_run(row):

```

```

94     for key in ("n_sources", "n_cleared", "measures", "switches", "clear_attempts", "failed_clears"):
95         require(type(row[key]) is int and row[key] >= 0, f"Invalid {key}")
96     n = row["n_sources"]
97     require(n > 0 and row["n_cleared"] <= n, "Invalid per-world denominator")
98     require(row["clear_attempts"] == row["n_cleared"] + row["failed_clears"],
99             "Clear-attempt decomposition mismatch")
100    require(row["distance_m"] >= 0, "Negative distance")
101    close(row["fraction_cleared"], row["n_cleared"] / n, "Clear fraction")
102    close(row["average_time_s"], row["virtual_time_s"] / n, "Per-world average")
103    total = (row["distance_m"] / 5 + 5 * row["measures"] + row["switches"]
104            + 3 * row["failed_clears"] + 5 * row["n_cleared"])
105    # Each accepted measurement/clear rounds its movement charge to microseconds.
106    rounding_bound = 0.5e-6 * (row["measures"] + row["clear_attempts"]) + 1e-8
107    close(total, row["virtual_time_s"], "Independent cost identity", tolerance=rounding_bound)
108
109
110    def metrics(rows):
111        require(bool(rows), "Empty group")
112        values = sorted(row["average_time_s"] for row in rows)
113        return {"cases": len(rows),
114                "all_cleared_cases": sum(r["n_cleared"] == r["n_sources"] for r in rows),
115                "mean_average_time_s": statistics.mean(values),
116                "median_average_time_s": statistics.median(values),
117                "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1]}
118
119
120    def check_summary(actual, summary, name):
121        for key, value in actual.items():
122            close(value, summary[key], f"{name}/{key}")
123
124
125    def paired(control, enhanced):
126        require([r["seed"] for r in control] == [r["seed"] for r in enhanced], "Unpaired seeds")
127        for a, b in zip(control, enhanced):
128            require(all(a[key] == b[key] for key in
129                      ("problem", "scenario", "error_mode", "n_sources")), "Unpaired world metadata")
130        delta = [a["average_time_s"] - b["average_time_s"] for a, b in zip(control, enhanced)]
131        mean = statistics.mean(delta)
132        return delta, {"cases": len(delta), "mean_saved_s_per_source": mean,
133                      "mean_reduction_pct": 100 * mean / metrics(control)["mean_average_time_s"],
134                      "faster_cases": sum(value > 0 for value in delta)}
135
136
137    def load_enhancement():
138        data = load("enhancement_experiments.json")
139        require(data["main_seed_range"] == [1100000, 1100499], "Unexpected held-out seeds")
140        require(data["stress_seed_range"] == [1110000, 1110029], "Unexpected pressure seeds")
141        groups = defaultdict(list)
142        for row in data["runs"]:
143            check_run(row)
144            require(row["strategy"] in POLICIES, "Unexpected enhancement policy")
145            groups[(row["problem"], row["scenario"], row["error_mode"], row["strategy"])].append(row)
146        expected = {(q, scenario, mode, policy) for q in (3, 4) for policy in POLICIES
147                   for scenario, mode in [("random", "smooth")] +
148                   [(s, m) for s in SCENARIOS for m in MODES]}
149        require(set(groups) == expected, "Missing or extra enhancement groups")
150        for key, rows in groups.items():
151            rows.sort(key=lambda r: r["seed"])
152            start, end = data["main_seed_range"] if key[1] == "random" else data["stress_seed_range"]
153            require([r["seed"] for r in rows] == list(range(start, end + 1)),
154                   f"{key}: incomplete or duplicate seeds")
155        require(len(data["runs"]) == data["total_runs"] == 3080, "Run total mismatch")
156        require(set(data["summary"]) == {f"Q{q}_{p}" for q in (3, 4) for p in POLICIES},
157               "Unexpected main summary groups")
158        require(set(data["comparison"]) == {"Q3", "Q4"}, "Unexpected main comparisons")
159        require(set(data["stress"]) == {f"Q{q}_{s}_{m}" for q in (3, 4)
160                                       for s in SCENARIOS for m in MODES}, "Expected 18 pressure groups")

```

```

161     for q in (3, 4):
162         controls = groups[q, "random", "smooth", "integrated"]
163         enhancements = groups[q, "random", "smooth", "enhanced"]
164         for policy in POLICIES:
165             check_summary(metrics(groups[q, "random", "smooth", policy]),
166                           data["summary"][f"Q{q}_{policy}"], f"Q{q}/{policy}")
167             check_summary(paired(controls, enhancements)[1], data["comparison"][f"Q{q}"], f"Q{q}/paired")
168         for scenario in SCENARIOS:
169             for mode in MODES:
170                 key = f"Q{q}_{scenario}_{mode}"
171                 for policy in POLICIES:
172                     check_summary(metrics(groups[q, scenario, mode, policy]),
173                                   data["stress"][key][policy], key + "/" + policy)
174                     check_summary(paired(*groups[q, scenario, mode, p] for p in POLICIES)[1],
175                                   data["stress"][key]["paired"], key + "/paired")
176     return data, groups
177
178
179 def enhancement_tables(data, groups):
180     performance_rows, cost_rows, macros = [], [], []
181     for q, word in ((3, "Three"), (4, "Four")):
182         control, enhanced = (groups[q, "random", "smooth", p] for p in POLICIES)
183         for policy, rows in zip(POLICIES, (control, enhanced)):
184             stats = metrics(rows)
185             label = "原集成" if policy == "integrated" else "增强"
186             performance_rows.append((f"Q{q}", label,
187                                     number(stats["mean_average_time_s"]),
188                                     number(stats["median_average_time_s"]),
189                                     number(stats["p90_average_time_s"]),
190                                     f'{stats["all_cleared_cases"]}/{stats["cases"]}')
191                                     # Average each world's cost/N; never divide pooled costs by pooled N.
192                                     components = [statistics.mean(value(r) / r["n_sources"] for r in rows)
193                                                     for value in (lambda r: r["distance_m"] / 5,
194                                                                lambda r: 5 * r["measures"], lambda r: r["switches"],
195                                                                lambda r: 3 * r["failed_clears"], lambda r: 5 * r["n_cleared"])]
196                                     rounding_bound = statistics.mean(
197                                         (0.5e-6 * (r["measures"] + r["clear_attempts"]) + 1e-8) / r["n_sources"]
198                                         for r in rows)
199                                     close(sum(components), stats["mean_average_time_s"], "Mean cost identity", tolerance=rounding_bound)
200             )
201             cost_rows.append((f"Q{q}", label, *(number(v) for v in components),
202                               number(stats["mean_average_time_s"])))
203             comparison = paired(control, enhanced)[1]
204             macros.extend(((f"EnhancedQ{word}Time", number(metrics(enhanced)["mean_average_time_s"])),
205                             (f"EnhancedQ{word}Gain", number(comparison["mean_reduction_pct"]))))
206         macros.extend(((("EnhancementRuns", str(len(data["runs"]))),
207                         ("ContractedRadius", number(data["contracted_ring"]["radius_m"]))))
208         atomic_bytes(GENERATED / "enhancement_numbers.tex",
209                      ("% Gains are percentage reductions of paired-group mean T/N.\n" +
210                       "\n".join("\newcommand{\%} + key + " + value + "}" for key, value in macros)
211                       + "\n").encode("utf-8"))
212     table("enhancement_table.tex", ("问题", "策略", "均值/s", "中位数/s", "$P_{90}$/s", "全清/世界"),
213          performance_rows, (0.9, 1.7, 2.6, 2.6, 2.6, 2.6))
214     table("enhancement_costs.tex", ("问题", "策略", "$L/(5N)$", "$5M/N$", "$S/N$", "$3F/N$", "$5K/N$", "合计/s"),
215          cost_rows, (0.9, 1.5, 2.1, 1.8, 1.7, 1.7, 1.7, 2.1))
216     scenario_labels = {"minimum_radius": "最小半径", "outward_boundary": "边界朝外", "clustered": "聚集"}
217     mode_labels = {"smooth": "平滑", "extreme": "极端", "iid_location": "位置独立"}
218     pressure_rows = []
219     for q in (3, 4):
220         for scenario in SCENARIOS:
221             for mode in MODES:
222                 control, enhanced = (groups[q, scenario, mode, p] for p in POLICIES)
223                 comparison = paired(control, enhanced)[1]
224                 both_clear = sum(a["n_cleared"] == a["n_sources"] and b["n_cleared"] == b["n_sources"]
225                                for a, b in zip(control, enhanced))
226                 pressure_rows.append((f"Q{q}", scenario_labels[scenario], mode_labels[mode],

```

```

226         number(metrics(control)["mean_average_time_s"]),
227         number(metrics(enhanced)["mean_average_time_s"]),
228         number(comparison["mean_saved_s_per_source"]),
229         f'{comparison["faster_cases"]}/{len(control)}',
230         f'{both_clear}/{len(control)}'))
231 table("enhancement_stress.tex", ("问题", "场景", "误差", "原集成/s", "增强/s", "节省/s", "更快/对", "双全清/
    对"),
232     pressure_rows, (0.8, 1.7, 1.7, 2.1, 2.1, 2.1, 1.6, 2.0), left_columns=(0, 1, 2))
233
234
235 def candidate_table():
236     static = load("route_candidate_comparison.json")
237     adaptive = load("adaptive_candidate_comparison.json")
238     require(set(static["summary"]) == set(adaptive["summary"])) == {"Q3", "Q4"},
239         "Unexpected candidate problems")
240     static_start, static_end = static["seed_range"]
241     adaptive_start, adaptive_end = adaptive["seed_range"]
242     require(type(static_start) is int and type(static_end) is int and static_end >= static_start,
243         "Invalid static seed range")
244     require(type(adaptive_start) is int and type(adaptive_end) is int and adaptive_end >= adaptive_start,
245         "Invalid adaptive seed range")
246     static_count, adaptive_count = static_end - static_start + 1, adaptive_end - adaptive_start + 1
247     static_groups = defaultdict(list)
248     for pair in static.get("runs", []):
249         require(pair["problem"] in (3, 4), "Unexpected static problem")
250         nearest, two_opt = pair["nearest"], pair["two_opt"]
251         check_run(nearest)
252         check_run(two_opt)
253         require(nearest["seed"] == two_opt["seed"] == pair["seed"],
254             and nearest["problem"] == two_opt["problem"] == pair["problem"],
255             "Static pair metadata mismatch")
256         static_groups[pair["problem"]].append(pair)
257     adaptive_groups = defaultdict(list)
258     for pair in adaptive.get("runs", []):
259         require(pair["problem"] in (3, 4), "Unexpected adaptive problem")
260         for policy in ("fixed", "adaptive"):
261             row = pair[policy]
262             check_run(row)
263             require(row["seed"] == pair["seed"] and row["problem"] == pair["problem"],
264                 "Adaptive pair metadata mismatch")
265             adaptive_groups[pair["problem"]].append(pair)
266     rows = []
267     for q in (3, 4):
268         pairs = sorted(static_groups[q], key=lambda p: p["seed"])
269         require([p["seed"] for p in pairs] == list(range(static_start, static_end + 1)),
270             "Static seed mismatch")
271         nearest = [p["nearest"] for p in pairs]
272         two_opt = [p["two_opt"] for p in pairs]
273         deltas = [a["average_time_s"] - b["average_time_s"]
274             for a, b in zip(nearest, two_opt)]
275         s = {"cases": static_count,
276             "all_nearest_cleared": sum(a["fraction_cleared"] == 1 for a in nearest),
277             "all_two_opt_cleared": sum(b["fraction_cleared"] == 1 for b in two_opt),
278             "nearest_mean_s_per_source": statistics.mean(a["average_time_s"] for a in nearest),
279             "two_opt_mean_s_per_source": statistics.mean(b["average_time_s"] for b in two_opt),
280             "mean_saved_s_per_source": statistics.mean(deltas),
281             "faster_cases": sum(d > 0 for d in deltas)}
282         for key, value in s.items():
283             if key in static["summary"][f"Q{q}"]:
284                 close(value, static["summary"][f"Q{q}"][key], f"Static Q{q}/{key}")
285         close(s["nearest_mean_s_per_source"] - s["two_opt_mean_s_per_source"],
286             s["mean_saved_s_per_source"], "Static mean difference")
287         rows.append((f"Q{q}", "静态2-opt", number(s["nearest_mean_s_per_source"]),
288             number(s["two_opt_mean_s_per_source"]), number(s["mean_saved_s_per_source"]),
289             f'{s["faster_cases"]}/{static_count}',
290             f'{s["all_nearest_cleared"]}/{s["all_two_opt_cleared"]}'))
291     pairs = sorted(adaptive_groups[q], key=lambda p: p["seed"])

```

```

292     require([p["seed"] for p in pairs] == list(range(adaptive_start, adaptive_end + 1)),
293             "Adaptive seed mismatch")
294     fixed, adapted = [p["fixed"] for p in pairs], [p["adaptive"] for p in pairs]
295     fixed_metrics, adapted_metrics = metrics(fixed), metrics(adapted)
296     delta, comparison = paired(fixed, adapted)
297     actual = {"cases": len(pairs), "fixed_all_cleared": fixed_metrics["all_cleared_cases"],
298             "adaptive_all_cleared": adapted_metrics["all_cleared_cases"],
299             "fixed_mean_s_per_source": fixed_metrics["mean_average_time_s"],
300             "adaptive_mean_s_per_source": adapted_metrics["mean_average_time_s"],
301             "mean_saved_s_per_source": comparison["mean_saved_s_per_source"],
302             "median_saved_s_per_source": statistics.median(delta),
303             "faster_cases": comparison["faster_cases"]}
304     check_summary(actual, adaptive["summary"][f"Q{q}"], f"Adaptive Q{q}")
305     rows.append((f"Q{q}", "自适应探测", number(actual["fixed_mean_s_per_source"]),
306             number(actual["adaptive_mean_s_per_source"]), number(actual["mean_saved_s_per_source"]),
307             f'{actual["faster_cases"]}/{adaptive_count}',
308             f'{actual["fixed_all_cleared"]}/{actual["adaptive_all_cleared"]}'))
309     table("candidate_table.tex", ("问题", "候选", "对照/s", "候选/s", "节省/s", "更快/对", "全清数\\newline 对照/
    候选"),
310         rows, (0.8, 2.4, 2.1, 2.1, 2.1, 1.9, 2.6))
311
312
313 def coverage_limits_table():
314     certificate = load("coverage_limits.json")
315     witnesses = certificate["witnesses"]
316     require(len(witnesses) == certificate["parameters"]["point_count"] == 31,
317             "Witness count mismatch")
318     target_distances = [row["target_distance"] for row in witnesses]
319     other_distances = [row["second_detector_distance"] for row in witnesses]
320     rows = [
321         ("Q3 q=900 lower bound", "7", "six boundary arcs at equality; origin uncovered"),
322         ("Q3 fixed seven-set at q=1000", "7", "five ring arcs leave a boundary gap"),
323         ("Q4 outer-boundary lower bound", "7", "arbitrary placements; outer detectors only"),
324         ("Q4 current lattice witnesses", str(len(witnesses))),
325         f"target d={min(target_distances):.3f}--{max(target_distances):.3f} m",
326         ("Q4 nearest other lattice point", f"{min(other_distances):.3f} m",
327         "every witness remains outside near threshold"),
328     ]
329     table("coverage_limits_table.tex", ("Statement", "Value", "Interpretation"), rows,
330         (4.1, 2.0, 8.7), left_columns=(0, 2))
331
332
333 def ring_sweep_table():
334     data = load("ring_sweep.json")
335     require(data["development_seed_range"] == [1000000, 1000149], "Ring development seeds")
336     require(data["held_out_seed_range"] == [2300000, 2300499], "Ring fresh confirmation seeds")
337     candidates = data["candidate_radii_m"]
338     require(candidates == [1200., 1250., 1300., 1350., 1400., 1450., 1500., 1550.],
339             "Unexpected frozen ring candidates")
340     count = 0
341     for phase, seed_key in (("development", "development_seed_range"),
342                             ("held_out", "held_out_seed_range")):
343         start, end = data[seed_key]
344         for key, group in data[phase].items():
345             runs = group["runs"]
346             require([r["seed"] for r in runs] == list(range(start, end + 1)), "Ring paired seeds")
347             count += len(runs)
348             radius = float(key)
349             envelope = max(radius / math.sqrt(3),
350                             math.sqrt(1800**2 + radius**2 - math.sqrt(3) * 1800 * radius))
351             close(envelope, group["exact_cover_radius_m"], "Ring envelope")
352             require(envelope < 1000, "Unsafe ring candidate")
353             for row in runs:
354                 check_run(row)
355                 require(row["problem"] == 3 and row["strategy"] == "enhanced"
356                         and row["scenario"] == "random" and row["error_mode"] == "smooth",
357                         "Ring world contract")

```

```

358         close(row["ring_radius_m"], radius, "Ring run geometry")
359         require(row["certificate_complete"] and row["timing_independently_verified"]
360                and row["n_cleared"] == row["n_sources"], "Incomplete ring run")
361         check_summary(metrics(runs), group["summary"], "Ring summary")
362         require(count == data["total_runs"] == 2200, "Ring run denominator")
363         ordered = sorted(data["development"], key=lambda k:
364                          data["development"][k]["summary"]["mean_average_time_s"])
365         winner, runner_up = ordered[:2]
366         close(float(winner), data["development_winner_m"], "Ring development winner")
367         close(float(runner_up), data["development_runner_up_m"], "Ring runner-up")
368         control = "1200.0" if winner != "1200.0" else runner_up
369         comparison = data["held_out_comparison"]
370         close(comparison["candidate_ring_radius_m"], float(winner), "Ring candidate")
371         close(comparison["control_ring_radius_m"], float(control), "Ring control")
372         deltas, calculated = paired(data["held_out"][control]["runs"], data["held_out"][winner]["runs"])
373         require(deltas == comparison["deltas_s_per_source"], "Ring raw paired deltas")
374         require(comparison["faster_candidate_cases"] == calculated["faster_cases"], "Ring win count")
375         close(calculated["mean_saved_s_per_source"], comparison["mean_saved_s_per_source"], "Ring mean")
376         se = statistics.stdev(deltas) / math.sqrt(len(deltas))
377         for observed, expected in zip(comparison["approx_95pct_ci_s"],
378                                     (statistics.mean(deltas) - 1.96 * se,
379                                      statistics.mean(deltas) + 1.96 * se), strict=True):
380             close(observed, expected, "Ring paired CI")
381     rows = []
382     for radius in candidates:
383         key = str(float(radius))
384         group = data["development"][key]
385         held = data["held_out"].get(key)
386         rows.append((f"{radius:.0f}", number(group["exact_cover_radius_m"]),
387                     number(group["summary"]["mean_average_time_s"]),
388                     number(held["summary"]["mean_average_time_s"]) if held else "--",
389                     number(comparison["mean_saved_s_per_source"]) if key == winner else "--",
390                     "保留" if key == winner else ("次优" if key == runner_up else "")))
391     table("ring_sweep_table.tex", ("环半径/m", "解析覆盖/m", "开发T/N", "新样本T/N", "相对对照节省/s", "选择"),
392          rows, (1.5, 2.2, 2.0, 2.1, 2.6, 1.0), left_columns=(5,))
393     macros = {
394         "RingSelectedTime": number(data["held_out"][winner]["summary"]["mean_average_time_s"]),
395         "RingControlTime": number(data["held_out"][control]["summary"]["mean_average_time_s"]),
396         "RingSaving": f'f{{comparison["mean_saved_s_per_source"]:.3f}}',
397         "RingLowerCI": f'f{{comparison["approx_95pct_ci_s"][0]:.3f}}',
398         "RingUpperCI": f'f{{comparison["approx_95pct_ci_s"][1]:.3f}}',
399         "RingFaster": str(comparison["faster_candidate_cases"]),
400     }
401     atomic_bytes(GENERATED / "ring_numbers.tex", "".join(
402         "\\newcommand{" + key + "}" + value + "\\n" for key, value in macros.items()).encode())
403
404
405 def geometry_figure(data, groups):
406     certificate = load("coverage_limits.json")
407     arena = certificate["parameters"]["arena_radius"]
408     radio = certificate["parameters"]["receive_radius"]
409     legacy = certificate["q1000_static_seven_irreducible"]["origin_to_ring"]
410     contracted = data["contracted_ring"]
411     radius = contracted["radius_m"]
412     for policy, expected in (("integrated", legacy), ("enhanced", radius)):
413         for row in groups[3, "random", "smooth", policy]:
414             close(row["ring_radius_m"], expected, "Ring geometry/run mismatch")
415     def exact_cover(r):
416         return max(r / math.sqrt(3), math.sqrt(arena**2 + r**2 - math.sqrt(3) * arena * r))
417     q = exact_cover(radius)
418     close(q, contracted["exact_cover_radius_m"], "Contracted cover radius")
419     close(radio - q, contracted["radio_margin_m"], "Contracted radial distance margin")
420     fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 4.5))
421     fig.subplots_adjust(left=0.10, right=0.99, bottom=0.37, top=0.85, wspace=0.30)
422     for ax, r, color, title in zip(axes, (legacy, radius), COLORS, ("Integrated", "Contracted")):
423         centers = [(0., 0.)] + [(r * math.cos(k * math.pi / 3), r * math.sin(k * math.pi / 3))
424                             for k in range(6)]

```

```

425     for x, y in centers:
426         ax.add_patch(Circle((x / 1000, y / 1000), radio / 1000,
427                             facecolor=color, edgecolor=color, alpha=0.12, linewidth=0.8))
428         if title == "Contracted":
429             ax.add_patch(Circle((x / 1000, y / 1000), q / 1000, fill=False,
430                                 edgecolor=color, linestyle="--", linewidth=0.7))
431     ax.add_patch(Circle((0, 0), arena / 1000, fill=False, color="black", linewidth=1.1))
432     ax.scatter([x / 1000 for x, _ in centers], [y / 1000 for _, y in centers],
433               color=color, s=16, zorder=4)
434     boundary = (arena * math.cos(math.pi / 6), arena * math.sin(math.pi / 6))
435     ax.plot([r / 1000, boundary[0] / 1000], [0, boundary[1] / 1000], color="black", linewidth=1.1)
436     ax.plot(boundary[0] / 1000, boundary[1] / 1000, "kx", markersize=5)
437     ax.set(xlim=(-2.7, 2.7), ylim=(-2.7, 2.7), xlabel="x (km)", ylabel="y (km)",
438           title=f"{title}\nr = {r:.2f} m")
439     ax.set_xticks((-2, 0, 2))
440     ax.set_yticks((-2, 0, 2))
441     ax.set_aspect("equal")
442     ax.grid(alpha=0.15)
443     ax.text(0.5, -0.27, f"q = {exact_cover(r):.2f} m\nmargin = {radio-exact_cover(r):.2f} m",
444            transform=ax.transAxes, ha="center", va="top", fontsize=10)
445     handles = [Line2D([], [], color="black", label="Source arena"),
446               Line2D([], [], color=COLORS[0], alpha=0.5, label=f"Radio: {radio:.0f} m"),
447               Line2D([], [], color=COLORS[1], linestyle="--", label=f"Cover: {q:.2f} m (right)")]
448     fig.legend(handles=handles, loc="lower center", bbox_to_anchor=(0.5, 0.005),
449              ncol=1, frameon=False, fontsize=10)
450     save_figure(fig, "enhancement_geometry.pdf")
451
452
453 def ecdf(ax, values, **kwargs):
454     ordered = sorted(values)
455     require(bool(ordered), "Empty ECDF")
456     ax.step([ordered[0]] + ordered, [0.] + [(i + 1) / len(ordered) for i in range(len(ordered))],
457            where="post", **kwargs)
458
459
460 def performance_figure(groups):
461     fig, axes = plt.subplots(2, 2, figsize=(WIDTH, 5.8), sharey=True)
462     fig.subplots_adjust(left=0.11, right=0.98, top=0.93, bottom=0.14, hspace=0.56, wspace=0.34)
463     for column, q in enumerate((3, 4)):
464         control, enhanced = (groups[q, "random", "smooth", p] for p in POLICIES)
465         delta, _ = paired(control, enhanced)
466         ax = axes[0, column]
467         ecdf(ax, delta, color=COLORS[column], linewidth=1.5)
468         ax.axvline(0, color="0.4", linestyle="--", linewidth=0.8)
469         ax.set(title=f"Q{q}: paired savings", xlabel="Integrated - enhanced (s/source)")
470         ax = axes[1, column]
471         for rows, color, label in zip((control, enhanced), COLORS, ("Integrated", "Enhanced")):
472             ecdf(ax, [r["average_time_s"] for r in rows], color=color, linewidth=1.5, label=label)
473         ax.set(title=f"Q{q}: time distributions", xlabel="T/N (s/source)")
474     for ax in axes.flat:
475         ax.set_ylim(0, 1.02)
476         ax.set_yticks((0, 0.5, 1))
477         ax.grid(alpha=0.18)
478         ax.tick_params(axis="x", labelrotation=0)
479     for ax in axes[:, 0]:
480         ax.set_ylabel("Empirical probability")
481     handles, labels = axes[1, 0].get_legend_handles_labels()
482     fig.legend(handles, labels, loc="lower center", ncol=2, frameon=False, bbox_to_anchor=(0.5, 0.015))
483     save_figure(fig, "enhancement_performance.pdf")
484
485
486 def main():
487     plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
488                        "axes.titlesize": 10, "axes.labelsize": 10,
489                        "xtick.labelsize": 10, "ytick.labelsize": 10,
490                        "legend.fontsize": 10, "pdf.fonttype": 42,
491                        "savefig.transparent": False, "figure.facecolor": "white"})

```

```

492 data, groups = load_enhancement()
493 enhancement_tables(data, groups)
494 candidate_table()
495 coverage_limits_table()
496 ring_sweep_table()
497 geometry_figure(data, groups)
498 performance_figure(groups)
499 print(f"Generated enhancement assets from {len(data['runs'])} saved runs; "
500       "raw candidate pairs and ring sweep validated. No simulator executed.")
501 input_names = ("enhancement_experiments.json", "route_candidate_comparison.json",
502               "adaptive_candidate_comparison.json", "coverage_limits.json",
503               "ring_sweep.json", "refinement_verification.json", "anchor_development.json")
504 output_names = ("enhancement_numbers.tex", "enhancement_table.tex", "enhancement_costs.tex",
505                "enhancement_stress.tex", "candidate_table.tex", "coverage_limits_table.tex",
506                "ring_sweep_table.tex", "ring_numbers.tex", "enhancement_geometry.pdf", "
507                enhancement_performance.pdf")
508 metadata = {
509     "evidence": "held_out_local_synthetic_not_official",
510     "policy": {"Q3": "enhanced ring radius 1200m selected by a development-only sweep, nearest anchor",
511               "Q4": "enhanced 990m triangular lattice, rolling remaining-anchor route",
512               "probe": "fixed probe and bounded recovery; adaptive candidate is negative control"},
513     "input_sha256": {"results/" + name: hashlib.sha256((ROOT / "results" / name).read_bytes()).hexdigest()
514                      for name in input_names},
515     "outputs": list(output_names),
516     "main_seed_range": data["main_seed_range"],
517     "stress_seed_range": data["stress_seed_range"],
518     "total_runs": data["total_runs"],
519     "ring_sweep_runs": 2200,
520     "ring_confirmation_seed_range": [2300000, 2300499],
521     "metric": "per-world T/N; paired savings are control minus enhanced",
522     "official_calls": 0,
523     "reproducibility": "Generated after validating every raw run, paired seed and cost decomposition",
524 }
525 atomic_bytes(GENERATED / "enhancement_metadata.json",
526              (json.dumps(metadata, ensure_ascii=False, indent=2) + "\n").encode("utf-8"))
527
528 if __name__ == "__main__":
529     main()

```

Listing 28: 25 点极坐标覆盖候选与解析证书

```

1  """Independent Q4 polar discovery candidates; no planner/default integration.
2
3  The certificate is an analytic triangulation, not a sampled coverage result.
4  Each unresolved channel must still complete every waypoint obligation. Returned
5  coordinates approximate the ideal real construction; the certificate explicitly
6  allows a bounded Euclidean error at each commanded measurement location.
7  """
8
9  from __future__ import annotations
10
11  import math
12
13  Point = tuple[float, float]
14
15
16  def _parameters(variant: str) -> tuple[float, float, float, float, float]:
17      if variant == "seed25":
18          return 990.0, 1865.0, 1.0, 0.25, 990.0
19      if variant == "compact25":
20          return 940.0, 1870.0, 5.0, 1.0, 993.0
21      raise ValueError("variant must be 'seed25' or 'compact25'")
22
23
24  def polar_waypoints(variant: str = "seed25") -> list[Point]:
25      """Return origin, 12 inner points, then 12 outer points shifted by 15 deg.

```

```

26
27 Index 1+k has angle 30*k degrees; index 13+k has angle 30*k+15.
28 This order specifies stable obligation indices, not a recommended route.
29 """
30 inner, outer, _, _, _ = _parameters(variant)
31 points: list[Point] = [(0.0, 0.0)]
32 for radius, phase in ((inner, 0.0), (outer, math.pi / 12.0)):
33     for k in range(12):
34         angle = k * math.pi / 6.0 + phase
35         points.append((radius * math.cos(angle), radius * math.sin(angle)))
36 return points
37
38
39 def polar_certificate(variant: str = "seed25") -> dict:
40     """Return finite indices, closed-form constants, and conservative bounds.
41
42     Floats in ``analytic_values_m`` are evaluations for independent checking,
43     not interval-arithmetic proofs. The exact proof uses the rational bounds
44     9659/10000 < cos(pi/12) and 1/4 < sin(pi/12) < 259/1000.
45     The integer/rational inequalities are documented in the companion notes.
46     """
47     inner, outer, shift, error, edge_bound = _parameters(variant)
48     sine = (math.sqrt(6.0) - math.sqrt(2.0)) / 4.0
49     cosine = (math.sqrt(6.0) + math.sqrt(2.0)) / 4.0
50     inner_chord = 2.0 * inner * sine
51     outer_chord = 2.0 * outer * sine
52     cross_edge = math.sqrt(inner**2 + outer**2 - 2.0 * inner * outer * cosine)
53     triangles: list[tuple[int, int, int]] = []
54     for k in range(12):
55         i = 1 + k
56         j = 1 + (k + 1) % 12
57         o = 13 + k
58         p = 13 + (k + 1) % 12
59         triangles.extend(((0, i, j), (i, o, j), (j, o, p)))
60     edges = sorted({tuple(sorted((triangle[k], triangle[(k + 1) % 3]))
61                             for triangle in triangles for k in range(3))})
62     route = list(range(13)) + list(range(24, 12, -1))
63     return {
64         "variant": variant,
65         "evidence": "analytic finite triangulation; not an experiment",
66         "source_radius_m": 1800.0,
67         "minimum_radio_radius_m": 1000.0,
68         "point_count": 25,
69         "inside_source_disk_count": 13,
70         "outside_source_disk_count": 12,
71         "origin_index": 0,
72         "inner_indices": list(range(1, 13)),
73         "outer_indices": list(range(13, 25)),
74         "inner_radius_m": inner,
75         "outer_radius_m": outer,
76         "outer_phase_degrees": 15.0,
77         "points": polar_waypoints(variant),
78         "triangles_ccw": triangles,
79         "edges": edges,
80         "boundary_ccw": list(range(13, 25)),
81         "triangle_count": 36,
82         "edge_count": 60,
83         "closed_form": {
84             "cos_15": "(sqrt(6)+sqrt(2))/4",
85             "sin_15": "(sqrt(6)-sqrt(2))/4",
86             "outer_apothem": "b*cos(pi/12)",
87             "inner_chord": "2*a*sin(pi/12)",
88             "outer_chord": "2*b*sin(pi/12)",
89             "cross_edge": "sqrt(a*a+b*b-2*a*b*cos(pi/12))",
90             "maximum_edge": "max(a,inner_chord,outer_chord,cross_edge)",
91         },
92         "analytic_values_m": {

```

```

93         "outer_apothem": outer * cosine,
94         "inner_chord": inner_chord,
95         "outer_chord": outer_chord,
96         "cross_edge": cross_edge,
97         "maximum_edge": max(inner, inner_chord, outer_chord, cross_edge),
98     },
99     "rational_trig_bounds": {
100         "cos_15_lower": (9659, 10000),
101         "sin_15_lower": (1, 4),
102         "sin_15_upper": (259, 1000),
103     },
104     "robust_certificate": {
105         "shift_m": shift,
106         "per_waypoint_euclidean_error_bound_m": error,
107         "triangle_diameter_upper_bound_m": edge_bound,
108         "guard_disk_radius_m": 1800.0 + shift,
109         "ideal_forward_projection_lower_bound_m": shift,
110         "perturbed_forward_projection_lower_bound_m": shift - error,
111         "perturbed_detection_distance_upper_bound_m": edge_bound + shift + error,
112         "proof": "h=g+shift*n lies in the outer polygon; a triangle containing h has a vertex v with n.(v-g
113     )>=shift and |v-g|<=diameter+shift",
114     },
115     "explicit_open_scan_route": {
116         "indices": route,
117         "length_formula": "a+22*(a+b)*sin(pi/12)+cross_edge",
118         "analytic_length_m": inner + 11.0 * (inner_chord + outer_chord) + cross_edge,
119         "returns_to_origin": False,
120         "is_optimal": False,
121     },
122     "baseline_comparison": {
123         "unchanged_baseline": "enhanced Q4: s=990, 31 triangular-lattice points",
124         "baseline_waypoint_count": 31,
125         "candidate_waypoint_count": 25,
126         "baseline_full_20_channel_measurement_obligations": 620,
127         "candidate_full_20_channel_measurement_obligations": 500,
128         "claimed_total_runtime_gain": None,
129     },
130     "smaller_candidate": {
131         "accepted": False,
132         "reason": "No fewer-point construction is certified here; the notes exclude equal two-ring sizes
133         <=11 and 12 regular outer vertices with <=11 inner vertices of radius <=1000 under the diameter-
134         triangulation certificate.",
135         "global_minimality_claim": False,
136     },
137 }

```

Listing 29: 发现与清除联合调度候选

```

1  """Joint, obligation-preserving open-route dispatch from public track summaries.
2
3  This module only chooses a task. It does not certify a target position, execute
4  an action, mark an anchor visited, or remove any discovery obligation.
5  """
6
7  from __future__ import annotations
8
9  from math import dist, isfinite
10
11  Point = tuple[float, float]
12  Task = tuple[str, int]
13
14
15  def choose_task(
16      position: Point,
17      anchors: dict[int, Point],
18      targets: dict[int, tuple[Point, float]],
19      *,

```

```

20     choice: str = "guarded",
21 ) -> Task:
22     """Choose the first task of a joint nearest-insertion/open-2-opt route.
23
24     ``center`` minimizes a point-center open-route surrogate. ``guarded``
25     adds the first task's MEC radius (zero for anchors), discouraging an
26     immediate commitment to an uncertain center. Neither surrogate predicts
27     localization actions or total simulator time. Rebuild inputs after every
28     completed task; a returned target must use the existing certified localizer,
29     not clear at a surrogate point.
30
31     All tasks remain in the internal permutation, even at identical positions.
32     Ties use target before anchor, then ascending integer ID; insertion gaps
33     and 2-opt segments are scanned left to right. Inputs are not modified.
34     Both mappings empty, invalid choice, or nonfinite geometry raises ValueError.
35     Runtime is  $O(n^3)$ , storage  $O(n^2)$ , with at most  $n$  improving 2-opt moves.
36     """
37     if choice not in ("center", "guarded"):
38         raise ValueError("choice must be 'center' or 'guarded'")
39     tasks: list[Task] = [("target", key) for key in sorted(targets)]
40     tasks.extend(("anchor", key) for key in sorted(anchors))
41     n = len(tasks)
42     if not n:
43         raise ValueError("cannot dispatch an empty task set")
44
45     coords = [targets[key][0] for key in sorted(targets)]
46     coords.extend(anchors[key] for key in sorted(anchors))
47     radii = [targets[key][1] for key in sorted(targets)]
48     radii.extend(0.0 for _ in anchors)
49     if any(len(point) != 2 or not all(isfinite(x) for x in point)
50         for point in [position, *coords]):
51         raise ValueError("positions must be finite two-dimensional points")
52     if any(not isfinite(radius) or radius < 0 for radius in radii):
53         raise ValueError("MEC radii must be finite and nonnegative")
54     penalty = radii if choice == "guarded" else [0.0] * n
55     origin = [dist(position, point) for point in coords]
56     edges = [[0.0] * n for _ in range(n)]
57     for i in range(n):
58         for j in range(i):
59             edges[i][j] = edges[j][i] = dist(coords[i], coords[j])
60     if (any(not isfinite(value) for value in origin)
61         or any(not isfinite(value) for row in edges for value in row)):
62         raise ValueError("pairwise distances must be finite")
63
64     # Nearest insertion: select the task nearest the current route (including
65     # the fixed origin), then insert it in the least-cost open-route gap.
66     first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
67     route = [first]
68     remaining = set(range(n))
69     remaining.remove(first)
70     nearest = [min(origin[i], edges[first][i]) for i in range(n)]
71     while remaining:
72         node = min(remaining, key=lambda i: (nearest[i], i))
73         best_gap = 0
74         best_delta = (origin[node] + edges[node][route[0]] - origin[route[0]]
75                     + penalty[node] - penalty[route[0]])
76         for gap in range(1, len(route) + 1):
77             before = route[gap - 1]
78             delta = edges[before][node]
79             if gap < len(route):
80                 after = route[gap]
81                 delta += edges[node][after] - edges[before][after]
82             if delta < best_delta:
83                 best_gap, best_delta = gap, delta
84         route.insert(best_gap, node)
85         remaining.remove(node)
86     for other in remaining:

```

```

87         nearest[other] = min(nearest[other], edges[node][other])
88
89     # Symmetric internal edges cancel on reversal; an open tail has no return
90     # edge. A reversal starting at zero also changes the first-radius penalty.
91     # Cap accepted moves to make online CPU cost predictable, not a claim of
92     # reaching a local or global optimum.
93     for _ in range(n):
94         changed = False
95         for left in range(n - 1):
96             for right in range(left + 1, n):
97                 old = (origin[route[left]] if left == 0
98                      else edges[route[left - 1]][route[left]])
99                 new = (origin[route[right]] if left == 0
100                      else edges[route[left - 1]][route[right]])
101                 if right + 1 < n:
102                     old += edges[route[right]][route[right + 1]]
103                     new += edges[route[left]][route[right + 1]]
104                 if left == 0:
105                     old += penalty[route[left]]
106                     new += penalty[route[right]]
107                 if new + 1e-12 * max(1.0, old, new) < old:
108                     route[left:right + 1] = reversed(route[left:right + 1])
109                     changed = True
110                     break
111             if changed:
112                 break
113         if not changed:
114             break
115     return tasks[route[0]]

```

Listing 30: 分层光学覆盖候选（非默认）

```

1  """Deterministic, whole-region optical-cover candidate for convex polygons.
2
3  The existing geometry.optical_cover remains the explicit fallback/control.
4  Only public feasible-region vertices are used. No actions or state are mutated.
5  """
6
7  from __future__ import annotations
8
9  from fractions import Fraction
10 from math import ceil, dist, hypot, isfinite, sqrt, ulp
11
12 from geometry import _farthest_pair, optical_cover
13
14 Point = tuple[float, float]
15 _ExactPoint = tuple[Fraction, Fraction]
16
17
18 def _cycle_length(points: list[Point]) -> float:
19     return sum(dist(points[i - 1], point) for i, point in enumerate(points))
20
21
22 def _strip_bounds(
23     vertices: list[_ExactPoint],
24     edges: list[tuple[Fraction, Fraction, Fraction, Fraction]],
25     left: Fraction,
26     right: Fraction,
27 ) -> tuple[Fraction, Fraction] | None:
28     """Exact transverse extrema of polygon intersected with a closed slab."""
29     values = [y for x, y in vertices if left <= x <= right]
30     # Edge tuple: left endpoint x/y, right endpoint x, exact dy/dx.
31     # Vertical edges need no intersections: their endpoints are already above.
32     for x, y, end, slope in edges:
33         if x <= left <= end:
34             values.append(y + (left - x) * slope)
35         if x <= right <= end:

```

```

36         values.append(y + (right - x) * slope)
37     if not values:
38         return None
39     return min(values), max(values)
40
41
42 def _certified_center(
43     left: Fraction,
44     right: Fraction,
45     bottom: Fraction,
46     top: Fraction,
47     origin: _ExactPoint,
48     axis: _ExactPoint,
49     radius_squared: Fraction,
50 ) -> Point | None:
51     """Certify the actual returned float center against four exact corners."""
52     ox, oy = origin
53     ux, uy = axis
54     x, y = (left + right) / 2, (bottom + top) / 2
55     center = (float(ox + x * ux - y * uy), float(oy + x * uy + y * ux))
56     if not all(isfinite(value) for value in center):
57         return None
58     cx, cy = Fraction(center[0]), Fraction(center[1])
59     for x in (left, right):
60         for y in (bottom, top):
61             dx = ox + x * ux - y * uy - cx
62             dy = oy + x * uy + y * ux - cy
63             if dx * dx + dy * dy > radius_squared:
64                 return None
65     return center
66
67
68 def certified_optical_cover(poly: list[Point], radius: float = 19.9) -> list[Point]:
69     """Cover the entire convex region using certified, locally trimmed slabs.
70
71     Accepts either winding, repeated vertices, points and line segments. Empty
72     input raises ValueError (the legacy planner also treats it as infeasible).
73     Output centers may lie outside the polygon/arena, as with optical_cover.
74
75     Every new cell has an exact rational four-corner disk certificate for the
76     returned float center. Any unsuccessful candidate is discarded in full;
77     the existing cover is the safe fallback. Selection never increases cover
78     count or cyclic travel relative to that control, but entry travel depends
79     on the caller's position and is not covered by that comparison.
80     """
81     if not isfinite(radius) or radius <= 0:
82         raise ValueError("Optical covering radius must be finite and positive")
83     if not poly:
84         raise ValueError("An infeasible region has no optical cover")
85     if any(not (isfinite(x) and isfinite(y)) for x, y in poly):
86         raise ValueError("Geometry requires finite coordinates")
87
88     control = optical_cover(poly, radius)
89     if len(control) <= 1:
90         return control
91     origin_float, end, diameter = _farthest_pair(poly)
92     if not isfinite(diameter) or diameter == 0:
93         return control
94     ux = (end[0] - origin_float[0]) / diameter
95     uy = (end[1] - origin_float[1]) / diameter
96     norm = hypot(ux, uy)
97     if not isfinite(norm) or norm == 0:
98         return control
99     axis = Fraction(ux / norm), Fraction(uy / norm)
100     origin = Fraction(origin_float[0]), Fraction(origin_float[1])
101     norm_squared = axis[0] ** 2 + axis[1] ** 2
102     metric_scale = sqrt(float(norm_squared))

```

```

103 # Treat the float axis as exact, not as an exactly unit vector. This exact
104 # inverse and the corner check remove reliance on rounded orthogonality.
105 vertices = []
106 for px, py in poly:
107     dx, dy = Fraction(px) - origin[0], Fraction(py) - origin[1]
108     vertices.append(
109         ((dx * axis[0] + dy * axis[1]) / norm_squared,
110          (-dx * axis[1] + dy * axis[0]) / norm_squared)
111     )
112 edges = []
113 for index, a in enumerate(vertices):
114     b = vertices[index - 1]
115     if a[0] == b[0]:
116         continue
117     if a[0] > b[0]:
118         a, b = b, a
119     edges.append((a[0], a[1], b[0], (b[1] - a[1]) / (b[0] - a[0])))
120 xmin = min(x for x, _ in vertices)
121 xmax = max(x for x, _ in vertices)
122 span = xmax - xmin
123 height = float(max(y for _, y in vertices) - min(y for _, y in vertices))
124 scale = max(diameter, *(max(abs(x), abs(y)) for x, y in poly))
125 guard = max(1e-9, 64.0 * ulp(max(1.0, scale)))
126 safe_radius = radius - 8.0 * guard
127 if safe_radius <= 0 or not isfinite(height):
128     return control
129 local_radius = safe_radius / metric_scale
130 # Square-ish slabs serve broad regions; elongated slabs serve narrow ones.
131 widths = [sqrt(2.0) * local_radius, sqrt(3.0) * local_radius, local_radius]
132 if height < 2.0 * local_radius:
133     widths.append(2.0 * sqrt((local_radius - height / 2.0)
134                             * (local_radius + height / 2.0)))
135 radius_squared = Fraction(radius) ** 2
136 control_cycle = _cycle_length(control)
137 best = control
138 best_cost = 3.0 * len(control) + control_cycle / 5.0
139 tried_counts: set[int] = set()
140 for width in widths:
141     if width <= 0 or not isfinite(width):
142         continue
143     count = max(1, ceil(span / Fraction(width)))
144     if count in tried_counts or count > len(control):
145         continue
146     tried_counts.add(count)
147     slab_width = span / count
148     half_width = float(slab_width) / 2.0
149     if half_width >= local_radius:
150         continue
151     capacity = 2.0 * sqrt((local_radius - half_width)
152                          * (local_radius + half_width))
153     if capacity <= 0 or not isfinite(capacity):
154         continue
155     candidate: list[Point] = []
156     valid = True
157     for index in range(count):
158         left = xmin + index * slab_width
159         right = xmin + (index + 1) * slab_width
160         bounds = _strip_bounds(vertices, edges, left, right)
161         if bounds is None:
162             # For a nonempty convex polygon every slab across its x-span
163             # intersects it. Do not silently skip an unexplained hole.
164             valid = False
165             break
166         bottom, top = bounds
167         rows = max(1, ceil((top - bottom) / Fraction(capacity)))
168         if len(candidate) + rows > len(control):
169             valid = False

```

```

170         break
171     step = (top - bottom) / rows
172     indices = range(rows) if index % 2 == 0 else range(rows - 1, -1, -1)
173     for row in indices:
174         point = _certified_center(
175             left, right, bottom + row * step, bottom + (row + 1) * step,
176             origin, axis, radius_squared,
177         )
178         if point is None:
179             valid = False
180             break
181         candidate.append(point)
182     if not valid:
183         break
184     if not valid:
185         continue
186     cycle = _cycle_length(candidate)
187     cost = 3.0 * len(candidate) + cycle / 5.0
188     if cycle <= control_cycle and cost < best_cost:
189         best, best_cost = candidate, cost
190     return best

```

Listing 31: 独立开发集上的候选筛选

```

1  """Paired development screen of independent candidates; no official calls."""
2  from __future__ import annotations
3
4  import argparse
5  import json
6  import math
7  import statistics
8  from pathlib import Path
9
10 from experiments import aggregate, run_case
11
12 # Fixed candidates, not a Cartesian hyperparameter search. Every run retains
13 # full discovery obligations and the same bounded-probe localization contract.
14 CONFIGS = {
15     "control": {},
16     "optical": {"optical_policy": "slabs"},
17     "joint_center": {"dispatch_policy": "center"},
18     "joint_guarded": {"dispatch_policy": "guarded"},
19     "polar_seed": {"coverage_policy": "seed25"},
20     "polar_compact": {"coverage_policy": "compact25"},
21     "polar_joint": {"coverage_policy": "compact25", "dispatch_policy": "center"},
22     "polar_guarded": {"coverage_policy": "compact25", "dispatch_policy": "guarded"},
23     "combined": {"coverage_policy": "compact25", "optical_policy": "slabs", "dispatch_policy": "center"},
24     "joint_unknown": {"dispatch_policy": "center", "scan_policy": "unknown"},
25     "joint_useful": {"dispatch_policy": "center", "scan_policy": "useful"},
26     "polar_unknown": {"coverage_policy": "compact25", "dispatch_policy": "guarded", "scan_policy": "unknown"},
27     "polar_useful": {"coverage_policy": "compact25", "dispatch_policy": "guarded", "scan_policy": "useful"},
28 }
29
30
31 def paired(control, candidate):
32     assert [r["seed"] for r in control] == [r["seed"] for r in candidate]
33     values = [a["average_time_s"] - b["average_time_s"] for a, b in zip(control, candidate, strict=True)]
34     mean = statistics.mean(values)
35     sem = statistics.stdev(values) / math.sqrt(len(values)) if len(values) > 1 else 0.0
36     return {"mean_saved_s_per_source": mean,
37             "reduction_pct": 100 * mean / statistics.mean(r["average_time_s"] for r in control),
38             "approx_95pct_ci_s": [mean - 1.96 * sem, mean + 1.96 * sem],
39             "faster_cases": sum(value > 0 for value in values), "cases": len(values)}
40
41
42 def main():
43     parser = argparse.ArgumentParser()

```

```

44 parser.add_argument("--cases", type=int, default=150)
45 parser.add_argument("--seed", type=int, default=3100000)
46 parser.add_argument("--output", type=Path, default=Path("results/refinement_development.json"))
47 args = parser.parse_args()
48 if args.cases < 2:
49     parser.error("At least two cases are required")
50 rows, summary = [], {}
51 for problem in (3, 4):
52     names = ("control", "optical", "joint_center", "joint_guarded", "joint_unknown", "joint_useful") if
53     problem == 3 else tuple(name for name in CONFIGS if name not in ("joint_unknown", "joint_useful"))
54     control = None
55     for name in names:
56         group = []
57         for seed in range(args.seed, args.seed + args.cases):
58             row = run_case(seed, problem, "enhanced", **CONFIGS[name])
59             row["configuration"] = name
60             group.append(row)
61         rows.extend(group)
62         if name == "control":
63             control = group
64             key = f"Q{problem}_{name}"
65             summary[key] = {**aggregate(group), **paired(control, group)}
66             print(key, json.dumps(summary[key], ensure_ascii=False), flush=True)
67 result = {"evidence": "development_only_local_synthetic_not_official",
68         "seed_range": [args.seed, args.seed + args.cases - 1],
69         "configs": CONFIGS, "total_runs": len(rows), "summary": summary, "runs": rows}
70 args.output.parent.mkdir(parents=True, exist_ok=True)
71 args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n")
72
73 if __name__ == "__main__":
74     main()

```

Listing 32: 冻结策略的样本外消融与压力实验

```

1  """Frozen holdout/ablation/robustness study; all observations are synthetic.
2
3  Development uses 3100000..3100149 only. No choice is made from this file's
4  holdout results: refined defaults are frozen before its first execution.
5  """
6  from __future__ import annotations
7
8  import json
9  from pathlib import Path
10
11  from experiments import aggregate, run_case
12  from refinement_study import paired
13
14  CONFIGS = {
15      3: {
16          "control": ("enhanced", {}),
17          "joint_all": ("enhanced", {"dispatch_policy": "center"}),
18          "refined": ("refined", {}),
19      },
20      4: {
21          "control": ("enhanced", {}),
22          "polar_only": ("enhanced", {"coverage_policy": "compact25"}),
23          "joint_all": ("enhanced", {"coverage_policy": "compact25", "dispatch_policy": "guarded"}),
24          "refined": ("refined", {}),
25      },
26  }
27
28
29  def group(problem, name, seeds, scenario="random", mode="smooth", **override):
30      strategy, config = CONFIGS[problem][name]
31      result = []
32      for seed in seeds:

```

```

33     row = run_case(seed, problem, strategy, scenario, mode, **(config | override))
34     row["configuration"] = name
35     result.append(row)
36     return result
37
38
39 def main():
40     rows, summary, comparisons, stress, sensitivity = [], {}, {}, {}, {}
41     for problem in (3, 4):
42         current = {}
43         for name in CONFIGS[problem]:
44             current[name] = group(problem, name, range(4100000,4101000))
45             rows.extend(current[name])
46             summary[f"Q{problem}_{name}"] = aggregate(current[name])
47             comparisons[f"Q{problem}"] = paired(current["control"],current["refined"])
48             order=list(current)
49             comparisons[f"Q{problem}_incremental"] = {
50                 f"{a}_to_{b}":paired(current[a],current[b]) for a,b in zip(order,order[1:])}
51             print(f"holdout Q{problem}",json.dumps(comparisons[f"Q{problem}"]),flush=True)
52             scenarios=[(scenario,mode,range(4200000,4200050))
53                         for scenario in ("minimum_radius","outward_boundary","clustered")
54                         for mode in ("smooth","extreme","iid_location")]
55             scenarios.extend(("random",mode,range(4210000,4210200)) for mode in ("extreme","iid_location"))
56             for scenario,mode,seeds in scenarios:
57                 control=group(problem,"control",seeds,scenario,mode)
58                 refined=group(problem,"refined",seeds,scenario,mode)
59                 rows.extend(control);rows.extend(refined)
60                 key=f"Q{problem}_{scenario}_{mode}"
61                 stress[key]={"control":aggregate(control),"refined":aggregate(refined),"paired":paired(control,
62                 refined)}
63                 print(key,json.dumps(stress[key]["paired"]),flush=True)
64             for scale in (.12,.22,.4):
65                 selected=group(problem,"refined",range(4300000,4300050),probe_scale=scale)
66                 rows.extend(selected)
67                 sensitivity[f"Q{problem}_probe_{scale}"]=aggregate(selected)
68             result={"evidence":"frozen_held_out_local_synthetic_not_official",
69                     "selection":"Q3 center/unknown, Q4 compact25/guarded/useful frozen using development
70                     3100000..3100149 only; rectangle optical cover retained",
71                     "development_seed_range":[3100000,3100149],"main_seed_range":[4100000,4100999],
72                     "stress_seed_range":[4200000,4200049],"alternate_error_seed_range":[4210000,4210199],
73                     "sensitivity_seed_range":[4300000,4300049],"main_cases_per_problem":1000,
74                     "configs":CONFIGS,"total_runs":len(rows),"summary":summary,"comparison":comparisons,
75                     "stress":stress,"sensitivity":sensitivity,"runs":rows}
76             Path("results/research_experiments.json").write_text(json.dumps(result,ensure_ascii=False,indent=2)+"\n")
77             print(json.dumps({"total_runs":len(rows),"summary":summary,"comparison":comparisons},ensure_ascii=False,
78                             indent=2))
79
80 if __name__=="__main__":
81     main()

```

Listing 33: 新覆盖与策略行为的独立核验

```

1  """Independent geometry/behavior verification for the new research candidates."""
2  from __future__ import annotations
3
4  from collections import Counter
5  from decimal import Decimal, localcontext
6  from fractions import Fraction
7  import json
8  import math
9  import random
10 from pathlib import Path
11
12 from geometry import clip_bearing, initial_polygon, optical_cover
13 from joint_dispatch_candidate import choose_task
14 from optical_policy_candidate import certified_optical_cover

```

```

15 from planner import Planner, Track
16 from verify_terminal import DeadlineAdapter
17 from polar_coverage_candidate import polar_certificate, polar_waypoints
18
19
20 def cross(a, b, c):
21     return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
22
23
24 def verify_polar(variant):
25     cert = polar_certificate(variant)
26     points = polar_waypoints(variant)
27     triangles = cert["triangles_ccw"]
28     counts = Counter(tuple(sorted((t[k], t[(k+1)%3])) for t in triangles for k in range(3)))
29     boundary = {tuple(sorted((13+k, 13+(k+1)%12))) for k in range(12)}
30     assert len(points) == len(set(points)) == 25 and points[0] == (0., 0.)
31     assert {edge for edge, count in counts.items() if count == 1} == boundary
32     assert all(count == (1 if edge in boundary else 2) for edge, count in counts.items())
33     assert len(counts) == 60 and len(triangles) == 36
34     assert all(cross(*(points[i] for i in triangle)) > 0 for triangle in triangles)
35     directed = Counter((t[k], t[(k+1)%3]) for t in triangles for k in range(3))
36     for a,b in counts:
37         if (a,b) not in boundary:
38             assert directed[a,b] == directed[b,a] == 1
39     for a,b in counts:
40         for c,d in counts:
41             if len({a,b,c,d}) < 4:
42                 continue
43             # Strict crossings would invalidate the triangulation argument.
44             assert not (cross(points[a],points[b],points[c])*cross(points[a],points[b],points[d]) < -1e-8
45                         and cross(points[c],points[d],points[a])*cross(points[c],points[d],points[b]) < -1e-8)
46     area = sum(cross(*(points[i] for i in t))/2 for t in triangles)
47     outer_area = sum(cross((0,0),points[13+k],points[13+(k+1)%12])/2 for k in range(12))
48     assert abs(area-outer_area) < 1e-6
49     maximum = max(math.dist(points[a],points[b]) for a,b in counts)
50     bound = cert["robust_certificate"]
51     assert maximum <= bound["triangle_diameter_upper_bound_m"] + 1e-9
52     # Exact rational lower/upper radical bounds used by the written proof.
53     assert Fraction(14142,10000)**2 < 2 < Fraction(14143,10000)**2
54     assert Fraction(24494,10000)**2 < 6 < Fraction(24495,10000)**2
55     a,b = map(int, (cert["inner_radius_m"], cert["outer_radius_m"]))
56     assert Fraction(b*9659,10000) > bound["guard_disk_radius_m"]
57     assert a*a+b*b-2*a*b*Fraction(9659,10000) < bound["triangle_diameter_upper_bound_m"]**2
58     with localcontext() as context:
59         context.prec = 70
60         root2, root3, root6 = Decimal(2).sqrt(), Decimal(3).sqrt(), Decimal(6).sqrt()
61         c15, s15 = (root6+root2)/4, (root6-root2)/4
62         expected = [(Decimal(0), Decimal(0))]
63         for radius, start in ((a,(Decimal(1),Decimal(0))), (b,(c15,s15))):
64             x,y = start
65             for k in range(12):
66                 expected.append((radius*x,radius*y))
67                 x,y = x*root3/2-y/2, x/2+y*root3/2
68             error = max(float(((Decimal(px)-ex)**2+(Decimal(py)-ey)**2).sqrt())
69                        for (px,py),(ex,ey) in zip(points,expected,strict=True))
70     assert error < 1e-8 < bound["per_waypoint_euclidean_error_bound_m"]
71     rng = random.Random(340091)
72     positions = [(1800*math.cos(k*math.pi/180),1800*math.sin(k*math.pi/180)) for k in range(360)]
73     positions += points[:13]
74     positions += [(points[a][0]+points[b][0])/2, (points[a][1]+points[b][1])/2)
75                 for a,b in counts if math.hypot((points[a][0]+points[b][0])/2,
76                 (points[a][1]+points[b][1])/2)<=1800]
77     for _ in range(400):
78         r,phi = 1800*math.sqrt(rng.random()),rng.uniform(0,math.tau)
79         positions.append((r*math.cos(phi),r*math.sin(phi)))
80     checks, worst_distance, least_forward = 0, 0.0, float("inf")
81     shift = bound["shift_m"]

```

```

82     perturbation_cases = 0
83     special = list(points[:13])
84     special.extend(((points[a][0]+points[b][0])/2, (points[a][1]+points[b][1])/2)
85                     for a,b in counts if math.hypot((points[a][0]+points[b][0])/2,
86                                                         (points[a][1]+points[b][1])/2)<1790)
87     for h in special:
88         n = (math.cos(.713), math.sin(.713))
89         positions.append((h[0]-shift*n[0], h[1]-shift*n[1]))
90     for g in positions:
91         angles = [k*math.pi/12 for k in range(24)] + [.713]
92         for angle in angles:
93             n = math.cos(angle), math.sin(angle)
94             h = g[0]+shift*n[0], g[1]+shift*n[1]
95             containing = next(t for t in triangles if all(cross(points[t[i]], points[t[(i+1)%3]], h) >= -1e-7 for
96                     i in range(3)))
97             v = max((points[i] for i in containing), key=lambda p: (p[0]-g[0])*n[0]+(p[1]-g[1])*n[1])
98             projection = (v[0]-g[0])*n[0]+(v[1]-g[1])*n[1]
99             distance = math.dist(v, g)
100             assert projection >= shift-1e-8 and distance <= maximum+shift+1e-8
101             epsilon = bound["per_waypoint_euclidean_error_bound_m"]
102             for offset in ((-epsilon*n[0], -epsilon*n[1]),
103                             (epsilon*(v[0]-g[0])/distance, epsilon*(v[1]-g[1])/distance)):
104                 q = v[0]+offset[0], v[1]+offset[1]
105                 assert (q[0]-g[0])*n[0]+(q[1]-g[1])*n[1] >= shift-epsilon-1e-7
106                 assert math.dist(q, g) <= bound["perturbed_detection_distance_upper_bound_m"]+1e-7
107                 perturbation_cases += 1
108                 checks += 1
109                 worst_distance=max(worst_distance, distance)
110                 least_forward=min(least_forward, projection)
111 route=cert["explicit_open_scan_route"]["indices"]
112 route_length=sum(math.dist(points[a], points[b]) for a,b in zip(route, route[1:]))
113 assert abs(route_length-cert["explicit_open_scan_route"]["analytic_length_m"])<1e-7
114 return {"variant":variant, "triangles":len(triangles), "edges":len(counts),
115         "noncrossing_tiling_verified":True, "maximum_edge_m":maximum,
116         "coordinate_error_against_decimal_m":error, "directional_cases":checks,
117         "adverse_single_vertex_perturbations":perturbation_cases,
118         "maximum_witness_distance_m":worst_distance, "minimum_witness_projection_m":least_forward,
119         "explicit_route_length_m":route_length, "robust_certificate":bound}
120
121 def verify_optical():
122     rng=random.Random(342091)
123     polygons=[[(0.,0.), [(0.,0.), (1000.,0.), [(0.,0.), (1000.,1e-9), (500.,0.),
124                 [(0.,0.), (1500.,0.), (1500.,52.)],
125                 [(-100.,-40.), (100.,-40.), (100.,40.), (-100.,40.)]]]
126     for _ in range(120):
127         phi=rng.uniform(0,math.tau)
128         r=1800*math.sqrt(rng.random())
129         g=r*math.cos(phi), r*math.sin(phi)
130         poly=initial_polygon()
131         for j in range(rng.randrange(1,5)):
132             theta=rng.uniform(0,math.tau); distance=rng.uniform(6,1500)
133             s=g[0]+distance*math.cos(theta), g[1]+distance*math.sin(theta)
134             poly=clip_bearing(poly, s, (math.degrees(theta)+180+rng.uniform(-1,1))%360)
135         assert poly
136         polygons.append(poly)
137     samples=0; improved=0; max_distance=0.0
138     for poly in polygons:
139         old=optical_cover(poly); new=certified_optical_cover(poly)
140         old_cost=3*len(old)+sum(math.dist(p, old[i-1]) for i,p in enumerate(old))/5
141         new_cost=3*len(new)+sum(math.dist(p, new[i-1]) for i,p in enumerate(new))/5
142         assert new_cost<=old_cost+1e-7
143         improved+=new_cost<old_cost-1e-7
144         probes=list(poly)
145         for a,b in zip(poly, poly[1:]+poly[:1]):
146             probes.extend(((1-t/20)*a[0]+t/20*b[0], (1-t/20)*a[1]+t/20*b[1]) for t in range(21))
147         for _ in range(100):

```

```

148         weights=[rng.random() for _ in poly]; total=sum(weights)
149         probes.append(tuple(sum(w*p[k] for w,p in zip(weights,poly))/total for k in (0,1)))
150     for p in probes:
151         distance=min(math.dist(p,c) for c in new)
152         assert distance<=19.9+1e-7
153         max_distance=max(max_distance,distance);samples+=1
154     return {"polygons":len(polygons),"interior_and_boundary_samples":samples,
155           "maximum_sampled_distance_m":max_distance,"lower_cover_proxy_cost_cases":improved,
156           "scope":"interior sampling supplements exact per-cell corner certificates, not a proof by sampling"
157     }
158
159 def verify_dispatch():
160     anchors={7:(-2.,0.),9:(10.,0.)};targets={3:((1.,0.),20.)}
161     kinds=[]
162     for mode in ("center","guarded"):
163         remaining_a=dict(anchors);remaining_t=dict(targets);position=(0.,0.);visited=[]
164         while remaining_a or remaining_t:
165             kind,key=choose_task(position,remaining_a,remaining_t,choice=mode)
166             position=remaining_a.pop(key) if kind=="anchor" else remaining_t.pop(key)[0]
167             visited.append((kind,key))
168             assert set(visited)=={("anchor",7),("anchor",9),("target",3)}
169             kinds.append({"choice":mode,"completed":visited})
170     assert choose_task((0,0),{1:(0,0)},{2:((0,0),0)}) in (("anchor",1),("target",2))
171     try:
172         choose_task((0,0),{},{})
173     except ValueError:
174         pass
175     else:
176         raise AssertionError("Empty dispatch must not fabricate a task")
177     return kinds
178
179
180 def verify_scan_obligations():
181     records = []
182     for problem, mode in ((3,"unknown"),(4,"useful")):
183         adapter = DeadlineAdapter(max_virtual=360000)
184         policy = Planner(adapter,problem=problem)
185         policy.action("/enter")
186         # A previously detected, unresolved singleton: a ready optical target,
187         # not an absent channel even when every other channel has been scanned.
188         policy.tracks[1] = Track(polygon=[(100.,0.)],readings=[((0.,0.),0.)])
189         policy.channel_state[1] = "detected"
190         for index in sorted(policy.pending):
191             policy.scan_anchor(index)
192             assert not policy.scan_records[1]
193             assert not policy._result()["certificate_complete"]
194             assert policy.channel_state[1] == "detected"
195             policy.measure(1,(200.,0.))
196             assert policy.channel_state[1] == "detected"
197             policy.clear(1,(100.,0.))
198             result = policy._result()
199             assert result["certificate_complete"] and result["n_cleared"] == 1
200             assert all(state == ("cleared" if channel == 1 else "absent-certified")
201                       for channel,state in result["channel_certificates"].items())
202             records.append({"problem":problem,"scan_policy":mode,
203                           "known_skip_never_certified_absent":True,
204                           "negative_reading_kept_detected":True,
205                           "success_completed_remaining_obligation":True})
206     return records
207
208
209 def main():
210     polar=[verify_polar(variant) for variant in ("seed25","compact25")]
211     assert polar[0]["explicit_route_length_m"]-polar[1]["explicit_route_length_m"]>179.5
212     result={"evidence":"independent_finite_geometry_and_behavior_checks_not_official",
213           "polar":polar,"optical":verify_optical(),"dispatch":verify_dispatch(),

```

```

214         "scan_obligations":verify_scan_obligations()}}
215     Path("results/research_verification.json").write_text(json.dumps(result,ensure_ascii=False,indent=2)+"\n")
216     print(json.dumps(result,ensure_ascii=False,indent=2))
217
218
219 if __name__=="__main__":
220     main()

```

Listing 34: 新研究数字、图表与轨迹生成

```

1  """Generate checked research tables, vector figures and observable trajectories."""
2  from __future__ import annotations
3
4  from collections import defaultdict
5  import hashlib
6  import json
7  import math
8  import statistics
9  from pathlib import Path
10
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 import numpy as np
15
16 from coverage import directional_waypoints
17 from environment import LocalSimulator
18 from experiments import aggregate
19 from make_assets import atomic_bytes, table, save_figure, number, write_json, GENERATED, ROOT
20 from planner import Planner
21 from polar_coverage_candidate import polar_certificate, polar_waypoints
22 from refinement_study import paired
23
24 plt.rcParams.update({"font.size":10,"axes.labelsize":10,"legend.fontsize":9,
25                     "pdf.fonttype":42,"ps.fonttype":42,"axes.spines.top":False,"axes.spines.right":False})
26 LABELS={"control":"原增强", "refined":"最终策略", "polar_only":"仅双环", "joint_all":"联合调度",
27         "optical":"条带光学", "joint_center":"中心联合", "joint_guarded":"半径联合",
28         "joint_unknown":"中心联合+未知扫描", "joint_useful":"中心联合+筛选补测",
29         "polar_seed":"双环990/1865", "polar_compact":"双环940/1870",
30         "polar_joint":"双环+中心联合", "polar_guarded":"双环+半径联合",
31         "polar_unknown":"双环联合+未知扫描", "polar_useful":"双环联合+筛选补测", "combined":"双环中心联合+条带"}
32
33
34 def checked_rows(data):
35     assert data["total_runs"] == len(data["runs"])
36     for row in data["runs"]:
37         assert row["evidence"] == "local_synthetic_not_official"
38         assert row["certificate_complete"] and row["timing_independently_verified"]
39         assert row["n_cleared"] == row["n_sources"] and row["fraction_cleared"] == 1
40         assert row["exit_reason"] == "user_exit"
41         assert math.isclose(row["average_time_s"],row["virtual_time_s"]/row["n_cleared"],abs_tol=1e-9)
42         cost=row["distance_m"]/5+5*row["measures"]+row["switches"]+3*row["failed_clears"]+5*row["n_cleared"]
43         assert abs(cost-row["virtual_time_s"]) <= (row["measures"]+row["clear_attempts"])*.000001+1e-8
44     return data["runs"]
45
46
47 def main_tables(data):
48     groups=defaultdict(list)
49     for row in checked_rows(data):
50         if 4100000 <= row["seed"] <= 4100999:
51             assert row["scenario"]=="random" and row["error_mode"]=="smooth"
52             groups[row["problem"],row["configuration"]].append(row)
53     comparison_rows=[];cost_rows=[];ablation_rows=[];macro=[]
54     for problem,configs in data["configs"].items():
55         problem=int(problem)
56         for name in configs:
57             current=sorted(groups[problem,name],key=lambda r:r["seed"])

```

```

58     assert [row["seed"] for row in current]==list(range(4100000,4101000))
59     assert aggregate(current)==data["summary"][f"Q{problem}_{name}"]
60     s=aggregate(current)
61     comparison_rows.append([f"Q{problem}",LABELS[name],number(s["mean_average_time_s"]),
62         number(s["median_average_time_s"]),number(s["p90_average_time_s"]),
63         number(s["worst_average_time_s"]),f'{s["all_cleared_cases"]}/{s["cases"]}')])
64     if name in ("control","refined"):
65         parts=[statistics.mean(row["distance_m"]/(5*row["n_sources"] for row in current),
66             statistics.mean(5*row["measures"]/row["n_sources"] for row in current),
67             statistics.mean(row["switches"]/row["n_sources"] for row in current),
68             statistics.mean(3*row["failed_clears"]/row["n_sources"] for row in current),5.]
69         cost_rows.append([f"Q{problem}",LABELS[name],*[number(v) for v in parts],number(s["
mean_average_time_s"])]])
70     pair=paired(groups[problem,"control"],groups[problem,"refined"])
71     assert pair==data["comparison"][f"Q{problem}"]
72     word="Three" if problem==3 else "Four"
73     macro.extend([(f"ResearchQ{word}Time",number(data["summary"][f"Q{problem}_refined"]["
mean_average_time_s"])),
74         (f"ResearchQ{word}Gain",number(pair["reduction_pct"])))])
75     macro.extend([(f"ResearchQ{word}Saved",number(pair["mean_saved_s_per_source"])),
76         (f"ResearchQ{word}Lower",number(pair["approx_95pct_ci_s"][0])),
77         (f"ResearchQ{word}Upper",number(pair["approx_95pct_ci_s"][1]))])
78     for key,value in data["comparison"][f"Q{problem}_incremental"].items():
79         a,b=key.split("_to_")
80         assert paired(groups[problem,a],groups[problem,b])==value
81         ablation_rows.append([f"Q{problem}",LABELS[a]+"至"+LABELS[b],number(value["mean_saved_s_per_source"
]),
82             f'[{number(value["approx_95pct_ci_s"][0])}, {number(value["approx_95pct_ci_s"][1])}]',f'{value
["faster_cases"]}/1000')])
83     macro.append(("ResearchRuns",str(data["total_runs"])))
84     atomic_bytes(GENERATED/"research_numbers.tex",("\n".join("\newcommand{\"+k+\">{+v+}\" for k,v in macro)+\"
\n").encode())
85     table("research_comparison.tex",["问题","配置","均值","中位数","$p_{90}$","最坏","全清"],comparison_rows)
86     table("research_costs.tex",["问题","配置","移动","检测","换频","失败","成功","合计"],cost_rows)
87     table("research_ablation.tex",["问题","单步增量","节省均值","r近似95%区间","更快案例"],ablation_rows)
88     return groups
89
90
91 def auxiliary_tables(data,development):
92     scenario_labels={"minimum_radius":"最小半径","outward_boundary":"边界外向","clustered":"边缘聚集","random":"
随机位置"}
93     mode_labels={"smooth":"平滑","extreme":"端点","iid_location":"位置哈希"}
94     pressure=[]
95     for key,item in data["stress"].items():
96         problem=int(key[1]);suffix=key[3:]
97         scenario=next(s for s in scenario_labels if suffix.startswith(s+"_"))
98         mode=suffix[len(scenario)+1:]
99         control=[r for r in data["runs"] if r["problem"]==problem and r["scenario"]==scenario and r["error_mode"
]=="mode and r["configuration"]=="control" and 4200000<=r["seed"]<4220000]
100         refined=[r for r in data["runs"] if r["problem"]==problem and r["scenario"]==scenario and r["error_mode"
]=="mode and r["configuration"]=="refined" and 4200000<=r["seed"]<4220000]
101         assert aggregate(control)==item["control"] and aggregate(refined)==item["refined"]
102         assert paired(control,refined)==item["paired"]
103         pressure.append([f"Q{problem}",scenario_labels[scenario],mode_labels[mode],
104             number(item["control"]["mean_average_time_s"]),number(item["refined"]["mean_average_time_s"]),
105             number(item["paired"]["mean_saved_s_per_source"]),f'{item["paired"]["faster_cases"]}/{len(control)}
'])
106     table("research_stress.tex",["问题","位置场景","误差场","对照","最终","节省","更快"],pressure)
107     sensitivities=[]
108     for key,value in data["sensitivity"].items():
109         p=int(key[1]);scale=float(key.split("_")[-1])
110         current=[r for r in data["runs"] if r["problem"]==p and 4300000<=r["seed"]<4300050 and r["probe_scale"
"]==scale]
111         assert aggregate(current)==value
112         sensitivities.append([f"Q{p}",str(scale),number(value["mean_average_time_s"]),number(value["
p90_average_time_s"]),f'{value["all_cleared_cases"]}/50')])
113     table("research_sensitivity.tex",["问题","横偏移比例","均值","$p_{90}$","全清"],sensitivities)

```

```

114     checked_rows(development)
115     screens=[]
116     for key,value in development["summary"].items():
117         p=int(key[1]);name=key[3:]
118         current=[r for r in development["runs"] if r["problem"]==p and r["configuration"]==name]
119         assert len(current)==150
120         screens.append([f"Q{p}",LABELS[name],number(value["mean_average_time_s"]),number(value["
mean_saved_s_per_source"])] ,f'{value["all_cleared_cases"]}/150'])
121     table("research_screen.tex",["问题","开发候选","均值","较对照节省","全清"],screens)
122
123
124 def coverage_figure():
125     fig,axes=plt.subplots(1,2,figsize=(16/2.54,3.5),layout="constrained")
126     for ax,points,label in zip(axes,[directional_waypoints(),polar_waypoints("compact25")],["Control: 31
lattice points","Refined: 25 polar points"]):
127         points=np.asarray(points)
128         ax.add_patch(plt.Circle((0,0),1800,fill=False,color="black",lw=1.4,label="Source disk"))
129         ax.scatter(points[:,0],points[:,1],s=15,color="#0072B2",zorder=3)
130         if "25" in label:
131             for t in polar_certificate("compact25")["triangles_ccw"]:
132                 p=points[list(t)+[t[0]]]
133                 ax.plot(p[:,0],p[:,1],color="#56B4E9",lw=.5,alpha=.7)
134             ax.set(xlim=(-2800,2800),ylim=(-2800,2800),xlabel="East (m)",ylabel="North (m)",title=label)
135             ax.set_aspect("equal");ax.set_xticks([-2000,0,2000]);ax.set_yticks([-2000,0,2000]);ax.grid(alpha=.15)
136         save_figure(fig,"research_coverage.pdf")
137
138
139 def performance_figure(groups):
140     fig,axes=plt.subplots(1,2,figsize=(16/2.54,3.2),layout="constrained")
141     for p,ax in zip((3,4),axes):
142         for name,color in (("control","#0072B2"),("refined","#D55E00")):
143             values=sorted(r["average_time_s"] for r in groups[p,name])
144             ax.plot(values,np.arange(1,len(values)+1)/len(values),label="Control" if name=="control" else "
Refined",color=color,lw=1.5)
145             ax.set(xlabel="Total time / cleared source (s)",ylabel="Empirical cumulative probability",title=f"Q{p}:
1,000 paired worlds",ylim=(0,1))
146             ax.legend(loc="lower right");ax.grid(alpha=.2)
147         save_figure(fig,"research_performance.pdf")
148
149
150 def trajectories():
151     fig,axes=plt.subplots(1,2,figsize=(16/2.54,3.8),layout="constrained")
152     summaries={}
153     for p,ax in zip((3,4),axes):
154         environment=LocalSimulator(20260910,problem=p)
155         planner=Planner(environment,problem=p)
156         result=planner.run();truth_after_exit=environment.summary()
157         assert result["strategy"]=="refined" and result["certificate_complete"]
158         assert result["n_cleared"]==truth_after_exit["n_sources"]
159         route=[(0.,0.)];clears=[]
160         for event in environment.log:
161             request,response=event["request"],event["response"]
162             if response["accepted"] and "position" in request:
163                 point=(request["position"] ["x"],request["position"] ["y"])
164                 route.append(point)
165                 if response.get("clear_result")=="success":clears.append(point)
166         route=np.asarray(route);clears=np.asarray(clears);anchors=np.asarray(planner.waypoints)
167         ax.add_patch(plt.Circle((0,0),1800,fill=False,color="black",lw=1))
168         ax.plot(route[:,0],route[:,1],color="#0072B2",lw=.65,alpha=.75,label="Accepted movement")
169         ax.scatter(anchors[:,0],anchors[:,1],color="gray",s=12,marker="+",label="Discovery points")
170         ax.scatter(clears[:,0],clears[:,1],color="#D55E00",s=18,marker="x",label="Successful clear positions")
171         ax.set(xlim=(-2350,2350),ylim=(-2350,2350),xlabel="East (m)",ylabel="North (m)",title=f"Q{p}: T={result
['virtual_time_s']:.1f} s")
172         ax.set_aspect("equal");ax.set_xticks([-2000,0,2000]);ax.set_yticks([-2000,0,2000]);ax.grid(alpha=.15)
173         evidence={"evidence":"observable_local_synthetic_trajectory_not_official","seed":20260910,
174                 "planner":result,"post_exit_evaluation":truth_after_exit,"observable_log":environment.log}
175         write_json(f"research_trajectory_q{p}.json",evidence)

```

```

176     summaries[f"Q{p}"]={k:result[k] for k in ("n_cleared","virtual_time_s","average_time_s","
certificate_complete")}]
177 handles,labels=axes[0].get_legend_handles_labels()
178 fig.legend(handles,labels,loc="outside lower center",ncol=1,fontsize=8)
179 save_figure(fig,"research_trajectories.pdf")
180 return summaries
181
182
183 def main():
184     data=json.loads((ROOT/"results/research_experiments.json").read_text())
185     development=json.loads((ROOT/"results/refinement_development.json").read_text())
186     assert data["total_runs"]==10700 and development["total_runs"]==2550
187     groups=main_tables(data);auxiliary_tables(data,development)
188     coverage_figure();performance_figure(groups);traces=trajectories()
189     inputs=["results/research_experiments.json","results/refinement_development.json","results/
research_verification.json",
190            "planner.py","geometry.py","coverage.py","environment.py","polar_coverage_candidate.py",
191            "joint_dispatch_candidate.py","make_research_assets.py","research_experiments.py","refinement_study
.py"]
192     write_json("research_metadata.json",{
"evidence":data["evidence"],"official_calls":0,"total_runs":10700,
193            "development_runs":2550,"input_sha256":{name:hashlib.sha256((ROOT/name).read_bytes()).hexdigest
() for name in inputs},
194            "summary":data["summary"],"comparison":data["comparison"],"trajectory_summary":traces})
195     print(json.dumps({"research_runs":10700,"development_runs":2550,"main_groups":len(groups),"
trajectory_summary":traces},indent=2))
196
197
198 if __name__=="__main__":
199     main()

```

A.2 匿名文件清单与结果接口

文件或目录	用途
results/final_experiments.json	4430 次最终合成运行
results/enhancement_experiments.json	3080 次样本外增强运行
results/refinement_development.json	2550 次独立开发运行（非样本外）
results/research_experiments.json	10700 次冻结策略研究运行
results/research_verification.json	新覆盖、光学与行为证书核验
results/research_review.json	本轮最终独立自动审查
results/ring_sweep.json	Q3 环半径开发筛选及样本外配对证据
results/coverage_limits.json	全部 31 点不可删构造
results/refinement_verification.json	收缩环与角区间上界验证
results/geometry_verification.json	几何交叉核验与回归
results/intersection_verification.json	原始交会区域三种状态
results/http_verification.json	幂等重试与计时验证
results/http_deadline_verification.json	慢速响应绝对期限验证
results/second_point_example.json	第二测点候选评分
results/recovery_development.json	恢复策略开发对照

<code>results/visual_review.json</code>	PDF 逐页视觉检查记录
<code>results/enhancement_review.json</code>	增强策略独立审阅记录
<code>results/paper_review.json</code>	非官方自动审阅记录
<code>results/final_review.json</code>	新结论与新世界证据的最终 独立审查
<code>report/generated/</code>	自动数字、表格与轨迹证据
<code>report/figures/</code>	基础、历史增强及新研究矢 量图
<code>report/ai_usage.tex</code>	AI 使用详情源文件
<code>reproduce.txt</code> 、 <code>requirements.txt</code>	命令、环境及依赖
<code>MANIFEST.json</code>	支撑包内文件校验值
<code>results/route_candidate_comparison.json</code>	路由候选负结果
<code>results/adaptive_candidate_comparison.json</code>	自适应测点候选负结果与子 指标
<code>results/terminal_verification.json</code>	虚拟/现实期限终端回归
<code>theory_additions.txt</code> 、 <code>paper_expansion.txt</code>	独立理论草稿
<code>report/generated/coverage_limits_table.tex</code>	不可删性汇总表
<code>polar_coverage_notes.txt</code>	极坐标覆盖推导草稿
<code>joint_dispatch_notes.txt</code>	联合调度推导草稿
<code>optical_policy_notes.txt</code>	光学候选推导草稿
<code>report/research_theory.tex</code>	正文直接引用的新理论片段
<code>report/generated/research_metadata.json</code>	新图表输入哈希与分母记录
<code>contracts.json</code> 、 <code>report_contracts.json</code>	接口与证据口径的描述性快 照

图 表 由`make_assets.py`、`make_enhancement_assets.py` 和`make_research_assets.py` 读取对应 JSON 生成，完整可运行代码如上。理论片段已在正文引用，不重复以代码字体排印中文。`notes` 文件只记录推导，实际结果以结果 JSON 为准。默认 `refined` 与历史 `enhanced` 分开标注；2550 次开发运行不进入 10700 次冻结研究的分母。`audit_pdf.py` 渲染全部页面并检查文字边界；`package.py--verify` 解压支撑包，完整重建五组共 22960 次实验运行并逐项比较，仅排除真实墙钟 `real_duration_s` 及其派生最大值 `max_real_duration_s`，保留其余行为和统计字段；随后重建图表与编译，不以原目录存在文件代替解压可复现性。

含有官方 `robot_id` 的私有 HTTP 日志不进入匿名包；如正式流程要求提交官方加密导出，应保持官方给出的原文件名和原始字节，不在公开包中复制或改写。官方模拟器尚未运行，故本附录不提供官方输出。